

R Course: Beginner to Expert

Sri Ram

2025-11-28

Table of contents

1	Welcome	10
2	About this Course	11
2.1	How to Use	11
3	R Basics	12
4	R as a Calculator	13
5	Objects & Assignment	14
6	object naming rules	15
7	Basic Operations in R	16
8	Comments in R	18
9	Getting Help in R	19
10	Installing and Loading Packages in R	21
10.0.1	Saving and Loading Workspaces in R	21
11	Working Directory	22
12	Vectors (Atomic)	23
13	Exercises	24
14	Data Types & Data Structures	25
15	Vectors	26
16	Lists	40
17	Matrices	52
18	Data Frames	73

19 Factors	82
20 Arrays	84
21 Summary	85
22 Manipulating Vectors, Data Frames, and Lists	86
23 Vectors: Indexing & Vectorized Ops	87
24 Data Frames with dplyr	106
25 Read sas dataset	120
26 keeping selected columns	121
27 dropping columns	122
28 Rename	124
29 Arrange (sorting)	125
30 convert var names to lower case	126
31 Creating new variables	127
32 bind rows (set operator in SAS)	128
33 recode (similar to PROC FORMAT in SAS)	129
34 factor levels	130
35 joins	131
35.1 innter join	132
35.2 left join	132
35.3 right join	133
35.4 Full Join	133
35.5 Semi Join	133
35.6 Anti join	134
36 Reshaping data with tidyr	136
37 counting with n()	137
38 dplyr distinct()	139

39	case_when()	140
40	create seq no.	141
41	Lists: lapply, purrr	142
42	Exercises	143
43	String Functions	144
44	String Functions in Base R	145
44.1	Basic String Functions	145
44.2	Pattern Matching and Replacement	146
44.3	String Splitting and Joining	147
44.4	Regular Expressions	147
44.5	Example Usage	149
45	Overall differences	150
46	Detect matches	153
46.1	str_detect(): Detect the presence or absence of a pattern in a string	153
46.2	str_which(): Find positions matching a pattern	153
46.3	str_count(): Count the number of matches in a string	154
46.4	str_locate(): Locate the position of patterns in a string	154
46.5	str_subset(): Keep strings matching a pattern, or find positions	155
46.6	str_extract(): Extract matching patterns from a string	156
46.7	str_match(): Extract matched groups from a string	157
47	Manage lengths	159
47.1	str_length(): The length of a string	159
47.2	str_pad(): Pad a string	160
47.3	str_trunc(): Truncate a character string	161
47.4	str_trim(): Trim whitespace from a string	161
47.5	str_wrap(): Wrap strings into nicely formatted paragraphs	162
48	Mutate strings	164
48.1	str_replace(): Replace matched patterns in a string	164
48.2	case: Convert case of a string	164
49	Join and split	167
49.1	str_flatten(): Flatten a string	167
49.2	str_dup(): duplicate strings within a character vector	167
49.3	str_split(): Split up a string into pieces	168
49.4	str_glue(): Interpolate strings	169

50 Order strings	171
50.1 <code>str_order()</code> : Order or sort a character vector	171
51 date and time functions	175
51.0.1 Date and Time Base Functions in R	175
51.0.2 Creating Date and Time Objects	175
51.0.3 Year Formats	175
51.0.4 Month Formats	175
51.0.5 Day Formats	176
51.0.6 Weekday Formats	176
51.0.7 Time Formats	176
51.1 1. Date & Time Basics in R	180
51.1.1 1.1 Core classes	180
51.1.2 1.2 Converting numerics to Date (origins matter)	180
51.1.3 1.3 Custom date formats	181
51.1.4 1.4 Datetime strings (ISO 8601 & time zones)	181
51.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic	181
51.2.1 2.1 Intuitive parsers	181
51.2.2 2.2 Accessors and rounding	182
51.2.3 2.3 Timespans	182
51.2.4 3 ADaM-Style Partial Date Imputation (Start & End)	182
51.2.5 Start date (AESTDTC)	182
51.2.6 End date (AEENDTC)	183
51.2.7 Censoring by death / last alive date	183
51.2.8 Imputation flags	183
51.3 4. Durations & Analysis Variables	185
51.3.1 4.1 AE duration (AEDUR) in days	185
52 Using <code>difftime</code> (base) or <code>as.numeric</code> on Date differences	186
52.0.1 6.2 Handy lubridate parsers (subset)	186
53 Reading SAS Datasets (+ Cleaning)	187
53.1 Handling Labels & Missing	189
53.2 Labelled to Factor (if needed)	189
53.3 Common Cleaning	189
54 Exercises	191
55 Base R Functions & Apply Family	192
56 Common Utilities	193
57 Apply Family	194

58 Subsetting Essentials	195
59 Exercises	196
60 Custom Functions & Validation	197
61 Function introduction	198
61.1 Function components	198
61.2 Signalling conditions	199
61.3 Ignoring conditions	200
61.4 tryCatch function	200
61.5 function scope	201
61.6 Creating Custom Functions	202
61.7 Function Arguments	202
61.8 User-defined Functions in R Programming Language	202
62 difference !,!!,!!!	207
63 Error Handling	211
64 Unit Testing with testthat	212
65 Document with roxygen2	213
66 Exercises	214
67 R Package Development	215
67.1 1. Prerequisites	215
67.2 2. Create the Package Skeleton	215
67.2.1 2.1 Create a new package directory	215
67.2.2 2.2 Initialize Git and a README	216
67.2.3 2.3 The minimal structure (what you get)	216
67.3 3. DESCRIPTION: Package Metadata	216
67.4 4. NAMESPACE: Exports and Imports (auto-generated)	217
67.5 5. Write Functions + roxygen2 Headers	217
67.6 6. Generate Documentation	218
67.7 7. Add Data, Vignettes, Tests (Optional but Recommended)	219
67.7.1 7.1 Package data	219
67.7.2 7.2 Vignettes	219
67.7.3 7.3 Tests with testthat	219
67.8 8. Manage Dependencies	220
67.9 9. Check Your Package Thoroughly	220
67.10 10. Build & Install Locally	221
67.11 11. Continuous Integration (optional)	221

67.1212. Package Website (optional)	221
67.1313. Release Checklist	222
67.13.1 Appendix A — Common Files (at a glance)	222
67.13.2 Appendix B — Troubleshooting Snippets	222
68 Git in RStudio (Setup & Auth)	224
68.1 One-Time Setup	224
68.2 Initialize Git for the Current Project	224
68.3 Connect to GitHub	224
68.4 Typical Workflow	224
68.5 Remove Git from a Project (macOS/RStudio)	225
69 Creating ADaM: ADSL from SDTM-like Inputs	226
70 TLFs: Table, Figure, Listing	234
70.1 Create demographic table from ADSL	234
70.2 Create adverse event table from ADAE	240
71 ggplot2 in R with CDISC ADaM Examples	243
72 Introduction	244
73 Creating Simulated ADaM-Style Data	245
73.1 ADSL: Subject-Level Data	245
73.2 ADLB: Laboratory Data (e.g. ALT)	245
73.3 ADTTE: Time-to-Event Data (e.g. PFS)	245
74 Grammar of Graphics with ADaM	247
75 Basic Plots from ADSL (Beginner Level)	248
75.1 Scatterplot: AGE vs BMIBL	248
75.1.1 Mapping vs Setting Aesthetics	248
75.2 Bar Plot: Subject Counts per Arm	250
75.3 Histogram and Density: Age Distribution	252
75.4 Boxplot: BMIBL by ARM	254
76 Longitudinal Plots from ADLB (Intermediate)	257
76.1 Line Plot: Mean ALT Over Time by ARM	257
76.2 Individual Profiles (Spaghetti Plot)	259
77 Faceting with ADaM Data	260
77.1 Example: Facet ALT by Sex	260
78 Scales, Labels, and Themes in ADaM Context	262
78.1 Changing Axis Labels and Titles	262

78.2 Controlling Color Scales	263
78.3 Transforming Scales (e.g., log10 ALT)	264
79 Themes and Publication-Style Graphics	266
79.1 Using Built-In Themes	266
79.2 Custom Theme for Clinical Reports	268
80 Coordinates and Positions	270
80.1 Flipping Coordinates: Boxplot of BMI by Sex	270
80.2 Stacked vs Dodged Bar Plots: TRTA by SEX	271
81 Adding Statistical Layers	273
81.1 Linear Regression: BMIBL vs AGE	273
81.2 Summaries: Mean ALT $\pm$ SD per Visit and TRTA	274
82 Tidy Workflow: dplyr + ggplot2 with ADaM	275
83 Advanced: Simple Kaplan–Meier Plot from ADTTE	277
84 Advanced: Annotations and Secondary Axes	279
84.1 Annotations on an ADaM Plot	279
84.2 Secondary Axis (Use with Care)	280
85 Writing Reusable Plot Functions for ADaM	282
86 Saving Plots for Reports	284
87 Introduction	285
88 Figure 1 – Bar Graph with Individual Lesion Lines	286
88.1 1.1 Create bar-graph data	286
88.2 1.2 Reshape to long format for lines	287
88.3 1.3 Build the bar + line plot	288
89 Figure 2 – Waterfall Plot of Best Percentage Change from Baseline	291
89.1 2.1 Create waterfall data	291
89.2 2.2 Sort patients by change	292
89.3 2.3 Build the waterfall plot	293
90 Figure 3 – Swimmer Plot: Duration of Response	296
90.1 3.1 Create swimmer data	296
90.2 3.2 Build the swimmer plot	297
91 Figure 4 – Spider Plot: Tumor Trajectories Over Time	300
91.1 4.1 Create spider data	300

91.2 4.2 Build the spider plot	301
92 Figure 5 – Kaplan–Meier Survival Curve	303
92.1 5.1 Create KM data	303
92.2 5.2 Fit the KM model and plot	304
93 Figure 6 – Forest Plot for Subgroup Hazard Ratios	306
93.1 6.1 Load example data and prepare	306
93.2 6.2 Define forest theme and build the plot	308
94 Summary and Next Steps	311
95 Capstone: End-to-End Mini Workflow	312
95.1 Parameters	312
95.2 1) Read (or Synthesize) SDTM	312
95.3 2) Derive ADSL (Minimal Demo)	313
95.4 3) TLFs	314
95.5 4) Save Outputs	315
96 Appendix: Tips, Profiles, .libPaths	316
96.1 Useful Profiles	316
96.2 Custom Library Paths	316
96.3 Format vs formatC (quick recap)	316
96.4 POSIXct vs POSIXlt	317
96.5 Recommended Packages	317
96.6 Short Glossary	317

1 Welcome

2 About this Course

This Course from **R beginner** to **expert**, with a practical focus on **clinical programming** (CDISC/ADaM and TLFs).

Each chapter includes step-by-step explanations, runnable code, and short exercises.

2.1 How to Use

1. Install: R (4.2), RStudio (or VS Code).
2. Follow along chapter-by-chapter, running code and completing exercises.
3. Use sample data or your own clinical trial datasets (SAS/CSV). ## Structure (Highlights)
 - **Basics & Data:** R syntax, data types/structures, vectors/data frames/lists.
 - **I/O:** Read SAS datasets (with **haven**), handle labels, and clean raw data.
 - **Programming:** Base functions, write your own functions, validate with tests.
 - **DevOps:** Create an R package, connect Git in RStudio/GitHub.
 - **CDISC:** Build ADaM (ADSL) from SDTM-like inputs.
 - **TLFs:** Produce a baseline Table 1, a KM plot, and a listing.

Tip: If you don't have sample SDTM/ADaM data yet, the chapters generate **small synthetic data** as a fallback so everything runs end-to-end. ## contact For questions or feedback, reach out to **r2sas2025@gmail.com**

3 R Basics

4 R as a Calculator

```
1 + 1
```

```
[1] 2
```

```
3 * (4 + 5)
```

```
[1] 27
```

5 Objects & Assignment

```
x <- 10  
y <- 3.5  
x + y
```

```
[1] 13.5
```

6 object naming rules

- R variable names can contain letters, numbers, periods, and underscores. However, they cannot start with a number or underscore. R is case-sensitive, so `age`, `Age`, and `AGE` would be considered different variables.
- R variable names should be descriptive and meaningful. Avoid using reserved words or function names as variable names.
- A variable can have a short name (like `x` and `y`) or a more descriptive name (`age`, `carname`, `total_volume`). Rules for R variables are:
- A variable name must start with a letter and can be a combination of letters, digits, period(`.`) and underscore(`_`). If it starts with period(`.`), it cannot be followed by a digit.
- A variable name cannot start with a number or underscore (`_`) Variable names are case-sensitive (`age`, `Age` and `AGE` are three different variables) Reserved words cannot be used as variables (`TRUE`, `FALSE`, `NULL`, `if...`)
- Variable names should not contain spaces. Use underscore (`_`) or period (`.`) to separate words in a variable name.
- Variable names should be meaningful and descriptive. Avoid using single-letter variable names except for temporary variables in loops or functions.

7 Basic Operations in R

R supports various basic operations, including: * Arithmetic Operations: Addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (^). Example:

```
a <- 10
b <- 5
sum <- a + b
diff <- a - b
prod <- a * b
quot <- a / b
exp <- a ^ b
sum; diff; prod; quot; exp
```

```
[1] 15
```

```
[1] 5
```

```
[1] 50
```

```
[1] 2
```

```
[1] 1e+05
```

- Comparison Operations: Equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). Example:

```
x <- 10
y <- 5
eq <- x == y
neq <- x != y
gt <- x > y
lt <- x < y
gte <- x >= y
lte <- x <= y
eq; neq; gt; lt; gte; lte
```



```
[1] FALSE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

- Logical Operations: AND (&), OR (|), and NOT (!). Example:

```
p <- TRUE  
q <- FALSE  
and <- p & q  
or <- p | q  
not <- !p  
and; or; not
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

8 Comments in R

Comments in R are created using the `#` symbol. Anything following the `#` on the same line is considered a comment and is ignored by R during execution. Example:

```
# This is a comment
x <- 10 # Assigning value to x
y <- 5  # Assigning value to y
sum <- x + y # Calculating the sum of x and y
sum # Output the sum
```

```
[1] 15
```

9 Getting Help in R

R provides several ways to get help and documentation for functions and packages: *

- `?function_name`: Displays the documentation for a specific function. Example:

```
?mean
```

- `help(function_name)`: Another way to access the documentation for a function. Example:

```
help(mean)
```

- `help.search("keyword")`: Searches for help topics related to a specific keyword. Example:

```
help.search("regression")
```

- `example(function_name)`: Shows examples of how to use a specific function. Example:

```
example(mean)
```

```
mean> x <- c(0:10, 50)
```

```
mean> xm <- mean(x)
```

```
mean> c(xm, mean(x, trim = 0.10))  
[1] 8.75 5.50
```

- `vignette("package_name")`: Opens the vignette (detailed documentation) for a specific package. Example:

```
vignette("dplyr")
```

```
starting httpd help server ... done
```

- `??keyword`: Searches for help topics related to a specific keyword (similar to `help.search`). Example:

```
??regression
```

10 Installing and Loading Packages in R

R has a vast ecosystem of packages that extend its functionality. To use a package, you need to install it first and then load it into your R session. * Installing a Package: Use the `install.packages("package_name")` function to install a package from CRAN. Example:

```
install.packages("ggplot2")
```

- Loading a Package: Use the `library(package_name)` function to load an installed package into your R session. Example:

```
library(ggplot2)
```

```
# Now you can use functions from the ggplot2 package
```

10.0.1 Saving and Loading Workspaces in R

You can save your R workspace (all objects in memory) to a file and load it later * Saving Workspace: Use the `save.image("file_name.RData")` function to save the entire workspace to a file. Example:

```
save.image("my_workspace.RData")
```

- Loading Workspace: Use the `load("file_name.RData")` function to load a saved workspace from a file. Example:

```
load("my_workspace.RData")
```

11 Working Directory

```
getwd()
```

```
[1] "/home/runner/work/r4sas/r4sas"
```

```
# setwd("/path/you/want") # avoid in reproducible code; prefer here::here() for projects
```

12 Vectors (Atomic)

```
nums <- c(1, 2, 3, 4)
chars <- c("a", "b", "c")
logical <- c(TRUE, FALSE, TRUE)
typeof(nums); typeof(chars); typeof(logical)
```

```
[1] "double"
```

```
[1] "character"
```

```
[1] "logical"
```

13 Exercises

1. Create an object `z` that stores $(5^2 + 7)/3$.
2. Use `?seq` and create a sequence from 0 to 1 by 0.1.
3. Inspect `typeof()` for a few objects you create.

14 Data Types & Data Structures

R has several built-in data structures to store and manipulate different types of data. These include vectors, lists, matrices, data frames, and factors. Below is an overview of each structure along with code examples.

15 Vectors

Vectors are the simplest data structure in R. They store elements of the same type (numeric, character, logical, etc.).

```
# Creating numeric and character vectors
numeric_vector <- c(1, 2, 3, 4)
char_vector1 <- c("apple", "banana", "cherry")
char_vector2 <- c(2, 3, 4, 5, "a")
logical_vector <- c(TRUE, FALSE, TRUE)

# Accessing elements
numeric_vector[1] # Access the first element
```

```
[1] 1
```

```
v_logical <- c(T,F,T) # logical vector
v_logical
```

```
[1] TRUE FALSE TRUE
```

```
is.vector(v_logical)
```

```
[1] TRUE
```

```
is.atomic(v_logical)
```

```
[1] TRUE
```

```
typeof(v_logical)
```

```
[1] "logical"
```

```
v_integer <- c(1L,2L,5L) # integer vector  
v_integer
```

```
[1] 1 2 5
```

```
is.vector(v_integer)
```

```
[1] TRUE
```

```
is.atomic(v_integer)
```

```
[1] TRUE
```

```
typeof(v_integer)
```

```
[1] "integer"
```

```
v_double <- c(1.3,2.1,5.0) # double vector  
v_double
```

```
[1] 1.3 2.1 5.0
```

```
is.vector(v_double)
```

```
[1] TRUE
```

```
is.atomic(v_double)
```

```
[1] TRUE
```

```
typeof(v_double)
```

```
[1] "double"
```

```
v_character <- c("a", "b", "c") # character vector
v_character
```

```
[1] "a" "b" "c"
```

```
is.vector(v_character)
```

```
[1] TRUE
```

```
is.atomic(v_character)
```

```
[1] TRUE
```

```
typeof(v_character)
```

```
[1] "character"
```

```
v_NULL <- NULL # NULL
v_NULL
```

```
NULL
```

```
typeof(v_NULL)
```

```
[1] "NULL"
```

```
# Mix type vector (type coercion or conversion)
v_mix <- c(T, 1L, 1.25, "a")
v_mix # all elements converted to characters (based on hierarchy)
```

```
[1] "TRUE" "1"    "1.25" "a"
```

```
is.vector(v_mix)
```

```
[1] TRUE
```

```
typeof(v_mix)
```

```
[1] "character"
```

```
# Vector properties
```

```
v <- c(1,2,3,4,5)
```

```
# vector length
```

```
length(v)
```

```
[1] 5
```

```
# type
```

```
typeof(v)
```

```
[1] "double"
```

```
class(v)
```

```
[1] "numeric"
```

```
# naming elements
```

```
names(v) # without names
```

```
NULL
```

```
vnames <- c("first", "second", "third", "fourth", "fifth") # element names
```

```
names(v) <- vnames # naming elements
```

```
v
```

```
first second  third fourth  fifth
      1      2      3      4      5
```

```
names(v) # new names
```

```
[1] "first" "second" "third" "fourth" "fifth"
```

```
# Create vector, access elements, modify vector
```

```
# create using c()
```

```
v <- c(1,3,5,8,0)
```

```
# create using operator :
```

```
1:100
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
10:-10
```

```
[1] 10 9 8 7 6 5 4 3 2 1 0 -1 -2 -3 -4 -5 -6 -
7 -8
[20] -9 -10
```

```
# using sequence seq()
```

```
v <- seq(from = 1, to = 100, by = 1)
```

```
v
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
v <- seq(from = 0, to = 1, by = 0.01)
```

```
v
```

```
[1] 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
[16] 0.15 0.16 0.17 0.18 0.19 0.20 0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29
[31] 0.30 0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.40 0.41 0.42 0.43 0.44
[46] 0.45 0.46 0.47 0.48 0.49 0.50 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59
[61] 0.60 0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.70 0.71 0.72 0.73 0.74
[76] 0.75 0.76 0.77 0.78 0.79 0.80 0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89
[91] 0.90 0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.00
```

```
v <- seq(from = 0, to = 10, length.out = 5)
v
```

```
[1] 0.0 2.5 5.0 7.5 10.0
```

```
# let's create a vector for accessing vector elements
v <- 1:10
names(v) <- c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")
v
```

```
 a  b  c  d  e  f  g  h  i  j
1  2  3  4  5  6  7  8  9 10
```

```
# access vector elements using integer vector index
v[c(1,5,10)]
```

```
 a  e  j
1  5 10
```

```
v[1:5] # range index selection (slicing)
```

```
 a b c d e
1 2 3 4 5
```

```
v[seq(from = 1, to = 9, by = 2)]
```

```
 a c e g i
1 3 5 7 9
```

```
v[10:1] # reverse order selection
```

```
 j  i  h  g  f  e  d  c  b  a
10 9  8  7  6  5  4  3  2  1
```

```
v[c(10,1,5,3)] # mix order selection
```

```
 j  a  e  c
10 1  5  3
```

```
# access vector elements using logical vector index
v[c(T,F,F,F,F,F,F,F,F)] # access first element
```

```
a
1
```

```
v[c(F,F,F,F,F,F,F,T,T,T)] # access last three elements
```

```
h i j
8 9 10
```

```
# access elements using names
v[c("a","c","e")]
```

```
a c e
1 3 5
```

```
v[c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j")]
```

```
a b c d e f g h i j
1 2 3 4 5 6 7 8 9 10
```

```
# modify vector elements
v
```

```
a b c d e f g h i j
1 2 3 4 5 6 7 8 9 10
```

```
v[2] <- 20 # alter second element
v
```

```
a b c d e f g h i j
1 20 3 4 5 6 7 8 9 10
```

```
v[c(1,5,10)] <- c(0,0,0) # alter multiple elements
v
```

```
a b c d e f g h i j
0 20 3 4 0 6 7 8 9 0
```



```
# modify elements with value 0
v
```

a	b	c	d	e	f	g	h	i	j
0	20	3	4	0	6	7	8	9	0

```
v[v==0] # filter with condition
```

a	e	j
0	0	0

```
v[v==0] <- 1000
v
```

a	b	c	d	e	f	g	h	i	j
1000	20	3	4	1000	6	7	8	9	1000

```
# truncate vector to first 3 elements
v <- v[1:3]
v
```

a	b	c
1000	20	3

```
# transpose vector change row to column vector or vice versa
v
```

a	b	c
1000	20	3

```
t(v)
```

	a	b	c
[1,]	1000	20	3

```
# delete or remove a vector
v <- NULL
v
```

NULL

```
rm(v)

# combine 2 different vectors
v1 <- 1:3
v2 <- 100:105
v1
```

```
# combine 2 different vectors
v1 <- 1:3
v2 <- 100:105
v1
```

```
v1 <- 1:3
v2 <- 100:105
v1
```

```
v2 <- 100:105  
v1
```

[1] 1 2 3

v2

```
[1] 100 101 102 103 104 105
```

```
v3 <- c(v1,v2) # combine vectors
v3
```

```
[1] 1 2 3 100 101 102 103 104 105
```

```
# repet elements of a vector
rep(x = v1, times = 2)
```

```
rep(x = v1, times = 2)
```

[1] 1 2 3 1 2 3

```
rep(x = v1, times = 100)
```

[illegible]

```
rep(10,10)
```

```
[1] 10 10 10 10 10 10 10 10 10 10
```

```
# Vector arithmetics
```

```
# vector - scalar (scalar with each vector element)
```

```
v <- 1:5
```

```
a <- 10
```

```
v
```

```
[1] 1 2 3 4 5
```

```
a
```

```
[1] 10
```

```
# Addition +
```

```
v + a
```

```
[1] 11 12 13 14 15
```

```
# Subtraction -
```

```
v - a
```

```
[1] -9 -8 -7 -6 -5
```

```
# Multiplication *
```

```
v * a
```

```
[1] 10 20 30 40 50
```

```
# Division /
```

```
v / a
```

```
[1] 0.1 0.2 0.3 0.4 0.5
```

```
# Exponent ^ **
```

```
v^a
```

```
[1]      1    1024  59049 1048576 9765625
```

```
# Modulus (Remainder from division) %%  
v %% 2
```

```
[1] 1 0 1 0 1
```

```
# Integer Division %/%  
v %/% 2
```

```
[1] 0 1 1 2 2
```

```
# Other functions on vector elements  
sqrt(v)
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

```
log(v)
```

```
[1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379
```

```
sum(v)
```

```
[1] 15
```

```
# vector - vector (vector element to element | member-by-member)  
v1 <- seq(10,30,10)  
v2 <- rep(3,3)  
  
# Addition +  
v1 + v2
```

```
[1] 13 23 33
```

```
# Subtraction -  
v1 - v2
```

```
[1] 7 17 27
```

```
# Multiplication *  
v1 * v2
```

```
[1] 30 60 90
```

```
# Division /  
v1 / v2
```

```
[1] 3.333333 6.666667 10.000000
```

```
# Exponent ^ **  
v1^v2
```

```
[1] 1000 8000 27000
```

```
# Modulus (Remainder from division) %%  
v1 %% v2
```

```
[1] 1 2 0
```

```
# Integer Division %/%  
v1 %/% v2
```

```
[1] 3 6 10
```

```
# Vector-matrix style multiplication  
v1
```

```
[1] 10 20 30
```

```
v2
```

```
[1] 3 3 3
```

```
10*3 + 20*3 + 30*3
```

```
[1] 180
```

```
t(v1) %*% v2
```

```
      [,1]  
[1,] 180
```

```
v1 %*% v2
```

```
      [,1]  
[1,] 180
```

```
v1 %*% t(v2)
```

```
      [,1] [,2] [,3]  
[1,]    30    30    30  
[2,]    60    60    60  
[3,]    90    90    90
```

```
# Recycling rule  
v1 <- c(1,1,1)  
v2 <- 1:6  
v1
```

```
[1] 1 1 1
```

```
v2
```

```
[1] 1 2 3 4 5 6
```

```
v1 + v2
```

```
[1] 2 3 4 5 6 7
```

```
# Set operations  
v1 <- c("a", "b", "c")  
v2 <- c("c", "d", "e")  
v1
```

```
[1] "a" "b" "c"
```

```
v2
```

```
[1] "c" "d" "e"
```

```
union(v1,v2) # union of both sets (all unique elements)
```

```
[1] "a" "b" "c" "d" "e"
```

```
intersect(v1,v2) # intersection of both sets (elements in both sets)
```

```
[1] "c"
```

```
setdiff(v1,v2) # difference of elements (elements in v1 and not in v2)
```

```
[1] "a" "b"
```

```
identical(v1, v2) # check if vectors are identical
```

```
[1] FALSE
```

```
identical(c(1,2,3), c(1,2,3))
```

```
[1] TRUE
```

16 Lists

A list can contain elements of different types, including other lists or vectors or data structures.

```
# Creating a list
my_list <- list(name = "John", age = 25, scores = c(90, 85, 88))

# Accessing elements by name
my_list$name # Output: "John"
```

```
[1] "John"
```

```
# Create a list (and name elements)

# lets create some variables (different types)
a <- 10
b <- 2L
c <- TRUE
d <- "word"
v <- 1:10
names(v) <- paste("i", v, sep = "")
M <- matrix(data = seq(10,40,by = 10), nrow = 2, dimnames = list(c("r1", "r2"), c("c1", "c2")))
A <- array(data = 1:8, dim = c(2,2,2), dimnames = list(c("r1", "r2"), c("c1", "c2"), c("M1", "M2", "M3")))

# create list and include all variables (elements)
lst <- list(a, b, c, d, v, M, A)
lst
```

```
[[1]]
[1] 10
```

```
[[2]]
[1] 2
```

```
[[3]]
[1] TRUE
```



```
[[4]]
[1] "word"
```

```
[[5]]
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
[[6]]
      c1 c2
r1 10 30
r2 20 40
```

```
[[7]]
, , M1
```

```
      c1 c2
r1  1  3
r2  2  4
```

```
, , M2
```

```
      c1 c2
r1  5  7
r2  6  8
```

```
str(lst) # check list structure
```

```
List of 7
 $ : num 10
 $ : int 2
 $ : logi TRUE
 $ : chr "word"
 $ : Named int [1:10] 1 2 3 4 5 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 $ : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
 ..- attr(*, "dimnames")=List of 3
 .. ..$ : chr [1:2] "r1" "r2"
```

```
.. ..$ : chr [1:2] "c1" "c2"  
.. ..$ : chr [1:2] "M1" "M2"
```

```
typeof(lst) # check type
```

```
[1] "list"
```

```
class(lst) # check class
```

```
[1] "list"
```

```
is.list(lst) # check if object is list
```

```
[1] TRUE
```

```
# name each list member  
names(lst) <- c("a", "b", "c", "d", "v", "M", "A")  
lst
```

```
$a  
[1] 10
```

```
$b  
[1] 2
```

```
$c  
[1] TRUE
```

```
$d  
[1] "word"
```

```
$v  
 i1 i2 i3 i4 i5 i6 i7 i8 i9 i10  
  1  2  3  4  5  6  7  8  9  10
```

```
$M  
   c1 c2  
r1 10 30  
r2 20 40
```

```
$A  
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```
      c1 c2  
r1  5  7  
r2  6  8
```

```
# alternative: define names as tags when list is created  
list(a=a, b=b, c=c, d=d, v=v, M=M, A=A)
```

```
$a  
[1] 10
```

```
$b  
[1] 2
```

```
$c  
[1] TRUE
```

```
$d  
[1] "word"
```

```
$v  
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
  1   2   3   4   5   6   7   8   9  10
```

```
$M  
      c1 c2  
r1 10 30  
r2 20 40
```

```
$A  
, , M1
```

```
      c1 c2
```

```
r1  1  3
r2  2  4
```

```
, , M2
```

```
      c1 c2
r1    5  7
r2    6  8
```

```
# Access list elements
```

```
# single square bracket [] (return a list)
```

```
lst1 <-lst[1] # access first list elements (return a list)
```

```
str(lst1)
```

```
List of 1
```

```
$ a: num 10
```

```
class(lst1)
```

```
[1] "list"
```

```
lst123 <-lst[c(1,2,3)] # access first three elements with index vector (return a list)
lst123
```

```
$a
```

```
[1] 10
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
class(lst123)
```

```
[1] "list"
```

```
# double square brackets [[]] (return original member)
ele <-lst[[5]] # extract 5th member-element (returns original element)
ele
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10
```

```
is.vector(ele)
```

```
[1] TRUE
```

```
# use $ operator - extract by member name (return original member)
ele <- lst$M
ele
```

```
      c1 c2
r1 10 30
r2 20 40
```

```
class(ele)
```

```
[1] "matrix" "array"
```

```
# Modify list

# remove element from a list
lst
```

```
$a
[1] 10
```

```
$b
[1] 2
```

```
$c
[1] TRUE
```

```
$d
[1] "word"
```

```
$v
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
$M
    c1 c2
r1 10 30
r2 20 40
```

```
$A
, , M1
```

```
    c1 c2
r1  1  3
r2  2  4
```

```
, , M2
```

```
    c1 c2
r1  5  7
r2  6  8
```

```
lst[1] <- NULL # remove first member
lst
```

```
$b
[1] 2
```

```
$c
[1] TRUE
```

```
$d
[1] "word"
```

```
$v
  i1 i2 i3 i4 i5 i6 i7 i8 i9 i10
  1  2  3  4  5  6  7  8  9  10
```

```
$M
    c1 c2
r1 10 30
```

```
r2 20 40
```

```
$A
```

```
, , M1
```

```
      c1 c2  
r1    1  3  
r2    2  4
```

```
, , M2
```

```
      c1 c2  
r1    5  7  
r2    6  8
```

```
# add element to a list (at the end)  
length(lst)
```

```
[1] 6
```

```
lst[7] <- 1000  
lst
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
$d
```

```
[1] "word"
```

```
$v
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
   1   2   3   4   5   6   7   8   9  10
```

```
$M
```

```
      c1 c2  
r1    10 30  
r2    20 40
```

```
$A
```

```
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```
      c1 c2  
r1  5  7  
r2  6  8
```

```
[[7]]  
[1] 1000
```

```
# update value of a member in alist  
lst[[7]] <- 500  
lst[7]
```

```
[[1]]  
[1] 500
```

```
# update value within a vector (on a list)  
lst[[4]][5] <- 5000  
lst[[4]]
```

```
  i1  i2  i3  i4  i5  i6  i7  i8  i9  i10  
   1   2   3   4 5000   6   7   8   9  10
```

```
# convert list to a vector  
vec <- unlist(lst)  
vec
```

```
      b      c      d  v.i1  v.i2  v.i3  v.i4  v.i5  v.i6  v.i7  v.i8  
"2" "TRUE" "word"  "1"   "2"   "3"   "4" "5000"  "6"   "7"   "8"  
v.i9 v.i10      M1      M2      M3      M4      A1      A2      A3      A4      A5  
"9"  "10"  "10"  "20"  "30"  "40"  "1"   "2"   "3"   "4"   "5"  
A6   A7   A8  
"6"  "7"  "8" "500"
```



```
is.vector(vec)
```

```
[1] TRUE
```

```
# Merging lists & nested lists

# create another list
lst1 <- list(e11 = c(1,5,10), e12 = TRUE)

# merge both lists
lst_merged <- c(lst, lst1)
lst_merged
```

```
$b
```

```
[1] 2
```

```
$c
```

```
[1] TRUE
```

```
$d
```

```
[1] "word"
```

```
$v
```

i1	i2	i3	i4	i5	i6	i7	i8	i9	i10
1	2	3	4	5000	6	7	8	9	10

```
$M
```

	c1	c2
r1	10	30
r2	20	40

```
$A
```

```
, , M1
```

	c1	c2
r1	1	3
r2	2	4

```
, , M2
```

	c1	c2
--	----	----

```
r1 5 7
r2 6 8
```

```
[[7]]
[1] 500
```

```
$e11
[1] 1 5 10
```

```
$e12
[1] TRUE
```

```
str(lst_merged)
```

```
List of 9
 $ b : int 2
 $ c : logi TRUE
 $ d : chr "word"
 $ v : Named num [1:10] 1 2 3 4 5000 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ M : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 $ A : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
 ..- attr(*, "dimnames")=List of 3
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 .. ..$ : chr [1:2] "M1" "M2"
 $ : num 500
 $ e11: num [1:3] 1 5 10
 $ e12: logi TRUE
```

```
names(lst_merged)
```

```
[1] "b" "c" "d" "v" "M" "A" "" "e11" "e12"
```

```
# nested list (recursive procedure)
list3 <- list(1, c(T,F,F)) # list sub-level 3
```

```
list2 <- list(list3) # list sub-level 2
list1 <- list(list2) # list sub-level 1

str(list1)
```

```
List of 1
 $ :List of 1
  ..$ :List of 2
   .. ..$ : num 1
   .. ..$ : logi [1:3] TRUE FALSE FALSE
```

```
# extract list level 2
list1[[1]]
```

```
[[1]]
[[1]][[1]]
[1] 1
```

```
[[1]][[2]]
[1] TRUE FALSE FALSE
```

```
# extract list level 3
list1[[1]][[1]]
```

```
[[1]]
[1] 1
```

```
[[2]]
[1] TRUE FALSE FALSE
```

```
# extract 1st member from list level 3
list1[[1]][[1]][[1]]
```

```
[1] 1
```

```
# extract 2nd member from list level 3
list1[[1]][[1]][[2]]
```

```
[1] TRUE FALSE FALSE
```

17 Matrices

A matrix is a two-dimensional structure that contains elements of the same type (numeric, character, or logical).

```
# Creating a 3x3 numeric matrix
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)

# Accessing elements
my_matrix[1, 2] # Access the element in row 1, column 2
```

```
[1] 4
```

```
# using matrix()
M <- matrix(data = 1:9, nrow = 3, ncol = 3)
M
```

```
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
M <- matrix(data = 1:9, nrow = 3, ncol = 3, byrow = T)
M
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
matrix(data = 1:6, nrow = 2, ncol = 3)
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
# by merging multiple vectors
v1 <- c(1,2,3)
v2 <- c(4,5,6)
v3 <- c(7,8,9)

rbind(v1,v2,v3)
```

```
      [,1] [,2] [,3]
v1      1    2    3
v2      4    5    6
v3      7    8    9
```

```
cbind(v1,v2,v3)
```

```
      v1 v2 v3
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
```

```
# by altering vector dimension
v <- 1:9
v
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
dim(v) <- c(3,3)
v
```

```
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
# Matrix properties
```

```
# rownames & colnames
```

```
M <- matrix(1:12, nrow = 4, dimnames = list(c("r1","r2","r3", "r4"), c("c1","c2","c3")))
M
```

```

      c1 c2 c3
r1  1  5  9
r2  2  6 10
r3  3  7 11
r4  4  8 12

```

```
rownames(M)
```

```
[1] "r1" "r2" "r3" "r4"
```

```
colnames(M)
```

```
[1] "c1" "c2" "c3"
```

```
# matrix dimension
dim(M)
```

```
[1] 4 3
```

```
# get all attributes
attributes(M)
```

```
$dim
[1] 4 3
```

```
$dimnames
$dimnames[[1]]
[1] "r1" "r2" "r3" "r4"
```

```
$dimnames[[2]]
[1] "c1" "c2" "c3"
```

```
# change rownames & colnames
rownames(M) <- paste("row ", 1:4, sep = "")
colnames(M) <- paste("col ", 1:3, sep = "")
attributes(M)
```

```

$dim
[1] 4 3

$dimnames
$dimnames[[1]]
[1] "row 1" "row 2" "row 3" "row 4"

$dimnames[[2]]
[1] "col 1" "col 2" "col 3"

```

```
M
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```

# class and type
class(M)

```

```
[1] "matrix" "array"
```

```
typeof(M)
```

```
[1] "integer"
```

```

# check for matrix
is.matrix(M)

```

```
[1] TRUE
```

```

# Access matrix elements

# integer vector as index
M

```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
M[2,3]
```

```
[1] 10
```

```
M[c(1,2),3]
```

row 1	row 2
9	10

```
M[c(2,3),] # selected rows and all columns
```

	col 1	col 2	col 3
row 2	2	6	10
row 3	3	7	11

```
M[,c(2,3)] # selected columns and all rows
```

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11
row 4	8	12

```
# logical vector as index
M[c(T,T,F,F), c(T,T,T)]
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10


```
# character vector as index
M[c("row 2", "row 3"), c("col 1", "col 2")]
```

	col 1	col 2
row 2	2	6
row 3	3	7

```
# range of indexes (slicing rows and columns)
M[1:3,2:3]
```

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11

```
# Access matrix elements

# modify 1 element
M
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
M[1,1] <- 10
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
# modify more than one element
M[2:3,3] <- 20
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	20
row 3	3	7	20
row 4	4	8	12

```
# modify elements based on condition
M[M>10] <- 0
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

```
# transpose a matrix
t(M)
```

	row 1	row 2	row 3	row 4
col 1	10	2	3	4
col 2	5	6	7	8
col 3	9	0	0	0

```
# add row to matrix
M
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

```
rbind(M, c(0,0,0))
```

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0
	0	0	0

```
# add column to matrix
cbind(M, c(0,0,0, 0))
```

```
      col 1 col 2 col 3
row 1    10     5     9 0
row 2     2     6     0 0
row 3     3     7     0 0
row 4     4     8     0 0
```

```
# alter matrix dimensions
dim(M)
```

```
[1] 4 3
```

```
dim(M) <- c(3,4) # names are dropped
M
```

```
      [,1] [,2] [,3] [,4]
[1,]    10     4     7     0
[2,]     2     5     8     0
[3,]     3     6     9     0
```

```
# merge 2 matrices
M1 <- matrix(data = rep(0,4), nrow = 2, ncol = 2)
M2 <- matrix(data = rep(1,4), nrow = 2, ncol = 2)
M1
```

```
      [,1] [,2]
[1,]     0     0
[2,]     0     0
```

```
M2
```

```
      [,1] [,2]
[1,]     1     1
[2,]     1     1
```

```
rbind(M1,M2)
```

```
      [,1] [,2]  
[1,]    0    0  
[2,]    0    0  
[3,]    1    1  
[4,]    1    1
```

```
cbind(M1,M2)
```

```
      [,1] [,2] [,3] [,4]  
[1,]    0    0    1    1  
[2,]    0    0    1    1
```

```
# Matrix arithmetics
```

```
# matrix - scalar (scalar with each vector element)
```

```
M
```

```
      [,1] [,2] [,3] [,4]  
[1,]   10    4    7    0  
[2,]    2    5    8    0  
[3,]    3    6    9    0
```

```
a <- 10
```

```
# Addition +
```

```
M + a
```

```
      [,1] [,2] [,3] [,4]  
[1,]   20   14   17   10  
[2,]   12   15   18   10  
[3,]   13   16   19   10
```

```
# Subtraction -
```

```
M - a
```

```
      [,1] [,2] [,3] [,4]  
[1,]    0   -6   -3  -10  
[2,]   -8   -5   -2  -10  
[3,]   -7   -4   -1  -10
```

```
# Multiplication *
```

```
M * a
```

```
      [,1] [,2] [,3] [,4]
[1,]  100   40   70    0
[2,]   20   50   80    0
[3,]   30   60   90    0
```

```
# Division /
```

```
M / a
```

```
      [,1] [,2] [,3] [,4]
[1,]   1.0  0.4  0.7    0
[2,]   0.2  0.5  0.8    0
[3,]   0.3  0.6  0.9    0
```

```
# Exponent ^ **
```

```
M^a
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 1.0000e+10  1048576  282475249    0
[2,] 1.0240e+03   9765625 1073741824    0
[3,] 5.9049e+04  60466176 3486784401    0
```

```
# Modulus (Remainder from division) %%
```

```
M %% 2
```

```
      [,1] [,2] [,3] [,4]
[1,]    0    0    1    0
[2,]    0    1    0    0
[3,]    1    0    1    0
```

```
# Integer Division %/%
```

```
M %/% 2
```

```
      [,1] [,2] [,3] [,4]
[1,]    5    2    3    0
[2,]    1    2    4    0
[3,]    1    3    4    0
```

```
# Other functions on matrix elements
sqrt(M)
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 3.162278 2.000000 2.645751    0
[2,] 1.414214 2.236068 2.828427    0
[3,] 1.732051 2.449490 3.000000    0
```

```
log(M)
```

```
      [,1]      [,2]      [,3] [,4]
[1,] 2.3025851 1.386294 1.945910 -Inf
[2,] 0.6931472 1.609438 2.079442 -Inf
[3,] 1.0986123 1.791759 2.197225 -Inf
```

```
sum(M)
```

```
[1] 54
```

```
# matrix - vector (matrix element to element | member-by-member)
M1 <- matrix(data = 1:9, nrow = 3, byrow = T)
M2 <- matrix(data = rep(3,9), nrow = 3)

# Addition +
M1 + M2
```

```
      [,1] [,2] [,3]
[1,]    4    5    6
[2,]    7    8    9
[3,]   10   11   12
```

```
# Subtraction -
M1 - M2
```

```
      [,1] [,2] [,3]
[1,]   -2   -1    0
[2,]    1    2    3
[3,]    4    5    6
```

```
# Multiplication *
M1 * M2
```

```
      [,1] [,2] [,3]
[1,]    3    6    9
[2,]   12   15   18
[3,]   21   24   27
```

```
# Division /
M1 / M2
```

```
      [,1]      [,2] [,3]
[1,] 0.3333333 0.6666667 1
[2,] 1.3333333 1.6666667 2
[3,] 2.3333333 2.6666667 3
```

```
# Exponent ^ **
M1^M2
```

```
      [,1] [,2] [,3]
[1,]    1    8   27
[2,]   64  125  216
[3,]  343  512  729
```

```
# Modulus (Remainder from division) %%
M1 %% M2
```

```
      [,1] [,2] [,3]
[1,]    1    2    0
[2,]    1    2    0
[3,]    1    2    0
```

```
# Integer Division %/%
M1 %/% M2
```

```
      [,1] [,2] [,3]
[1,]    0    0    1
[2,]    1    1    2
[3,]    2    2    3
```

```
# matrix-matrix style multiplication
M1
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
M2
```

```
      [,1] [,2] [,3]
[1,]    3    3    3
[2,]    3    3    3
[3,]    3    3    3
```

```
t(M1) %*% M2
```

```
      [,1] [,2] [,3]
[1,]   36   36   36
[2,]   45   45   45
[3,]   54   54   54
```

```
M1 %*% M2
```

```
      [,1] [,2] [,3]
[1,]   18   18   18
[2,]   45   45   45
[3,]   72   72   72
```

```
# matrix algebra (matrix based functions)
M <- matrix(data = c(1,5,3,2,4,7,4,6,2), nrow = 3, byrow = T)

# get diagonal elements
diag(M)
```

```
[1] 1 4 2
```



```
# get matrix determinant
det(M)
```

```
[1] 74
```

```
# get inverse of a matrix M(-1)
solve(M)
```

```
      [,1]      [,2]      [,3]
[1,] -0.45945946  0.1081081  0.31081081
[2,]  0.32432432 -0.1351351 -0.01351351
[3,] -0.05405405  0.1891892 -0.08108108
```

```
# get eigen values
eigen(M)
```

```
eigen() decomposition
```

```
$values
```

```
[1] 11.778446+0.0000000i -2.389223+0.7578106i -2.389223-0.7578106i
```

```
$vectors
```

```
      [,1]      [,2]      [,3]
[1,] 0.4687233+0i  0.5211486+0.2411697i  0.5211486-0.2411697i
[2,] 0.6544420+0i -0.6642393+0.0000000i -0.6642393+0.0000000i
[3,] 0.5932993+0i  0.4573822-0.1408153i  0.4573822+0.1408153i
```

```
# calculate sum over rows or columns
M
```

```
      [,1] [,2] [,3]
[1,]    1    5    3
[2,]    2    4    7
[3,]    4    6    2
```

```
rowSums(M)
```

```
[1]  9 13 12
```

```
colSums(M)
```

```
[1] 7 15 12
```

```
# Lets solve simple matrix equation
# A * X = B
A <- matrix(data = c(1,2,4,5), nrow = 2, byrow = T)
B <- matrix(data = c(5,24,17,66), nrow = 2, byrow = T)
# X = A(-1) * B
X <- solve(A) %*% B
X
```

```
      [,1] [,2]
[1,]     3     4
[2,]     1    10
```

```
# test
A %*% X # should get B
```

```
      [,1] [,2]
[1,]     5    24
[2,]    17    66
```

```
# summarizing a matrix (apply)
M
```

```
      [,1] [,2] [,3]
[1,]     1     5     3
[2,]     2     4     7
[3,]     4     6     2
```

```
# sum of rows
apply(X = M, MARGIN = 1, FUN = sum)
```

```
[1] 9 13 12
```

```
# sum of columns
apply(X = M, MARGIN = 2, FUN = sum)
```

[1] 7 15 12

```
# create matrix of random numbers  
rnorm(n = 1000, mean = 0, sd = 2)
```

```
[1] 1.5876808494 -0.0154864293 -0.3785887901 1.3911768782 4.1681484099  
[6] 2.1634480456 -0.4083167893 -1.1077693263 -1.6917571836 1.0279706971  
[11] -0.7850051044 2.5369075027 -3.8265786001 -0.3194760068 1.6135748223  
[16] 0.1815621207 2.0072129534 -1.4060395373 0.6201674294 1.2125543757  
[21] -3.4810488921 0.7016837876 4.1075696459 0.8832516634 -1.6608491119  
[26] -0.5713415619 -0.0155289298 -2.5309632453 -1.4002226553 0.9992850841  
[31] -0.6116984885 1.4054111208 -3.0892106488 0.4265098711 0.0939083459  
[36] 1.6708956862 0.0464725599 -1.1039214970 -2.1043520727 1.7794034211  
[41] 0.1725381788 0.5883361214 1.5713653916 0.6356020955 -1.2346920922  
[46] -0.8303930978 2.3005420996 -1.9445609396 0.1170629393 1.1559567433  
[51] 1.6980241419 -0.8913793567 2.1703267146 -1.2434836142 -2.0508706759  
[56] -3.4199776342 2.3115216589 0.2574104909 0.4812095722 0.7464547916  
[61] -2.0584004543 1.6300534845 0.7279611892 1.7173942532 -0.0690115765  
[66] -0.7239149522 1.3780207116 3.4591948598 -2.8379798575 0.1470471212  
[71] -2.1061825829 4.5308094515 0.2564248410 0.4143266883 1.6443265838  
[76] 0.0165626891 1.4344860630 -2.6087162318 0.0612705405 -1.1299313555  
[81] 3.2833048148 0.9586024965 1.6033368345 2.1132131750 1.5820028103  
[86] 1.9082503643 2.8778334672 -1.8565122470 -5.2455824413 1.4084257440  
[91] -3.8722044060 0.5781528137 0.0003966981 0.1096881712 -0.3451848910  
[96] 1.8011269192 -1.8088933616 -1.3674138934 1.9242705556 2.9255904670  
[101] -3.4263462102 -3.0977106450 -0.6509320917 0.6291197560 -0.6636858379  
[106] 0.7861418700 1.4708715845 0.8665388694 2.2360800738 2.5596655780  
[111] -1.1686896705 -1.4792168617 1.1540664111 -0.5302597592 0.1518292113  
[116] -0.0403113139 1.5081132532 -0.0831488160 -1.3722728320 -0.3397238852  
[121] 1.6242334323 -0.4870875781 -3.0025380441 -2.2090389950 0.9951326847  
[126] 1.6970574544 1.4650620982 1.1553516781 -0.6318750026 0.2155930441  
[131] -1.8620489329 2.2344833795 -2.0209002874 -2.1382734941 -0.4586017097  
[136] -2.5202573319 -1.1844137573 -2.3675825375 1.1822260526 0.7954654288  
[141] -2.3264467965 -0.1400582210 1.3280506890 1.4630019965 2.6464867860  
[146] -1.8927053170 0.4045717944 -1.4061037987 0.2553947939 0.0016566652  
[151] 1.3449282445 2.0070072300 -2.5604441081 -2.1316741973 1.7180688814  
[156] 0.2172204653 -0.2067530697 3.8660386253 3.3750839596 -2.9733233586  
[161] -2.6809585640 0.8658874888 -1.3947910831 2.6756700306 1.2006203806  
[166] 1.8144316262 0.8316755200 0.4549608626 -1.8911186760 -0.4296420781  
[171] -3.0292680431 1.7245374306 -0.5610886722 1.7473215836 -3.4733081034  
[176] -1.9614446736 -0.7702803405 -0.6657332739 1.6277540423 3.2214002880  
[181] -3.0526331370 -0.0072863114 0.5435633538 3.6754547393 3.0041466698
```

[186]	0.6211349557	0.7668427487	5.2379579586	-0.6347327722	0.6621563829
[191]	2.0181792795	-1.2110870306	0.1506799358	-0.4703309217	4.3079500389
[196]	4.5281355889	0.5505727600	-1.6495658544	-0.0834590584	-1.0827560292
[201]	-0.7046622832	-2.5904773225	-0.0297015600	-0.3313995665	1.6502211792
[206]	-1.7251510678	-0.0928526233	0.7635427031	-1.3832053166	0.3532469046
[211]	-1.6317928189	2.0919851608	-1.1724360618	1.5263236457	-0.3764289784
[216]	-1.9536189273	2.8113782141	3.5795201780	-3.7209888305	-2.9019305696
[221]	4.1940583603	0.2302578428	-1.6539177082	0.4342718289	-1.8908050986
[226]	-0.3912795084	4.7566909690	1.4001294689	-2.1372075018	-1.5541010049
[231]	-1.1886038492	1.0886898200	1.8417004872	1.2511520362	0.8836033956
[236]	2.6806530888	-2.7157407946	0.0482039318	-0.6950392749	-1.0884479059
[241]	0.6306600467	-1.6605351992	-1.1039201509	-1.3887103176	0.1947370193
[246]	3.0548939304	-2.5235492833	2.3047494446	2.7396990815	0.5051745604
[251]	1.6267415482	-1.5473250357	-0.3053339715	2.0406227427	0.4659667282
[256]	-0.0304095695	0.8497272571	-5.9932621271	-1.8810909447	-2.7076067332
[261]	-0.1636194385	2.1491177402	2.0626673873	-1.5784866764	0.7635046483
[266]	-0.8349128013	-0.7494790196	3.4586530237	1.2230171801	3.1384636878
[271]	3.0070531747	-2.6340898360	-1.7942529843	0.4458006101	-1.0998068295
[276]	1.3007868645	1.4776270133	-3.1913182799	0.3260581715	2.4483184240
[281]	1.1826262047	-1.5226736686	2.0274803248	3.5412904545	-3.7739455603
[286]	2.7907952158	0.1103435694	2.4094794029	0.6227587027	2.2979823680
[291]	-1.2626702258	-0.8077408452	1.2131101512	1.4834726886	-0.2096864688
[296]	-1.8557417309	-2.3183649767	-0.0947310765	-0.9208388586	-2.1313591168
[301]	1.3307799129	0.8899018309	0.0688863834	1.4282642710	-1.6447049416
[306]	1.3686896211	-0.5539894681	2.7821308559	2.1538141068	0.4795813505
[311]	0.7574059394	-5.6236110148	0.8238919280	3.6945183619	2.5191902023
[316]	2.5848797892	-0.9188155436	5.7082332247	-4.4198279740	-1.5416407468
[321]	0.0564598117	1.8884687724	5.0209187868	1.0174006282	0.7955634563
[326]	-1.4396454484	-0.9888914005	-6.0846591623	-0.4213816014	-0.2401350359
[331]	2.5581722448	1.5923895932	4.8296044803	-2.4761193011	1.5041969339
[336]	-2.1291584243	0.5017202259	1.0428667818	0.4609230327	1.9101398469
[341]	1.8029657806	-2.0075092798	-2.3212382614	1.2452737559	-0.2069174260
[346]	0.7876852308	-1.3519192912	0.2336752388	0.7417448582	1.5362280598
[351]	3.5735347960	-1.5243678101	0.6737742549	-0.7873234292	2.8586937206
[356]	-1.4020496136	-3.2437441912	-1.3209337088	-4.0740102480	-3.0075459383
[361]	-1.0096407355	2.1047073833	1.7405437058	-0.7440215762	-3.6000637267
[366]	-0.7652598078	0.8999515647	-2.8948903622	0.2285075894	-0.7049275918
[371]	-3.0372629471	-3.0811954445	3.5656594309	-2.7186110194	-1.5528278968
[376]	1.3925843393	0.5523795039	-1.8080630518	-3.5679474888	-1.2252660184
[381]	3.6728565687	-0.7513320587	-1.3638281511	1.4429441788	0.7996863214
[386]	1.7024332803	-0.5975729879	-1.0966490883	-2.1003511721	1.2017141555
[391]	-0.0833157970	-1.2868967040	1.5472082695	-3.8885616200	3.3394067065
[396]	2.8117623016	1.4469246052	0.7728069509	2.3949932885	-3.8933214762

[401]	-3.3695593098	1.8849532359	-1.0181455135	-1.7267504481	0.1859480837
[406]	0.8067932742	-0.6859706651	1.2736305633	2.3881400625	1.1472322750
[411]	0.4723482600	0.6113661725	2.6834974310	0.3056164590	0.4836189954
[416]	-2.7281942160	1.7649311791	-1.8263951217	-1.1041089623	0.8364820305
[421]	-0.9065794115	1.5177707974	1.5042304609	2.3904267688	0.9353235209
[426]	0.6763770212	-0.4457000091	0.1878813136	0.2833174331	0.8309315342
[431]	-2.9176236853	-1.1920482997	2.0504357965	-0.5865887704	-3.3287550342
[436]	3.6012999843	0.8721737410	1.8904626921	-0.1625641632	-2.0871422212
[441]	0.2509815697	-0.5624894680	0.2408800999	0.5313318451	-3.2724062279
[446]	0.3655984195	0.2482117505	-0.6139737965	0.3596069328	0.6438845407
[451]	2.2172746888	0.6507942891	1.3942401698	-3.7193043753	-3.5907916295
[456]	2.7210045257	-0.4746593007	-3.7696304405	0.7017539968	2.2500281674
[461]	2.0169164179	-0.6350343607	-1.3073315076	-1.3797758713	-1.9189082302
[466]	3.3432169248	3.6649968512	1.3701581041	0.0709572039	-1.2364300501
[471]	0.0409848108	1.9186145422	-2.2409842399	-2.7525534690	2.8876894525
[476]	0.1046274109	2.9329911528	-2.0259481146	-2.9411704670	0.1390275412
[481]	-3.7543189778	-0.0280821458	3.4037818304	2.6085324324	1.6875298591
[486]	1.8950074217	0.3381316633	-0.5200309706	-1.5801224297	0.7709630640
[491]	2.0672198449	-1.1825217337	0.7869711506	-2.2121481501	-0.1640814399
[496]	-3.1219537928	-0.1278403185	-0.7765142769	0.8844752080	1.3616634129
[501]	-0.0940078482	1.1185781199	2.5319048797	-1.5480398979	3.0743831991
[506]	-1.8331814122	1.3913667659	0.1882208742	-0.8921328117	-2.6386462478
[511]	-0.0446123998	-1.3815125589	-1.0950772467	2.5335738082	1.6943361869
[516]	1.1302123211	1.0897899628	-1.8903556435	0.0637825634	-2.2607138630
[521]	1.3502932477	0.4783120307	4.1867337180	-0.7108781860	-2.1519783082
[526]	-1.4776996638	-0.4583680939	-2.4386079880	-0.6914516371	-1.4861831831
[531]	1.6996355843	1.7731108450	0.7798103338	-1.1636781773	0.9385328152
[536]	-2.6303341039	-3.3427932589	1.3690929691	2.4467827307	-1.2170867876
[541]	-3.9984278377	-0.2156985405	-1.4119104770	-0.0242564982	-0.2713729433
[546]	0.8927225048	0.3411476957	-1.2918171143	-0.4346439034	0.0513821787
[551]	-2.5315118657	1.5476853381	3.0659758728	-1.6312310475	1.2459165425
[556]	-1.5433716274	-6.5257710355	-0.5804369407	-1.1216803045	-0.9347880994
[561]	-5.4931626935	1.0910144419	-2.0342507467	-1.5086270093	-0.1076226810
[566]	0.8050366541	0.3486670434	-0.6908762605	0.3585289074	1.8863123259
[571]	-1.0353982748	-0.7888395923	-2.0786561115	2.7903633271	1.0008998052
[576]	-1.6379674133	0.8695250004	0.4655813425	-3.0324903772	1.0254904185
[581]	-0.3314759270	0.1602558633	0.5799041667	-1.8487906097	2.4712810769
[586]	0.7250332764	-1.2168406018	2.1882079424	-1.5677914003	-2.2775381184
[591]	2.4779615606	-2.3030103538	1.2522815199	0.7509252572	1.5288916737
[596]	0.3962440855	1.4908186025	0.7710022218	1.5685397310	2.0343785287
[601]	-2.7187167269	4.1979507734	-1.8652606865	-1.0046071369	-0.5191243168
[606]	-1.1976007098	-0.8986094006	-4.1055141382	2.3888866346	0.1712556994
[611]	-1.0856505100	-2.0287517816	0.8753534110	-0.4458737275	-2.0726228304

[616]	-0.9944898609	0.5248054011	2.0166573185	-0.5168022595	4.8591403172
[621]	-0.9230661804	1.7122302684	-0.6860411545	3.0102660546	-1.5740813052
[626]	-1.9862502770	0.3916791822	-0.0661782321	0.0429124969	1.2669151406
[631]	-0.9785970391	-0.3167366396	0.9874994235	-2.2092926727	0.1406668074
[636]	-1.8852523465	1.6012331352	-1.1532747117	1.5400535636	2.3663196830
[641]	-0.4958550987	-0.4042062793	3.5160900412	2.3561792144	0.7323919070
[646]	0.9123474868	2.1970952767	-0.3088814473	-0.2111258766	-0.3259807609
[651]	-1.4440748342	0.7647436350	-0.5769994630	-1.3621764276	-3.5029791426
[656]	-2.0583945774	-0.4788444040	-0.4173657034	2.4809628933	0.5846113985
[661]	-3.4112831180	1.3296127125	1.6721102981	2.5785608198	-0.9891630064
[666]	-0.5099541469	0.4780331468	2.5209175651	-0.0005110253	-2.6049299154
[671]	-1.3338055559	1.3828190311	-1.2818670839	0.7477775880	2.3688622196
[676]	-0.2566405083	0.2957323296	3.8185931310	-0.4398659773	-1.1076197538
[681]	-0.8831810735	4.4508431928	0.3625963685	-0.1152075560	-2.4901503799
[686]	1.0705226950	3.3222743988	0.4336015488	-0.0581875525	-1.1372717399
[691]	-1.1353016616	1.8887098021	2.4309125373	-1.5551825358	1.0340631219
[696]	2.2791603440	0.3324907091	0.8426987428	-2.5803608145	-1.8143336835
[701]	2.4682162417	0.9978700910	2.7523971971	-5.0343967576	-2.2490236708
[706]	-0.6333921122	1.8630685274	-0.2877321297	0.7335410432	0.6959387526
[711]	-0.8561574845	-0.9467986287	0.7212766736	-0.2076381527	-1.4739200131
[716]	-0.2851956344	1.7746380025	1.3955710211	-0.8830110059	1.4803913673
[721]	2.5340199011	0.7372566529	-0.3991961125	-0.3246966288	2.0129259313
[726]	0.9769844571	-2.1665723542	1.8118644277	0.6245767408	-0.1170741867
[731]	-1.3265517881	-2.0140798013	-1.5606497422	-0.3863700809	1.4069504486
[736]	-1.1703660284	0.5010404700	-1.3714182787	-1.0877109629	1.4348962966
[741]	0.6925140197	-0.4958208693	2.1082165808	2.7816665089	2.3820576287
[746]	0.9680611764	0.9498541693	-1.0406428068	1.2508633629	-1.4600486652
[751]	-0.5030545339	1.8300495605	-2.4893175303	1.4026167992	1.2651780287
[756]	-1.3898428855	-1.2339031229	-2.0486822957	1.7405922910	1.5061428093
[761]	-1.8571182341	0.4803316648	1.2506266213	0.8226389567	-1.6575259334
[766]	0.1885020479	1.9243838213	-0.8545075596	0.0649032821	-5.9020096420
[771]	1.9932788973	-1.2097460832	0.6216536509	-0.1309875177	-1.3313069468
[776]	-0.9199626679	-0.9811471650	-1.5017337934	1.4375262841	0.6551938393
[781]	-1.4497586225	-2.4999386319	-3.6599051064	-2.9819448499	-2.9136106390
[786]	-2.1908521463	0.4067792029	3.1891206747	0.3124727085	0.7284600744
[791]	0.3314603281	-0.8254807527	0.9171477812	4.1714596988	-1.9032302051
[796]	-0.8571109064	1.8635069517	-2.2192298605	2.0837371411	3.6423220760
[801]	1.9901786777	-1.1863694225	1.9290367988	-1.2059915926	-3.3199788573
[806]	-0.0358349578	-2.0528848300	0.6978922418	0.4029919892	-0.5783910781
[811]	3.4882107294	-1.1030938087	5.0765543831	-0.8108734177	0.7897762479
[816]	1.7696578412	-2.2579081632	-1.7872844709	1.0408170568	-2.7158406099
[821]	-1.9559724529	-0.2571070289	-3.3683695558	3.8571544988	-3.3961539301
[826]	0.0289114211	-1.8956870508	-2.2261361826	3.1088975930	4.1043153252

```

[831]  4.4028263486 -0.5276064610 -1.7488940280 -1.0225940665 -0.9055718473
[836] -1.5979830826 -1.2697470466 -2.5693808168 -0.5770620501  1.5868679018
[841] -0.3970045063 -0.6972255398  1.9730154231  2.8505446945 -2.3248802078
[846] -2.1481839922  1.4112496841 -0.5673588287  2.7718503240  1.8558957495
[851] -0.2968681537 -0.8026232473  0.8601034690 -0.9756271271  3.4440708509
[856] -1.4009223667  0.3195185370  0.9729389079 -1.4188643023  0.5582587576
[861]  2.3783137628 -2.6706847768  1.3853839134 -0.1395247707  3.1003955196
[866] -4.3300247004  1.6662959550 -1.4856699535 -0.0413834295  1.7156355518
[871] -0.3670216943  0.8230419886 -0.7220134027 -0.2626699000 -3.5078498138
[876] -0.9606623655 -0.6982931371  0.2320387501 -2.7744073296  0.1826934395
[881] -2.1141685622 -1.2323269872  3.9297891827 -3.6137593651 -1.2928707373
[886] -0.7449133462 -0.2475148598 -3.3062177936 -0.1349822246 -4.0369708053
[891] -1.9810181080 -1.2613526912 -1.0625243103 -0.8223974569  0.8083069382
[896] -0.5354954568  3.0074544643  1.0555190355 -0.9494691178 -0.1109845683
[901]  0.4658113308 -0.5577786903 -2.3622621837 -1.0700789987 -0.3470318523
[906] -1.3357592611  0.0651755452 -1.2209457577 -0.2738156031  0.3846897106
[911]  0.9986901847 -1.0104889501 -1.7816492210 -1.3846256934 -1.2181134417
[916]  0.7624254791  0.2855303252  1.8216082177  0.5622219451 -0.1606727196
[921] -2.1211798627 -3.6985421472 -1.2185746417 -1.9795176853 -2.9885034491
[926]  2.2115350875  0.0871630301  0.7338843665  0.1780677571  3.8827875050
[931] -1.7992207511  2.2672546579 -1.1699547671 -0.1659680076  1.5130461404
[936] -3.8497886138 -3.5595753909 -3.7121314246 -2.5955022606  1.3151475816
[941]  0.8583002631 -1.0942565930  0.4078826040 -1.0255286546 -2.3605952456
[946] -0.7968598062 -0.3802994724 -1.4469483291 -1.3597657087  1.1801709273
[951] -1.3275383616  2.8712401690 -2.5006440964 -2.1691777551 -0.2199765672
[956]  0.9162441879 -3.0162351339 -3.1446703065  2.0812386401 -1.0828196622
[961] -1.4175066263  1.8206473444  0.1241710991  0.0269917781  0.6226534092
[966] -3.4830418916 -2.7314682748  1.5668273371 -0.6973495386 -0.1010346725
[971]  1.5277245131 -0.5444909853  1.7695549096  0.4406765656 -0.4628928046
[976] -0.4288840309  3.7760028203 -0.2745157274  2.0073023503 -0.4902486677
[981]  1.8187790746  0.2286541151 -0.7024546432 -0.9145490492  0.5596835460
[986]  0.8309950502 -2.6001092318  2.8087240277  4.8489306012  1.3181989298
[991] -2.1019789844  1.3329018592 -1.6573549405  0.6705646711 -1.6470877156
[996]  1.1360236191 -2.8556275641  0.9900457175  1.3805216264  1.0541030786

```

```

A <- matrix(data = rnorm(n = 1000, mean = 0, sd = 2), nrow = 100, ncol = 10)

# get mean over columns
apply(A, 2, mean)

```

```

[1]  0.14366554 -0.09947342  0.29177338 -0.20992337  0.28670625  0.16661428
[7]  0.47877036  0.14551775 -0.27994062  0.18726831

```

```
# get mean over rows
apply(A, 1, mean)
```

```
[1] -0.11123252  0.25269261  1.07428044 -0.60963924 -0.59070967  0.19528229
[7]  0.29950823  0.42306102 -0.56125059 -0.16912478  0.21562841 -0.11086591
[13] -0.15368377  0.53569555  0.57717739  0.08698563 -0.01075613 -0.12827316
[19]  0.42787241  0.23596265 -0.18874820  0.70373552 -1.13359593  0.09395030
[25]  0.74480040  1.05860532 -0.17306840  1.35094577  0.34887303  0.40979913
[31]  0.46540806 -1.05761989 -0.34078157 -0.68746397  0.36001242  0.32440103
[37] -0.17868613  0.73224066  0.01150433  1.74558576  0.14859379  0.52874154
[43] -0.34208890  0.20407273  1.37539695  1.11192731 -0.92507823  0.01383881
[49]  0.21677429  0.93205143 -0.22781355 -1.00050947 -0.48326877 -0.04945245
[55]  0.87987478  0.10630157  0.04957086  0.23780120  0.76851330  0.04257522
[61] -0.05237888  0.25165309 -0.21302228 -0.19724344  0.03288053 -0.60542700
[67] -0.91275594 -0.27121885 -0.61023075 -0.17016129 -0.97208521  0.17647013
[73] -0.60073566  0.21958025  0.22961355 -0.49515488 -0.33648742  0.26069360
[79]  0.61117911 -0.65691925 -0.32158263 -0.50249778  0.04399044 -0.12368619
[85]  0.42993140  0.33473183  0.07571695  0.77755131  0.45909753  0.34190682
[91]  0.51409962  0.09473125  1.39688888  0.88198195  0.12049992 -0.23719500
[97]  0.56996695  0.69780201  0.66597254 -0.85470362
```

```
# calculate standard deviation for each column
apply(A, 2, sd)
```

```
[1] 1.900904 1.883296 1.734255 2.255404 1.821993 1.954093 1.948452 2.126121
[9] 2.210779 1.927645
```


18 Data Frames

A data frame is a table where each column can contain elements of different types (e.g., numbers, strings). It's the most common structure used for data sets.

```
# Creating a data frame
my_data <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(23, 30, 25),
  Gender = c("F", "M", "M")
)
```

```
my_data
```

	Name	Age	Gender
1	Alice	23	F
2	Bob	30	M
3	Charlie	25	M

```
# Accessing columns
my_data$Name # Output: "Alice", "Bob", "Charlie"
```

```
[1] "Alice" "Bob" "Charlie"
```

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03"))

# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)
```

```
df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)
```

```
# check DF structure
str(df2)
```

```
'data.frame':  10 obs. of  3 variables:
 $ num      : num  10 20 30 40 50 60 70 80 90 100
 $ date     : Date, format: "2020-01-01" "2020-01-08" ...
 $ string   : chr  "word" "word" "word" "word" ...
```

```
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
df3 <- as.data.frame(M)
df3
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60

row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```
nrow(df3)
```

```
[1] 10
```

```
ncol(df3)
```

```
[1] 10
```

```
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```
typeof(df3)
```

```
[1] "list"
```

```
# Accessing DF
```

```
# let's create DF - employees
```

```
df_emp <- data.frame(id = 1:6,
  name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),
  surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),
  age = c(55, 35, 46, 22, 60, 27),
  date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Date("2015-01-01"), as.Date("2018-05-01"), as.Date("2019-03-01"), as.Date("2020-07-01")),
  gender = c("M", "F", "M", "M", "F", "M"),
  manager_position = c(T, F, F, F, T, F)
)
```

```
# extract data as data frame (one column) - []  
df_extr <- df_emp["name"]  
df_extr
```

```
      name  
1      Max  
2      Jane  
3      John  
4      Tony  
5     Janis  
6     Helen
```

```
class(df_extr)
```

```
[1] "data.frame"
```

```
# extract data as vector (one column) [[]] $  
df_extr <- df_emp[["age"]]  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
df_extr <- df_emp$age  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
# extract multiple columns
df_extr <- df_emp[c("name", "age")]
df_extr
```

```
   name age
1  Max  55
2  Jane  35
3  John  46
4  Tony  22
5 Janis  60
6 Helen  27
```

```
# data frame slicing
df_emp
```

```
   id name surname age date_start_work gender manager_position
1  1  Max  Gordon  55    1985-09-01      M             TRUE
2  2  Jane   Smith  35    2010-10-01      F             FALSE
3  3  John    Don  46    1999-06-01      M             FALSE
4  4  Tony   Price  22    2019-03-01      M             FALSE
5  5 Janis    Jett  60    1980-04-15      F             TRUE
6  6 Helen   Dust  27    2015-02-20      M             FALSE
```

```
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```
df_emp[2,"name"]
```

```
[1] "Jane"
```

```
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

```
   gender manager_position
1      M             TRUE
2      F             FALSE
3      M             FALSE
4      M             FALSE
```

```
df_emp[1:4, c("gender", "manager_position")]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
# extract first column (all rows)  
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp[, "id"]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```
# extract last 2 rows (all columns)  
df_emp[5:6,]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
cols <- colnames(df_emp)  
df_emp[5:6, cols]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```

# Modifying data frame

# append column
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "an
df_emp$new_col <- 1

# append rows
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researcher

# problem with factor variables (new values not in factor levels)
# easy solution - append new row as data frame (rbind 2 data frames)!!!
# will show few rows later

# remove column
df_emp$new_col <- NULL

# remove row
df_emp <- df_emp[-7,]

# merge two data frames (row wise)
df_new_emp <- data.frame(id = 7,
                        name = "Mark",
                        surname = "Jax",
                        age = 32,
                        date_start_work = as.Date("2020-01-01"),
                        gender = "M",
                        manager_position = F,
                        role = "researcher")

df_emp <- rbind(df_emp, df_new_emp)

# merge two data frames (column wise)
df_attr <- data.frame(eye_color = c("blue", "green", "brown", "hazel", "blue", "brown", "bro
                        hair_color = c("blonde", "light brown", "black", "brown", "blonde", "d
df_emp <- cbind(df_emp, df_attr)

# Tips

# Df summary
summary(df_emp)

```

id		name		surname		age	
Min.	:1.0	Length:7		Length:7		Min.	:22.00
1st Qu.	:2.5	Class :character		Class :character		1st Qu.	:29.50
Median	:4.0	Mode :character		Mode :character		Median	:35.00
Mean	:4.0					Mean	:39.57
3rd Qu.	:5.5					3rd Qu.	:50.50
Max.	:7.0					Max.	:60.00

date_start_work		gender		manager_position		role	
Min.	:1980-04-15	Length:7		Mode :logical		Length:7	
1st Qu.	:1992-07-16	Class :character		FALSE:5		Class :character	
Median	:2010-10-01	Mode :character		TRUE :2		Mode :character	
Mean	:2004-05-06						
3rd Qu.	:2017-02-24						
Max.	:2020-01-01						

eye_color		hair_color	
Length:7		Length:7	
Class :character		Class :character	
Mode :character		Mode :character	

```
# rows subsetting
subset(x = df_emp, gender == "M")
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
subset(x = df_emp, gender == "F" & manager_position == T)
```

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
--	----	------	---------	-----	-----------------	--------	------------------	------	-----------


```

5 5 Janis Jett 60 1980-04-15 F TRUE CEO blue
   hair_color
5 blonde

```

```

rows <- which(df_emp[, "gender"] == "M")
df_emp[rows,]

```

```

   id name surname age date_start_work gender manager_position role
1  1  Max  Gordon  55 1985-09-01      M      TRUE  director
3  3  John   Don  46 1999-06-01      M     FALSE  analyst
4  4  Tony  Price  22 2019-03-01      M     FALSE researcher
6  6 Helen  Dust  27 2015-02-20      M     FALSE  analyst
7  7  Mark   Jax  32 2020-01-01      M     FALSE researcher
   eye_color hair_color
1    blue    blonde
3   brown    black
4   hazel    brown
6   brown dark brown
7   brown    brown

```

```

rows <- which(df_emp[, "gender"] == "F" & df_emp[, "manager_position"] == T)
df_emp[rows,]

```

```

   id name surname age date_start_work gender manager_position role eye_color
5  5 Janis Jett 60 1980-04-15      F      TRUE  CEO    blue
   hair_color
5 blonde

```

```

# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[, c("age", "manager_position")], 2, sum)

```

```

      age manager_position
277      2

```

19 Factors

Factors are used to represent categorical data. They store both the data values and the corresponding levels.

```
gender_factor <- factor(c("Male", "Female", "Male"))

# Display the factor and its levels
print(gender_factor)
levels(gender_factor)

# create factor variable (gender)
gender <- factor(x = c("male", "female", "female"))

# check new variable
gender
str(gender)
class(gender)
typeof(gender)

# create with ordering
gender <- factor(x = c("male", "female", "female"), ordered = T)
is.ordered(gender)

# check levels
levels(gender) # order of levels based on variable (string alphabetic order)

# we can define our own levels (custom levels order)
gender <- factor(x = c("male", "female", "female"), levels = c("male", "female"), ordered = T)
gender
levels(gender)

# factor properties
levels(gender)
is.factor(gender)
is.ordered(gender)
```

```
# create other object to factor
strings <- c("a", "b", "a", "c")
f_strings <- factor(strings)

#f_string
```

20 Arrays

Arrays are similar to matrices but can have more than two dimensions.

```
# Creating a 3-dimensional array
my_array <- array(1:24, dim = c(3, 4, 2))

# Accessing elements
my_array[1, 2, 1] # Access the element in the first dimension, second row, and first slice
```

```
[1] 4
```

21 Summary

- **Vector:** One-dimensional, homogeneous.
- **List:** One-dimensional, heterogeneous.
- **Matrix:** Two-dimensional, homogeneous.
- **Data Frame:** Two-dimensional, heterogeneous (columns can be different types).
- **Factor:** Categorical data representation.
- **Array:** Multi-dimensional, homogeneous.

22 Manipulating Vectors, Data Frames, and Lists

```
library(haven)  
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tidyr)
```

23 Vectors: Indexing & Vectorized Ops

```
#Creating vector
x=1:10
x=10:1
x=-5:10
x=c(1:10)
x=c(-5:10)
x=c(1,2,3,4,5,6,7,8,9,10)

# Naming a vector
a=c(1:3)
names(a) #Returns null
```

NULL

```
names(a)=c("one","two","three")
names(a)
```

```
[1] "one"  "two"  "three"
```

```
a
```

```
  one  two three
    1    2    3
```

```
#2)Accessing vector element
x=c(1,3,5,7)
x[2]
```

```
[1] 3
```

```
x[c(1,3)]
```

```
[1] 1 5
```

```
x[-2]
```

```
[1] 1 5 7
```

```
x[-c(1,3)]
```

```
[1] 3 7
```

```
x[0]
```

```
numeric(0)
```

```
y=x[10]  
class(y)
```

```
[1] "numeric"
```

```
#Note: X[0],x[10],output is numeric class only  
#*****  
#3)Modification of Vector elements  
x=c(9,3,5,7)  
x[2]=13  
x[-c(2,3)]=c(11,17)  
x[-1]=c(110,170,70)  
  
#*****  
x=c(11,3,5,7)  
x[9]=x[7]  
#*****  
x=c(11,3,5,7)  
x[9]=x[2]  
  
x=c(1,3,5,7)  
x[2]=x[11]
```



```

#####
x=c(1,3,5,7)
x[c(2,5)]=x[c(4,4)] # It assign 4 element of to 2nd element and 4th element to 5th element
##### from x=1,3,5,7
x=c(11,3,5,7)
x[c(2,7)]=x[c(1,3)]

x=c(1,3,5,7)
x[c(2,3)]=x[c(1,10)]

#4)Airthematic Operations on Vector
x=c(1,3,5,7)
x+10

```

```
[1] 11 13 15 17
```

```
x-5
```

```
[1] -4 -2 0 2
```

```
x*10
```

```
[1] 10 30 50 70
```

```
x/10
```

```
[1] 0.1 0.3 0.5 0.7
```

```

x=c(1,3,5,7)
x%%2

```

```
[1] 0 1 2 3
```

```

x=c(1,3,5,7)
x%%2

```

```
[1] 1 1 1 1
```

```
min(x)
```

```
[1] 1
```

```
max(x)
```

```
[1] 7
```

```
median(x)
```

```
[1] 4
```

```
mean(x)
```

```
[1] 4
```

```
range(x)
```

```
[1] 1 7
```

```
var(x)
```

```
[1] 6.666667
```

```
sd(x)
```

```
[1] 2.581989
```

```
quantile(x)
```

```
 0%  25%  50%  75% 100%  
1.0  2.5  4.0  5.5  7.0
```

```
quantile(1:20,probs=c(.25,0.9))
```

```
 25%  90%  
5.75 18.10
```

```
IQR(x)
```

```
[1] 3
```

```
#5)"WHICH" function  
x=c(2,3,4,5,11,112,133,33)  
x>5
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE
```

```
x=c(2,3,4,5,11,112,133,33)  
y=which(x>5) #Returns the position,not values  
y=x[which(x>5)]  
y=x[x>5]  
  
x=c(2,3,4,5,11,112,133,33)  
min(x)
```

```
[1] 2
```

```
which(x==min(x))
```

```
[1] 1
```

```
x[which(x==min(x))]  
#Returns the value
```

```
[1] 2
```

```
which.min(x)  
#Returns the position,not values
```

```
[1] 1
```

```
x=c(2,3,4,5,11,112,133,33)  
max(x)
```

```
[1] 133
```

```
which(x==max(x)) #Returns the position,not values
```

```
[1] 7
```

```
x[which(x==max(x))]  
#Returns the value
```

```
[1] 133
```

```
which.max(x)  
#Returns the position,not values
```

```
[1] 7
```

```
x=c(8,7,4,5,11,112,133,33)  
which(x>2 & x<5)
```

```
[1] 3
```

```
x[which(x>2 & x<5)]
```

```
[1] 4
```

```
x=c(8,7,4,5,11,112,133,33)  
which(x>7 | x<12)
```

```
[1] 1 2 3 4 5 6 7 8
```

```
x[which(x>7 | x<12)]
```

```
[1] 8 7 4 5 11 112 133 33
```

```
#6) "REP" function  
x=rep(1:5,times=10)  
x=rep(100,times=10)  
x=rep(c(3,6),times=4)  
x=rep("Kummam",times=5)  
x=rep(c("Ramesh","Kummam"),times=3)
```

```

x=rep(1:4,5:8)
#x=rep(1:4,1:2) #output as invalid argument
x=rep(1:4,c(2,3,5,7))
x=rep(1:4,each=3)
x=rep(1:4,each=2,times=3)

#7) "SEQ" function
x=seq(from=1,to=10,by=3)
#x=seq(from=1,to=10,by=-3) # wrong arguments
x=seq(from=10,to=1,by=-3)
x=seq(from=1,to=10,length=100)
x=seq(from=1,by=2,length=100)
y=seq(from=1,by=3,length=50)
z=c(x,y)

#8) seq_len() & seq_along() functions
x=c(8,7,4,5,11,112,133,33)
length(x)

```

```
[1] 8
```

```
seq_len(length(x))
```

```
[1] 1 2 3 4 5 6 7 8
```

```
seq_along(x) #Returns length of the object
```

```
[1] 1 2 3 4 5 6 7 8
```

```

#9) Dealing with missing values
x=c(11,3,5,7)
x[2]=NA
x[c(2,3)]=NA
is.na(x) #Output is a logical vector

```

```
[1] FALSE TRUE TRUE FALSE
```

```
x[!is.na(x)]
```

```
[1] 11 7
```

```
na.omit(x)
```

```
[1] 11 7  
attr(,"na.action")  
[1] 2 3  
attr(,"class")  
[1] "omit"
```

```
#Note: Observe the below operations carefully
```

```
x=c(1,NA,5,NA)  
#x==NA # not followed it so far  
#x[x==NA]# not followed it so far
```

```
#10) Naming a vector
```

```
x=c(1:3)  
names(x)=c("a","b","c")  
x[c("a","b")]
```

```
a b  
1 2
```

```
y=c("Ramesh","Kummam")  
names(y)=c("First","Last")  
y[c("Last","First")]
```

```
      Last      First  
"Kummam" "Ramesh"
```

```
#9) Checking the availability of elements in a vector
```

```
a=c(1:10)  
b=c(5:15)  
1 %in% a
```

```
[1] TRUE
```

```
1 %in% b
```

```
[1] FALSE
```

```
a %in% b
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
b %in% a
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
is.element(a,b)
```

```
[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
is.element(b,a)
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
# print strings  
print("string")
```

```
[1] "string"
```

```
# concatenate strings "a" + "b" = "ab"  
paste("a", "b", sep = "")
```

```
[1] "ab"
```

```
# paste objects of different length  
paste("i", 1:10, sep = ".")
```

```
[1] "i.1" "i.2" "i.3" "i.4" "i.5" "i.6" "i.7" "i.8" "i.9" "i.10"
```

```
paste(c("i","j", "k"), 1:10, sep=".")
```

```
[1] "i.1" "j.2" "k.3" "i.4" "j.5" "k.6" "i.7" "j.8" "k.9" "i.10"
```

```
# paste with collapsing
```

```
paste(c("i","j", "k"), 1:3, sep = "", collapse = "")
```

```
[1] "i1j2k3"
```

```
# paste without collapsing
```

```
paste(c("i","j", "k"), 1:3, sep = "")
```

```
[1] "i1" "j2" "k3"
```

```
# paste0() shorter version of paste(..., sep="")
```

```
paste0("Hello", "world", ",", "I", "use", "R")
```

```
[1] "Helloworld,IuseR"
```

```
paste("Hello", "world", ",", "I", "use", "R", sep=" ")
```

```
[1] "Hello world , I use R"
```

```
# concatenate strings with cat()
```

```
cat("Hello", "world", "!")
```

```
Hello world !
```

```
# it prints without quotes !!!
```

```
cat("Hello", "world", "!", sep = "/")
```

```
Hello/world/!
```

```
# counting number of characters nchar()
```

```
nchar("Hello world")
```

```
[1] 11
```



```
# load US states names from data frame regarding crime rate
df <- USArrests
head(df)
```

	Murder	Assault	UrbanPop	Rape
Alabama	13.2	236	58	21.2
Alaska	10.0	263	48	44.5
Arizona	8.1	294	80	31.0
Arkansas	8.8	190	50	19.5
California	9.0	276	91	40.6
Colorado	7.9	204	78	38.7

```
rownames(df)
```

[1] "Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5] "California"	"Colorado"	"Connecticut"	"Delaware"
[9] "Florida"	"Georgia"	"Hawaii"	"Idaho"
[13] "Illinois"	"Indiana"	"Iowa"	"Kansas"
[17] "Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21] "Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25] "Missouri"	"Montana"	"Nebraska"	"Nevada"
[29] "New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33] "North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37] "Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41] "South Dakota"	"Tennessee"	"Texas"	"Utah"
[45] "Vermont"	"Virginia"	"Washington"	"West Virginia"
[49] "Wisconsin"	"Wyoming"		

```
states <- rownames(df)

# convert all states names to upper case
states_upper <- toupper(states)

# convert all names to lower
states_lower <- tolower(states)

# or select which to apply with function casefold
casefold(x = states, upper = T)
```

[1] "ALABAMA"	"ALASKA"	"ARIZONA"	"ARKANSAS"
---------------	----------	-----------	------------

[5]	"CALIFORNIA"	"COLORADO"	"CONNECTICUT"	"DELAWARE"
[9]	"FLORIDA"	"GEORGIA"	"HAWAII"	"IDAHO"
[13]	"ILLINOIS"	"INDIANA"	"IOWA"	"KANSAS"
[17]	"KENTUCKY"	"LOUISIANA"	"MAINE"	"MARYLAND"
[21]	"MASSACHUSETTS"	"MICHIGAN"	"MINNESOTA"	"MISSISSIPPI"
[25]	"MISSOURI"	"MONTANA"	"NEBRASKA"	"NEVADA"
[29]	"NEW HAMPSHIRE"	"NEW JERSEY"	"NEW MEXICO"	"NEW YORK"
[33]	"NORTH CAROLINA"	"NORTH DAKOTA"	"OHIO"	"OKLAHOMA"
[37]	"OREGON"	"PENNSYLVANIA"	"RHODE ISLAND"	"SOUTH CAROLINA"
[41]	"SOUTH DAKOTA"	"TENNESSEE"	"TEXAS"	"UTAH"
[45]	"VERMONT"	"VIRGINIA"	"WASHINGTON"	"WEST VIRGINIA"
[49]	"WISCONSIN"	"WYOMING"		

```
casefold(x = states, upper = F)
```

[1]	"alabama"	"alaska"	"arizona"	"arkansas"
[5]	"california"	"colorado"	"connecticut"	"delaware"
[9]	"florida"	"georgia"	"hawaii"	"idaho"
[13]	"illinois"	"indiana"	"iowa"	"kansas"
[17]	"kentucky"	"louisiana"	"maine"	"maryland"
[21]	"massachusetts"	"michigan"	"minnesota"	"mississippi"
[25]	"missouri"	"montana"	"nebraska"	"nevada"
[29]	"new hampshire"	"new jersey"	"new mexico"	"new york"
[33]	"north carolina"	"north dakota"	"ohio"	"oklahoma"
[37]	"oregon"	"pennsylvania"	"rhode island"	"south carolina"
[41]	"south dakota"	"tennessee"	"texas"	"utah"
[45]	"vermont"	"virginia"	"washington"	"west virginia"
[49]	"wisconsin"	"wyoming"		

```
# character translation
chartr(old = "o", new = "0", x = "Hello World")
```

```
[1] "Hello W0rld"
```

```
# sorting strings
sort(states, decreasing = F) #ascending order
```

[1]	"Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5]	"California"	"Colorado"	"Connecticut"	"Delaware"
[9]	"Florida"	"Georgia"	"Hawaii"	"Idaho"

[13]	"Illinois"	"Indiana"	"Iowa"	"Kansas"
[17]	"Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21]	"Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25]	"Missouri"	"Montana"	"Nebraska"	"Nevada"
[29]	"New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33]	"North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37]	"Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41]	"South Dakota"	"Tennessee"	"Texas"	"Utah"
[45]	"Vermont"	"Virginia"	"Washington"	"West Virginia"
[49]	"Wisconsin"	"Wyoming"		

```
sort(states, decreasing = T) #descending order
```

[1]	"Wyoming"	"Wisconsin"	"West Virginia"	"Washington"
[5]	"Virginia"	"Vermont"	"Utah"	"Texas"
[9]	"Tennessee"	"South Dakota"	"South Carolina"	"Rhode Island"
[13]	"Pennsylvania"	"Oregon"	"Oklahoma"	"Ohio"
[17]	"North Dakota"	"North Carolina"	"New York"	"New Mexico"
[21]	"New Jersey"	"New Hampshire"	"Nevada"	"Nebraska"
[25]	"Montana"	"Missouri"	"Mississippi"	"Minnesota"
[29]	"Michigan"	"Massachusetts"	"Maryland"	"Maine"
[33]	"Louisiana"	"Kentucky"	"Kansas"	"Iowa"
[37]	"Indiana"	"Illinois"	"Idaho"	"Hawaii"
[41]	"Georgia"	"Florida"	"Delaware"	"Connecticut"
[45]	"Colorado"	"California"	"Arkansas"	"Arizona"
[49]	"Alaska"	"Alabama"		

```
# extracting parts of string
# sub string first 3 letters from state name Alabama
substr(x = "Alabama", start = 1, stop = 3)
```

```
[1] "Ala"
```

```
# String matching - back to toy example
help(regex)

# get all country names
#install.packages("countrycode")
require(countrycode)
```

Loading required package: countrycode

```
countries <- as.vector(countrycode::codelist$country.name.en)
```

```
# countries beginning with letter "A"
grep(pattern = "^A", x = countries)
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

```
countries[grep(pattern = "^A", x = countries)]
```

```
[1] "Afghanistan"      "Albania"          "Algeria"
[4] "American Samoa"   "Andorra"          "Angola"
[7] "Anguilla"         "Antarctica"       "Antigua & Barbuda"
[10] "Argentina"        "Armenia"          "Aruba"
[13] "Australia"        "Austria"          "Austria-Hungary"
[16] "Azerbaijan"
```

```
countries[grep(pattern = "^A", x = countries)]
```

```
[1] "Afghanistan"      "Albania"          "Algeria"
[4] "American Samoa"   "Andorra"          "Angola"
[7] "Anguilla"         "Antarctica"       "Antigua & Barbuda"
[10] "Argentina"        "Armenia"          "Aruba"
[13] "Australia"        "Austria"          "Austria-Hungary"
[16] "Azerbaijan"
```

```
# all country names that end with letter "y"
rez <- grep(pattern = "*y$", x = countries)
countries[rez]
```

```
[1] "Austria-Hungary"      "British Indian Ocean Territory"
[3] "Germany"             "Guernsey"
[5] "Hungary"             "Italy"
[7] "Jersey"              "Norway"
[9] "Paraguay"            "Saxony"
[11] "St. Barthélemy"      "Turkey"
[13] "Tuscany"             "Uruguay"
[15] "Vatican City"
```

```
# all country with 2 words or more for a country name
rez <- grep(pattern = "\\w\\s\\w", x = countries)
countries[rez]
```

```
[1] "American Samoa"
[2] "Bouvet Island"
[3] "British Indian Ocean Territory"
[4] "British Virgin Islands"
[5] "Burkina Faso"
[6] "Cape Verde"
[7] "Caribbean Netherlands"
[8] "Cayman Islands"
[9] "Central African Republic"
[10] "Channel Islands"
[11] "Christmas Island"
[12] "Cook Islands"
[13] "Costa Rica"
[14] "Côte d'Ivoire"
[15] "Dominican Republic"
[16] "El Salvador"
[17] "Equatorial Guinea"
[18] "Falkland Islands"
[19] "Faroe Islands"
[20] "French Guiana"
[21] "French Polynesia"
[22] "French Southern Territories"
[23] "German Democratic Republic"
[24] "Heard & McDonald Islands"
[25] "Hesse Electoral"
[26] "Hesse Grand Ducal"
[27] "Hong Kong SAR China"
[28] "Isle of Man"
[29] "Macao SAR China"
[30] "Marshall Islands"
[31] "Mecklenburg Schwerin"
[32] "Micronesia (Federated States of)"
[33] "Netherlands Antilles"
[34] "New Caledonia"
[35] "New Zealand"
[36] "Norfolk Island"
[37] "North Korea"
[38] "North Macedonia"
```

```

[39] "Northern Mariana Islands"
[40] "Orange Free State"
[41] "Palestinian Territories"
[42] "Papua New Guinea"
[43] "Pitcairn Islands"
[44] "Puerto Rico"
[45] "Republic of Vietnam"
[46] "Saint Martin (French part)"
[47] "San Marino"
[48] "Saudi Arabia"
[49] "Serbia and Montenegro"
[50] "Sierra Leone"
[51] "Sint Maarten"
[52] "Solomon Islands"
[53] "South Africa"
[54] "South Georgia & South Sandwich Islands"
[55] "South Korea"
[56] "South Sudan"
[57] "Sri Lanka"
[58] "Svalbard & Jan Mayen"
[59] "São Tomé & Príncipe"
[60] "Turks & Caicos Islands"
[61] "Two Sicilies"
[62] "U.S. Virgin Islands"
[63] "United Arab Emirates"
[64] "United Arab Republic"
[65] "United Kingdom"
[66] "United Province CA"
[67] "United States"
[68] "United States Minor Outlying Islands (the)"
[69] "Vatican City"
[70] "Western Sahara"
[71] "Yemen Arab Republic"
[72] "Yemen People's Republic"
[73] "Åland Islands"

```

```

# all country names that end with letter "e" or "i"
rez <- grep(pattern = "*e$|*i$", x = countries)
countries[rez]

```

```

[1] "Belize"           "Brunei"           "Burundi"
[4] "Cape Verde"       "Chile"             "Congo - Brazzaville"

```

```

[7] "Côte d'Ivoire"      "Djibouti"          "Eswatini"
[10] "Fiji"               "France"            "Greece"
[13] "Guadeloupe"         "Haiti"              "Kiribati"
[16] "Malawi"             "Mali"               "Martinique"
[19] "Mayotte"            "Mozambique"         "Niue"
[22] "Orange Free State"  "Sierra Leone"      "Singapore"
[25] "Suriname"           "São Tomé & Príncipe" "Timor-Leste"
[28] "Ukraine"            "Zimbabwe"

```

```

# all country names which contain combination of letters "gin"
rez <- grep(pattern = "*(gin)", x = countries)
countries[rez]

```

```

[1] "British Virgin Islands" "U.S. Virgin Islands"

```

```

# metacharacters and double backslash sign
strings <- c("dollar $", "dollar", "US dollar")

# how to escape metacharacters in R
# we would like to match a word with "$" dollar sign
strings

```

```

[1] "dollar $" "dollar"   "US dollar"

```

```

rez1 <- grep(pattern = "$", x = strings) # wrong way all are find $ indicating end of string
strings[rez1]

```

```

[1] "dollar $" "dollar"   "US dollar"

```

```

# we need to escape $ with \ backslash
#rez2 <- grep(pattern = "\$", x = strings) # wrong way error
#strings[rez2]

# right way $ escape with \\ double backslash
#rez3 <- grep(pattern = "\\$", x = strings) # wrong way error
#strings[rez3]

# Escape dot .
strings <-c("word.word", "word word")
rez <- grep(pattern = "\\.", x = strings)
strings[rez]

```

```
[1] "word.word"
```

```
# the hat sign beginning of the string and dollar sign end of string
strings <- c("Word", "worD")
strings[grepl("^W",strings)]
```

```
[1] "Word"
```

```
strings[grepl("D$",strings)]
```

```
[1] "worD"
```

```
# example of an anchor sequences in R
strings <- c("123", "onetwothree", "1twothree")
strings[grepl("\\d", strings)] # digit character
```

```
[1] "123"          "1twothree"
```

```
strings[grepl("\\D", strings)] # non-digit character
```

```
[1] "onetwothree" "1twothree"
```

```
# example of a character classes
strings <- c("123", "dollar", "shkjl")
strings[grepl("[aeiou]", strings)] # match word with any vowel
```

```
[1] "dollar"
```

```
strings[grepl("[0-9]", strings)] # match word with digits
```

```
[1] "123"
```

```
# find and replace sub() & gsub()
string <- "I have started to learn R, and I already love R."

# replace "R" with "X"
sub(pattern = "R", replacement = "X", x = string) #only first occurrence replaced
```

```
[1] "I have started to learn X, and I already love R."
```



```
gsub(pattern = "R", replacement = "X", x = string) #all occurrences replaced
```

```
[1] "I have started to learn X, and I already love X."
```

```
#replace white space with blank space  
gsub(pattern = "\\s", replacement = "", x = string)
```

```
[1] "IhavestartedtolearnR,andIalreadyloveR."
```

```
# string split, split given sentence by comma ","  
strsplit(x = string, split = ",")
```

```
[[1]]  
[1] "I have started to learn R" " and I already love R."
```

```
# split phone numbers by digits  
numbers <- c("310-555-123", "311-444-456")  
strsplit(x = numbers, split = "-")
```

```
[[1]]  
[1] "310" "555" "123"
```

```
[[2]]  
[1] "311" "444" "456"
```

24 Data Frames with dplyr

```
# create data frame
df1 <- data.frame(col1 = 1:3,
                  col2 = c("a", "b", "c"),
                  col3 = c(T, F, T),
                  col4 = c(as.Date("2020-01-01"), as.Date("2020-01-03"), as.Date("2020-01-03")))

# create data frame - vectors
col1 <- seq(10,100,10)
col2 <- seq(as.Date("2020-01-01"), length = 10, by = "weeks")
col3 <- rep("word", 10)

df2 <- data.frame(num = col1,
                  date = col2,
                  string = col3)

# check DF structure
str(df2)
```

```
'data.frame':  10 obs. of  3 variables:
 $ num      : num  10 20 30 40 50 60 70 80 90 100
 $ date     : Date, format: "2020-01-01" "2020-01-08" ...
 $ string   : chr  "word" "word" "word" "word" ...
```

```
# create data frame - matrix
M <- matrix(data = 1:100, nrow = 10, ncol = 10, byrow = T)
rownames(M) <- paste("row", 1:10, sep = "")
colnames(M) <- paste("col", 1:10, sep = "")
M
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20

row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
df3 <- as.data.frame(M)
df3
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
# check DF dimensions
dim(df3)
```

```
[1] 10 10
```

```
nrow(df3)
```

```
[1] 10
```

```
ncol(df3)
```

```
[1] 10
```

```
# check DF type / class
class(df3)
```

```
[1] "data.frame"
```

```
typeof(df3)
```

```
[1] "list"
```

```
# 4.3.2 Accessing DF
```

```
# let's create DF - employees
```

```
df_emp <- data.frame(id = 1:6,  
                     name = c("Max", "Jane", "John", "Tony", "Janis", "Helen"),  
                     surname = c("Gordon", "Smith", "Don", "Price", "Jett", "Dust"),  
                     age = c(55, 35, 46, 22, 60, 27),  
                     date_start_work = c(as.Date("1985-09-01"), as.Date("2010-10-01"), as.Date("2015-01-01"),  
                                          as.Date("2012-03-01"), as.Date("2018-05-01"), as.Date("2019-08-01")),  
                     gender = c("M", "F", "M", "M", "F", "M"),  
                     manager_position = c(T, F, F, F, T, F)  
                     )
```

```
# extract data as data frame (one column) - []
```

```
df_extr <- df_emp["name"]
```

```
df_extr
```

```
   name  
1   Max  
2  Jane  
3  John  
4  Tony  
5 Janis  
6 Helen
```

```
class(df_extr)
```

```
[1] "data.frame"
```

```
# extract data as vector (one column) [[]] $
```

```
df_extr <- df_emp[["age"]]
```

```
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
df_extr <- df_emp$age  
df_extr
```

```
[1] 55 35 46 22 60 27
```

```
class(df_extr) # vector factor
```

```
[1] "numeric"
```

```
# extract multiple columns  
df_extr <- df_emp[c("name", "age")]  
df_extr
```

```
      name age  
1    Max  55  
2   Jane  35  
3   John  46  
4   Tony  22  
5  Janis  60  
6  Helen  27
```

```
# data frame slicing  
df_emp
```

```
   id  name surname age date_start_work gender manager_position  
1  1   Max  Gordon  55    1985-09-01      M             TRUE  
2  2  Jane   Smith  35    2010-10-01      F             FALSE  
3  3  John    Don   46    1999-06-01      M             FALSE  
4  4  Tony   Price  22    2019-03-01      M             FALSE  
5  5 Janis    Jett  60    1980-04-15      F             TRUE  
6  6 Helen   Dust  27    2015-02-20      M             FALSE
```

```
#extract second row in name column (1 cell)
df_emp[2,2]
```

```
[1] "Jane"
```

```
df_emp[2,"name"]
```

```
[1] "Jane"
```

```
# extract first 4 rows of last 2 columns
df_emp[1:4, 6:7]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
df_emp[1:4, c("gender", "manager_position")]
```

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

```
# extract first column (all rows)
df_emp[,1]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp[,"id"]
```

```
[1] 1 2 3 4 5 6
```

```
df_emp$id
```

```
[1] 1 2 3 4 5 6
```

```
# extract last 2 rows (all columns)
df_emp[5:6,]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
cols <- colnames(df_emp)
df_emp[5:6, cols]
```

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
# 4.3.3 Modifying data frame
```

```
# append column
```

```
df_emp <- cbind(df_emp, role = c("director", "secretary", "analyst", "researcher", "CEO", "analyst"))
df_emp$new_col <- 1
```

```
# append rows
```

```
df_emp <- rbind(df_emp, list(7, "Mark", "Jax", 32, as.Date("2020-01-01"), "M", F, "researcher"))
```

```
# problem with factor variables (new values not in factor levels)
```

```
# easy solution - append new row as data frame (rbind 2 data frames)!!!
```

```
# will show few rows later
```

```
# remove column
```

```
df_emp$new_col <- NULL
```

```
# remove row
```

```
df_emp <- df_emp[-7,]
```

```
# merge two data frames (row wise)
```

```
df_new_emp <- data.frame(id = 7,
```

[illegible]

id		name		surname		age	
Min.	:1.0	Length:7		Length:7		Min.	:22.00
1st Qu.:	2.5	Class :character		Class :character		1st Qu.:	29.50
Median	:4.0	Mode :character		Mode :character		Median	:35.00
Mean	:4.0					Mean	:39.57
3rd Qu.:	5.5					3rd Qu.:	50.50
Max.	:7.0					Max.	:60.00
date_start_work		gender		manager_position		role	
Min.	:1980-04-15	Length:7		Mode :logical		Length:7	
1st Qu.:	1992-07-16	Class :character		FALSE:5		Class :character	
Median	:2010-10-01	Mode :character		TRUE :2		Mode :character	
Mean	:2004-05-06						
3rd Qu.:	2017-02-24						
Max.	:2020-01-01						
eye_color		hair_color					
Length:7		Length:7					
Class :character		Class :character					
Mode :character		Mode :character					


```
# rows subsetting
subset(x = df_emp, gender == "M")
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
subset(x = df_emp, gender == "F" & manager_position == T)
```

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE	CEO	blue

	hair_color
5	blonde

```
rows <- which(df_emp[, "gender"] == "M")
df_emp[rows,]
```

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

```
rows <- which(df_emp[, "gender"] == "F" & df_emp[, "manager_position"] == T)
df_emp[rows,]
```

```
   id name surname age date_start_work gender manager_position role eye_color
5  5 Janis      Jett  60      1980-04-15      F              TRUE   CEO      blue
   hair_color
5      blonde
```

```
# some calculations regarding data frames
nr_managers <- sum(df_emp$manager_position)
mean_age <- mean(df_emp$age)
df_emp$name_surname <- paste(df_emp$name, df_emp$surname, sep = " ") # merge name and surname

# use apply to sum over columns (age, manager_position)
apply(df_emp[, c("age", "manager_position")], 2, sum)
```

```
      age manager_position
      277                2
```

```
# if statement

x <- 3
y <- 14

if(x < y){
  print("x is less than y")
}
```

```
[1] "x is less than y"
```

```
# if-else statement

x <- 3
y <- 14

if(x > y){
  print("x is greater than y")
} else{
  print("x is less or equal to y")
}
```

```
[1] "x is less or equal to y"
```

```
# if-else if-else statement

x <- 14
y <- 14

if(x > y){
  print("x is greater than y")
} else if (x < y){
  print("x is less than y")
} else{
  print("x is equal to y")
}
```

```
[1] "x is equal to y"
```

```
# 5.1.2 relational operators

# scalar
x <- 3

x == 4
```

```
[1] FALSE
```

```
x > 0
```

```
[1] TRUE
```

```
x < 5
```

```
[1] TRUE
```

```
x <= 3
```

```
[1] TRUE
```

```
x >= 10
```

```
[1] FALSE
```

```
x %in% c(1,2,3,4,5)
```

```
[1] TRUE
```

```
# vectors
X <- c(F,T,0,10)
Y <- c(T,F,F,T)

X == Y # element-wise comparison
```

```
[1] FALSE FALSE TRUE FALSE
```

```
X > Y
```

```
[1] FALSE TRUE FALSE TRUE
```

```
X < Y
```

```
[1] TRUE FALSE FALSE FALSE
```

```
X >= Y
```

```
[1] FALSE TRUE TRUE TRUE
```

```
X <= Y
```

```
[1] TRUE FALSE TRUE FALSE
```

```
X != Y
```

```
[1] TRUE TRUE FALSE TRUE
```

```
X %in% Y
```

```
[1] TRUE TRUE TRUE FALSE
```

```
# 5.1.3 Logical operators
```

```
X | Y
```

```
[1] TRUE TRUE FALSE TRUE
```

```
#X || Y
```

```
X & Y
```

```
[1] FALSE FALSE FALSE TRUE
```

```
#X && Y
```

```
!X
```

```
[1] TRUE FALSE TRUE FALSE
```

```
# logical operators in if statement (flip coin twice)
```

```
c1 <- "H"
```

```
c2 <- "H"
```

```
if(c1 == "H" & c2 == "H"){  
  paste("You win 10$")  
} else if((c1 == "H" & c2 == "T") | (c1 == "T" & c2 == "H")){  
  paste("You win 2$")  
} else{  
  paste("You lose all the money")  
}
```

```
[1] "You win 10$"
```

```
# loop over iterations and print number of iteration
```

```
for(it in 1:10){  
  print(paste("iteration ", it, sep = ""))  
}
```

```
[1] "iteration 1"
[1] "iteration 2"
[1] "iteration 3"
[1] "iteration 4"
[1] "iteration 5"
[1] "iteration 6"
[1] "iteration 7"
[1] "iteration 8"
[1] "iteration 9"
[1] "iteration 10"
```

```
# sum of numbers in a sequence
sequence <- c(1,2,3,4,5)
s <- 0 # initial sum

for(val in sequence){
  s <- s + val
  print(paste("value = ", val, sep = ""))
  print(paste("s = ", s, sep = ""))
  print("-----")
}
```

```
[1] "value = 1"
[1] "s = 1"
[1] "-----"
[1] "value = 2"
[1] "s = 3"
[1] "-----"
[1] "value = 3"
[1] "s = 6"
[1] "-----"
[1] "value = 4"
[1] "s = 10"
[1] "-----"
[1] "value = 5"
[1] "s = 15"
[1] "-----"
```

```
# for loop with conditional statement
# (count number of even numbers in a vector)
x <- c(1,3,2,4,5,10,22,21,100) # given numbers
count <- 0 # counter for even numbers
```

```
for(val in x){  
  if(val %% 2 == 0){ # if number is divisible with 2  
    count <- count + 1  
  }  
}  
  
print(count)
```

[1] 5

25 Read sas dataset

```
dm <- haven::read_sas("data/sdtm/dm.sas7bdat")  
ae <- haven::read_sas("data/sdtm/ae.sas7bdat")
```


26 keeping selected columns

```
dm_1 <- dm |>
  dplyr::select(USUBJID,ARM) |>
  dplyr::filter(ARM == "Placebo")

dm_2 <- dm |>
  dplyr::select(USUBJID:ARM)

dm_2 <- dm |>
  dplyr::select(1,2,3)

dm_2 <- dm |>
  dplyr::select(1:3)

dm_2 <- dm |>
  dplyr::select(starts_with("AR"))

dm_2 <- dm |>
  dplyr::select(contains("DT"))

dm2 <- dplyr::select(dm,USUBJID,ARM)
```

27 dropping columns

To select all columns except certain ones, put a “-” in front of the variable to exclude it. For multiple variables, you can use the function `c()` to combine values into a vector or list. This will select all the variables in surveys except ARM, SITEID

```
adsl <- haven::read_sas("data/adam/adsl.sas7bdat")
adsl2 <- dplyr::select(adsl, -c(AGE, SEX, TRT01A))
adsl3 <- dplyr::select(adsl, -c(STUDYID:ARM))
adsl4 <- dplyr::select(adsl, -c(1:6))
adsl4 <- dplyr::select(adsl, -c(1:6, 43))
adsl5 <- dplyr::select(adsl, -starts_with("A"))
adsl6 <- dplyr::select(adsl, -ends_with("DTC"))
adsl7 <- dplyr::select(adsl, -contains("TRT"))
```

There are many helper functions available with `select()` like: `starts_with()`, `ends_with()`, `contains()` among others. These were imported from the `tidyselect` package. You can put a “-” in front of the helper function to negate it. Here is an example using `contains()`: # Filtering rows To choose rows based on a specific criteria, use `filter()`: To select rows where the planned treatment code equals “Placebo”:

```
adsl1 <- adsl |>
  dplyr::select(USUBJID, TRT01A) |>
  dplyr::filter(TRT01A == "Placebo" )

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & SEX == "M" & AGE == 70)

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & (SEX == "M" | AGE == 70))

adsl2 <- adsl |>
  dplyr::select(USUBJID, TRT01A, SEX, AGE) |>
  dplyr::filter(TRT01A == "Placebo" & SEX == "M" & AGE %in% c(70, 80))
```

```

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A=="Placebo" & !SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo","Xanomeline High Dose") & !SEX == "M" & !AGE %in% c(70,80))

adsl2 <- adsl |>
  dplyr::select(USUBJID,TRT01A,SEX,AGE) |>
  dplyr::filter(TRT01A %in% c("Placebo","Xanomeline High Dose") & !SEX == "M" & AGE >= 70)

```

28 Rename

```
ads11 <- ads1 |>
  dplyr::rename(usubjid=USUBJID)

ads11 <- ads1 |>
  dplyr::rename(usubjid=USUBJID,
                trt=TRT01P)
```

29 Arrange (sorting)

```
adsl1 <- adsl |>
  dplyr::select(USUBJID,SEX,AGE) |>
  dplyr::arrange(AGE,SEX)

adsl1 <- adsl |>
  dplyr::select(USUBJID,SEX,AGE) |>
  dplyr::arrange(AGE,desc(SEX))
```

30 convert var names to lower case

```
tolower(names(ads1))
```

```
[1] "studyid" "usubjid" "subjid" "siteid" "sitegr1" "arm"  
[7] "trt01p" "trt01pn" "trt01a" "trt01an" "trtsdt" "trtedt"  
[13] "trtdur" "avgdd" "cumdose" "age" "agegr1" "agegr1n"  
[19] "ageu" "race" "racen" "sex" "ethnic" "saffl"  
[25] "ittfl" "efffl" "comp8fl" "comp16fl" "comp24fl" "disconfl"  
[31] "dsraefl" "dthfl" "bmibl" "bmiblgr1" "heightbl" "weightbl"  
[37] "educlvl" "disonsdt" "durdis" "durdsgr1" "visit1dt" "rfstdtc"  
[43] "rfendtc" "visnumen" "rfendt" "dcdecod" "dcreascd" "mmsetot"
```

```
ads11 <- ads1
```

```
names(ads11) <- tolower(names(ads11))
```

31 Creating new variables

```
adsl1 <- adsl |>
  dplyr::mutate(name="Sri Ram")

adsl_test <- adsl |>
  dplyr::mutate(
    AGEGR1_t = dplyr::if_else(AGE < 65, "<65", ">=65"),           # character
    SAFFL_t  = dplyr::if_else(!is.na(TRTSDT), "Y", "N")         # character flag
  ) |>
  dplyr::select(USUBJID,AGE,AGEGR1_t,SAFFL_t,TRTSDT)
```

32 bind rows (set operator in SAS)

```
# bind rows
df_a <- data.frame(
  id = 1:3,
  value = c("A", "B", "C"),
  score = c(10, 20, 30)
)
df_b <- data.frame(
  id = 4:6,
  value = c("D", "E", "F"),
  score = c(40, 50, 60)
)

com <- dplyr::bind_rows(df_a,
                        df_b)

df_combined <- dplyr::bind_rows(df_a, df_b, df_a, .id = "source")

df_combined <- dplyr::bind_rows(df_a, df_b, df_a, .id = "source")

df_combined <- dplyr::bind_rows("dset1"=df_a,
                                "dset2"=df_b,
                                "test"=df_a, .id = "Source_var_name")

df_combined <- dplyr::bind_rows(select(df_a, id, value),
                                df_b)

adsl_tot <- dplyr::bind_rows(adsl |>
                             dplyr::select(USUBJID, TRT01P, TRT01PN),

                             adsl |>
                             dplyr::mutate(TRT01P="Total", TRT01PN=99) |>
                             dplyr::select(USUBJID, TRT01P, TRT01PN)
)
```


33 recode (similar to PROC FORMAT in SAS)

```
sex_fm <- c("M"="Male",
           "F"="Female"
)
adsl_test1 <- adsl |>
  dplyr::mutate(gender=dplyr::recode(SEX,!!!sex_fm))

adsl_test2 <- adsl |>
  dplyr::mutate(SEX=dplyr::recode(SEX,"M"="Male",
                                "F"="Female"))

adsl_test <- adsl |>
  dplyr::mutate(
    gn = dplyr::if_else(SEX=="M","Male","Female") # character
  ) |>
  dplyr::select(USUBJID,SEX,gn) |>
  head(5)
```

34 factor levels

```
adsl_test <- adsl |>
  dplyr::mutate(
    AGEGR1_f = factor(dplyr::if_else(AGE < 65, "<65", ">=65"), levels = c("<65", ">=65")),
    SAFFL_f = factor(dplyr::if_else(!is.na(TRTSDT), "Y", "N"), levels = c("Y", "N"))
  ) |>
  dplyr::select(USUBJID, AGE, AGEGR1_f, SAFFL_f, TRTSDT)
```

35 joins

The dplyr package provides functions to join tables based on shared keys, such as patient IDs. The main types of joins include:

- `inner_join()`
- `left_join()`
- `right_join()`
- `full_join()`
- `semi_join()`
- `anti_join()`

```
# Patient demographics data
patients <- data.frame(
  patient_id = c(101, 102, 103, 104),
  age = c(34, 45, 29, 56),
  gender = c("F", "M", "M", "F")
)

# Patient lab results data
lab_results <- data.frame(
  patient_id = c(103, 104, 105, 106),
  test = c("CBC", "Lipid Panel", "Blood Glucose", "CBC"),
  result = c("Normal", "High Cholesterol", "Elevated", "Normal")
)

# Display the data frames
patients
```

	patient_id	age	gender
1	101	34	F
2	102	45	M
3	103	29	M
4	104	56	F

```
lab_results
```

	patient_id	test	result
1	103	CBC	Normal
2	104	Lipid Panel	High Cholesterol
3	105	Blood Glucose	Elevated
4	106	CBC	Normal

35.1 inner join

`inner_join()`: returns only rows with matching patient IDs in both data frames.

```
# Inner join on patient_id
inner_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol

In this example, only patients with `patient_ids` 103 and 104 are in both tables, so only their information appears in the result.

35.2 left join

`left_join()` returns all rows from the patients data frame and matches rows from `lab_results` where possible. Unmatched rows in `lab_results` are filled with NA.

```
# Left join on patient_id
left_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol

Here, all patients from patients are included, with NA for lab results where no match is found.

35.3 right join

`right_join()` returns all rows from `lab_results`, matching rows from `patients` where possible. Unmatched rows in `patients` are filled with `NA`.

```
# Right join on patient_id
right_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol
3	105	NA	<NA>	Blood Glucose	Elevated
4	106	NA	<NA>	CBC	Normal

In this example, all patients with lab results are included, with demographic data from `patients` where available.

35.4 Full Join

`full_join()` returns all rows from both tables, with `NA` for missing matches in either table.

```
# Full join on patient_id
full_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol
5	105	NA	<NA>	Blood Glucose	Elevated
6	106	NA	<NA>	CBC	Normal

35.5 Semi Join

`semi_join()` returns only the rows in `patients` that have a match in `lab_results`, without bringing in columns from `lab_results`.

```
# Semi join on patient_id
semi_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender
1	103	29	M
2	104	56	F

Only patients with lab results are included here (patients 103 and 104).

35.6 Anti join

`anti_join()` returns only the rows in `patients` that do not have a match in `lab_results`.

```
# Anti join on patient_id
anti_join(patients, lab_results, by = "patient_id")
```

	patient_id	age	gender
1	101	34	F
2	102	45	M

```
#example
tab_a <- data.frame(
  id=c(001,002,003,004),
  age=c(25,50,30,40)
)

tab_b <- data.frame(
  id=c(001,002,004,005),
  sex=c("M","F","M","F")
)

#inner join:
injointab <- dplyr::inner_join(tab_a,tab_b,by="id")
#full join:
fulljointab <- full_join(tab_a,tab_b,by="id")
#left join
leftjointab <- left_join(x=tab_a,y=tab_b,by="id")
#right join
rightjointab <- right_join(x=tab_a,y=tab_b,by="id")
```

```
#anti_join  
antijointab <- anti_join(x=tab_a,y=tab_b,by="id")  
antijointab1 <- anti_join(x=tab_b,y=tab_a,by="id")
```

36 Reshaping data with tidyr

```
test <- data.frame(
  visit=c("wk0","wk2","wk4","wk6","wk12"),
  drug_a=c(10,20,30,40,50),
  drug_b=c(50,40,30,20,10),
  drug_c=c(15,25,35,45,55))
#tidyr::pivot_longer
library(tidyr)
test1 <- pivot_longer(data=test,
                      cols=c(drug_a,drug_b,drug_c),
                      names_to="drug",
                      values_to = "dose")

test2 <- pivot_wider(data=test1,
                    id_cols =visit,
                    names_prefix = "d_", # it is optional
                    values_from = dose,
                    names_from = drug)
```


37 counting with n()

When working with data, we often want to know the number of observations found for each factor or combination of factors. For this task, dplyr provides `count()`. For example, if we wanted to count the number of rows of data for each sex, we would do:

```
cnt <- adsl |>
  dplyr::count(SEX)

# race count by treatment
cnt2 <- adsl |>
  dplyr::count(TRT01P, RACE)

#treatment count
cnt3 <- adsl |>
  dplyr::filter(!is.na(TRT01P)) |>
  dplyr::count(TRT01P)

adae <- haven::read_sas("data/adam/adae.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

adsl <- haven::read_sas("data/adam/adsl.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

bign <- adsl |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::count(TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

bign1 <- dplyr::count(adsl, TRT01AN, TRT01A) |>
  dplyr::rename(TRTA = TRT01A, TRTAN = TRT01AN, bign = n)

adae1 <- adae |>
  dplyr::distinct(USUBJID, AEBODSYS, AEDECOD, TRTA, TRTAN, SAFFL)
```

```

ae_t <- adae1 |>
  dplyr::filter(SAFFL == "Y") |>
  dplyr::group_by(TRTA) |>
  dplyr::count(AEBODSYS, AEDECOD)

ae_t1 <- ae_t |>
  dplyr::left_join(bign, by = "TRTA") |>
  dplyr::mutate(
    pct = paste0("(", round(n / bign * 100, 2), "%"),
    val = paste0(n, " ", pct)
  )

ae2 <- pivot_wider(
  data = ae_t1,
  id_cols = c(AEBODSYS, AEDECOD),
  names_from = TRTA,
  values_from = val,
  values_fill = "0 (0.0)"
)

```

38 dplyr distinct()

The `distinct()` function in `dplyr` is used to return unique/distinct rows from a data frame or tibble. It removes duplicate rows based on the values in specified columns or all columns if none are specified.

```
# distinct
distinct_trt <- adsl |>
  dplyr::distinct(TRT01P, TRT01PN)
print(distinct_trt)
```

```
# A tibble: 3 x 2
  TRT01P          TRT01PN
  <chr>          <dbl>
1 Placebo          0
2 Xanomeline High Dose 81
3 Xanomeline Low Dose  54
```

39 case_when()

The `case_when()` function in `dplyr` is a powerful tool for creating new variables based on multiple conditions. It allows you to specify a series of conditions and corresponding values to assign when those conditions are met.

```
adsl1 <- adsl |>
  dplyr::mutate(
    agegrp1 = dplyr::case_when(
      AGE < 18 ~ "Child",
      AGE >= 18 & AGE < 65 ~ "Adult",
      AGE >= 65 ~ "Senior",
      TRUE ~ "Unknown" # Default case
    )
  ) |>
  dplyr::select(USUBJID, AGE, agegrp1) |>
  head(10)
```

40 create seq no.

```
class <- haven::read_sas("data/class.sas7bdat")

t <- dplyr::arrange(class, Age)
# Create t1 data set with first and last records for each unique value of 'age'

t1 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::mutate(seq = dplyr::row_number())

t2 <- class |>
  dplyr::mutate(seq = dplyr::row_number(), .by = Age) |>
  dplyr::arrange(Age)

t3 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == 1 )

t4 <- class |>
  dplyr::arrange(Age) |>
  dplyr::group_by(Age) |>
  dplyr::filter(dplyr::row_number() == n())

t5 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1 & row_number() == n())

t6 <- class |>
  dplyr::group_by(Age) |>
  dplyr::filter(row_number() == 1 | row_number() == n())
```

41 Lists: lapply, purrr

```
lst <- list(a=1:3, b=10:12)  
lapply(lst, mean)
```

```
$a  
[1] 2
```

```
$b  
[1] 11
```

42 Exercises

1. Using `across()`, standardize $(x - \text{mean}) / \text{sd}$ numeric columns.
2. Create a row-wise mean of `age` and `wt`.
3. Split `df` by `grp` and compute group means with `lapply` or `purrr::map`.

43 String Functions

44 String Functions in Base R

Strings (or character vectors) are a fundamental data type in R, and base R provides a variety of functions to manipulate and analyze strings. Here are some commonly used string functions in base R:

```
# Create a character vector
text <- c("Hello, World!", "R is great", "Data Science")
```

44.1 Basic String Functions

```
# nchar(): Returns the number of characters in a string
nchar(text)
```

```
[1] 13 10 12
```

```
# tolower(): Converts a string to lowercase
tolower(text)
```

```
[1] "hello, world!" "r is great"    "data science"
```

```
# toupper(): Converts a string to uppercase
toupper(text)
```

```
[1] "HELLO, WORLD!" "R IS GREAT"    "DATA SCIENCE"
```

```
# substr(): Extracts a substring from a string
substr(text, start = 1, stop = 5)
```

```
[1] "Hello" "R is " "Data "
```

```
# paste(): Concatenates strings together
paste("Hello", "R", sep = " ")
```

```
[1] "Hello R"
```

```
# paste0(): Concatenates strings without any separator
paste0("Hello", "R")
```

```
[1] "HelloR"
```

```
# trimws(): Trims leading and trailing whitespace from a string
trimws("  Hello, World!  ")
```

```
[1] "Hello, World!"
```

44.2 Pattern Matching and Replacement

```
# grep(): Searches for patterns in a character vector and returns the indices of matches
grep("R", text)
```

```
[1] 2
```

```
# grepl(): Searches for patterns and returns a logical vector indicating matches
grepl("R", text)
```

```
[1] FALSE TRUE FALSE
```

```
# sub(): Replaces the first occurrence of a pattern in a string
sub("great", "awesome", text)
```

```
[1] "Hello, World!" "R is awesome" "Data Science"
```

```
# gsub(): Replaces all occurrences of a pattern in a string
gsub(" ", "_", text)
```

```
[1] "Hello,_World!" "R_is_great" "Data_Science"
```

44.3 String Splitting and Joining

```
# strsplit(): Splits a string into substrings based on a specified delimiter
strsplit(text, split = " ")
```

```
[[1]]
[1] "Hello," "World!"

[[2]]
[1] "R"      "is"      "great"

[[3]]
[1] "Data"    "Science"
```

```
# unlist(): Converts a list to a vector (useful after strsplit)
unlist(strsplit(text, split = " "))
```

```
[1] "Hello," "World!" "R"      "is"      "great" "Data"    "Science"
```

44.4 Regular Expressions

```
# regexpr(): Finds the position of the first match of a pattern in a string
regexpr("R", text)
```

```
[1] -1  1 -1
attr("match.length")
[1] -1  1 -1
attr("index.type")
[1] "chars"
attr("useBytes")
[1] TRUE
```

```
# gregexpr(): Finds the positions of all matches of a pattern in a string
gregexpr(" ", text)
```

```
[[1]]
[1] 7
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[2]]
[1] 2 5
attr(,"match.length")
[1] 1 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[3]]
[1] 5
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
# regmatches(): Extracts the matched substrings based on the positions returned by regexpr or
regmatches(text, gregexpr(" ", text))
```

```
[[1]]
[1] " "
```

```
[[2]]
[1] " " " " "
```

```
[[3]]
[1] " "
```

44.5 Example Usage

```
# Example: Convert text to lowercase and replace spaces with underscores
cleaned_text <- tolower(gsub(" ", "_", text))
cleaned_text
```

```
[1] "hello,_world!" "r_is_great"    "data_science"
```

These functions provide a solid foundation for string manipulation in R. For more advanced string operations, you # using the **stringr** package, which offers a more consistent and user-friendly interface for string handling.

45 Overall differences

We'll begin with a lookup table between the most important stringr functions and their base R equivalents.

```
library(stringr)
data_stringr_base_diff <- tibble::tribble(
  ~stringr,
  "str_detect(string, pattern)",
  "str_dup(string, times)",
  "str_extract(string, pattern)",
  "str_extract_all(string, pattern)",
  "str_length(string)",
  "str_locate(string, pattern)",
  "str_locate_all(string, pattern)",
  "str_match(string, pattern)",
  "str_order(string)",
  "str_replace(string, pattern, replacement)",
  "str_replace_all(string, pattern, replacement)",
  "str_sort(string)",
  "str_split(string, pattern)",
  "str_sub(string, start, end)",
  "str_subset(string, pattern)",
  "str_to_lower(string)",
  "str_to_title(string)",
  "str_to_upper(string)",
  "str_trim(string)",
  "str_which(string, pattern)",
  "str_wrap(string)",
  ~base_r,
  "grepl(pattern, x)",
  "strrep(x, times)",
  "regmatches(x, m = regexpr(pattern, text))",
  "regmatches(x, m = gregexpr(pattern, text))",
  "nchar(x)",
  "regexpr(pattern, text)",
  "gregexpr(pattern, text)",
  "regmatches(x, m = regexec(pattern, text))",
  "order(...)",
  "sub(pattern, replacement, x)",
  "gsub(pattern, replacement, x)",
  "sort(x)",
  "strsplit(x, split)",
  "substr(x, start, stop)",
  "grep(pattern, x, value = TRUE)",
  "tolower(x)",
  "tools::toTitleCase(text)",
  "toupper(x)",
  "trimws(x)",
  "grep(pattern, x)",
  "strwrap(x)"
)

# create MD table, arranged alphabetically by stringr fn name
data_stringr_base_diff |>
  dplyr::mutate(dplyr::across(
    .cols = everything(),
    .fns = ~ paste0("`", .x, "`")
  ) |>
```

stringr	base R
<code>str_detect(string, pattern)</code>	<code>grepl(pattern, x)</code>
<code>str_dup(string, times)</code>	<code>strrep(x, times)</code>
<code>str_extract(string, pattern)</code>	<code>regmatches(x, m = regexpr(pattern, text))</code>
<code>str_extract_all(string, pattern)</code>	<code>regmatches(x, m = gregexpr(pattern, text))</code>
<code>str_length(string)</code>	<code>nchar(x)</code>
<code>str_locate(string, pattern)</code>	<code>regexpr(pattern, text)</code>
<code>str_locate_all(string, pattern)</code>	<code>gregexpr(pattern, text)</code>
<code>str_match(string, pattern)</code>	<code>regmatches(x, m = regexec(pattern, text))</code>
<code>str_order(string)</code>	<code>order(...)</code>
<code>str_replace(string, pattern, replacement)</code>	<code>sub(pattern, replacement, x)</code>
<code>str_replace_all(string, pattern, replacement)</code>	<code>gsub(pattern, replacement, x)</code>
<code>str_sort(string)</code>	<code>sort(x)</code>
<code>str_split(string, pattern)</code>	<code>strsplit(x, split)</code>
<code>str_sub(string, start, end)</code>	<code>substr(x, start, stop)</code>
<code>str_subset(string, pattern)</code>	<code>grep(pattern, x, value = TRUE)</code>
<code>str_to_lower(string)</code>	<code>tolower(x)</code>
<code>str_to_title(string)</code>	<code>tools::toTitleCase(text)</code>
<code>str_to_upper(string)</code>	<code>toupper(x)</code>
<code>str_trim(string)</code>	<code>trimws(x)</code>
<code>str_which(string, pattern)</code>	<code>grep(pattern, x)</code>
<code>str_wrap(string)</code>	<code>strwrap(x)</code>

```
dplyr::arrange(stringr) |>
dplyr::rename(`base R` = base_r) |>
gt::gt() |>
gt::fmt_markdown(columns = everything()) |>
gt::tab_options(column_labels.font.weight = "bold")
```

Overall the main differences between base R and stringr are:

1. stringr functions start with `str_` prefix; base R string functions have no consistent naming scheme.
2. The order of inputs is usually different between base R and stringr. In base R, the `pattern` to match usually comes first; in stringr, the `string` to manipulate always comes first. This makes stringr easier to use in pipes, and with `lapply()` or `purrr::map()`.
3. Functions in stringr tend to do less, where many of the string processing functions in base R have multiple purposes.

4. The output and input of stringr functions has been carefully designed. For example, the output of `str_locate()` can be fed directly into `str_sub()`; the same is not true of `regexpr()` and `substr()`.
5. Base functions use arguments (like `perl`, `fixed`, and `ignore.case`) to control how the pattern is interpreted. To avoid dependence between arguments, stringr instead uses helper functions (like `fixed()`, `regex()`, and `coll()`).

Next we'll walk through each of the functions, noting the similarities and important differences. These examples are adapted from the stringr documentation and here they are contrasted with the analogous base R operations.

46 Detect matches

46.1 `str_detect()`: Detect the presence or absence of a pattern in a string

Suppose you want to know whether each word in a vector of fruit names contains an “a”.

```
fruit <- c("apple", "banana", "pear", "pineapple")  
  
# base  
grepl(pattern = "a", x = fruit)
```

```
[1] TRUE TRUE TRUE TRUE
```

```
#stringr  
stringr::str_detect(fruit, pattern = "a")
```

```
[1] TRUE TRUE TRUE TRUE
```

In base you would use `grepl()` (see the “l” and think logical) while in `stringr` you use `str_detect()` (see the verb “detect” and think of a yes/no action).

46.2 `str_which()`: Find positions matching a pattern

Now you want to identify the positions of the words in a vector of fruit names that contain an “a”.

```
# base  
grep(pattern = "a", x = fruit)
```

```
[1] 1 2 3 4
```

```
# stringr
str_which(fruit, pattern = "a")
```

```
[1] 1 2 3 4
```

In base you would use `grep()` while in stringr you use `str_which()` (by analogy to `which()`).

46.3 `str_count()`: Count the number of matches in a string

How many “a”s are in each fruit?

```
# base
loc <- gregexpr(pattern = "a", text = fruit, fixed = TRUE)
sapply(loc, function(x) length(attr(x, "match.length")))
```

```
[1] 1 3 1 1
```

```
# stringr
str_count(fruit, pattern = "a")
```

```
[1] 1 3 1 1
```

This information can be gleaned from `gregexpr()` in base, but you need to look at the `match.length` attribute as the vector uses a length-1 integer vector (`-1`) to indicate no match.

46.4 `str_locate()`: Locate the position of patterns in a string

Within each fruit, where does the first “p” occur? Where are all of the “p”s?

```
fruit3 <- c("papaya", "lime", "apple")

# base
str(gregexpr(pattern = "p", text = fruit3))
```

List of 3

```
$ : int [1:2] 1 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int -1
..- attr(*, "match.length")= int -1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int [1:2] 2 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
```

```
# stringr
str_locate(fruit3, pattern = "p")
```

```
      start end
[1,]      1   1
[2,]     NA  NA
[3,]      2   2
```

```
str_locate_all(fruit3, pattern = "p")
```

```
[[1]]
      start end
[1,]      1   1
[2,]      3   3

[[2]]
      start end

[[3]]
      start end
[1,]      2   2
[2,]      3   3
```

46.5 str_subset(): Keep strings matching a pattern, or find positions

We may want to retrieve strings that contain a pattern of interest:

```
# base
grep(pattern = "g", x = fruit, value = TRUE)
```

```
character(0)
```

```
# stringr
str_subset(fruit, pattern = "g")
```

```
character(0)
```

46.6 str_extract(): Extract matching patterns from a string

We may want to pick out certain patterns from a string, for example, the digits in a shopping list:

```
shopping_list <- c("apples x4", "bag of flour", "10", "milk x2")

# base
matches <- regexpr(pattern = "\\d+", text = shopping_list) # digits
regmatches(shopping_list, m = matches)
```

```
[1] "4" "10" "2"
```

```
matches <- gregexpr(pattern = "[a-z]+", text = shopping_list) # words
regmatches(shopping_list, m = matches)
```

```
[[1]]
[1] "apples" "x"
```

```
[[2]]
[1] "bag" "of" "flour"
```

```
[[3]]
character(0)
```

```
[[4]]
[1] "milk" "x"
```

```
# stringr
str_extract(shopping_list, pattern = "\\d+")
```

```
[1] "4" NA "10" "2"
```

```
str_extract_all(shopping_list, "[a-z]+")
```

```
[[1]]
```

```
[1] "apples" "x"
```

```
[[2]]
```

```
[1] "bag" "of" "flour"
```

```
[[3]]
```

```
character(0)
```

```
[[4]]
```

```
[1] "milk" "x"
```

Base R requires the combination of `regexpr()` with `regmatches()`; but note that the strings without matches are dropped from the output. `stringr` provides `str_extract()` and `str_extract_all()`, and the output is always the same length as the input.

46.7 `str_match()`: Extract matched groups from a string

We may also want to extract groups from a string. Here I'm going to use the scenario from Section 14.4.3 in [R for Data Science](#).

```
head(sentences)
```

```
[1] "The birch canoe slid on the smooth planks."
```

```
[2] "Glue the sheet to the dark blue background."
```

```
[3] "It's easy to tell the depth of a well."
```

```
[4] "These days a chicken leg is a rare dish."
```

```
[5] "Rice is often served in round bowls."
```

```
[6] "The juice of lemons makes fine punch."
```

```
noun <- "([A]a|[Tt]he) ([^ ]+)"

# base
matches <- regexec(pattern = noun, text = head(sentences))
do.call("rbind", regmatches(x = head(sentences), m = matches))
```

```
      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] "The juice" "The" "juice"
```

```
# stringr
str_match(head(sentences), pattern = noun)
```

```
      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] NA         NA    NA
[5,] NA         NA    NA
[6,] "The juice" "The" "juice"
```

As for extracting the full match base R requires the combination of two functions, and inputs with no matches are dropped from the output.

47 Manage lengths

47.1 `str_length()`: The length of a string

To determine the length of a string, base R uses `nchar()` (not to be confused with `length()` which gives the length of vectors, etc.) while `stringr` uses `str_length()`.

```
# base
nchar(letters)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
# stringr
str_length(letters)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

There are some subtle differences between base and `stringr` here. `nchar()` requires a character vector, so it will return an error if used on a factor. `str_length()` can handle a factor input.

```
# base
nchar(factor("abc"))
```

```
Error in nchar(factor("abc")): 'nchar()' requires a character vector
```

```
# stringr
str_length(factor("abc"))
```

```
[1] 3
```

Note that “characters” is a poorly defined concept, and technically both `nchar()` and `str_length()` returns the number of code points. This is usually the same as what you’d consider to be a character, but not always:

```
x <- c("\u00fc", "u\u0308")
x
```

```
[1] "ü" "ü"
```

```
nchar(x)
```

```
[1] 1 2
```

```
str_length(x)
```

```
[1] 1 2
```

47.2 str_pad(): Pad a string

To pad a string to a certain width, use stringr's `str_pad()`. In base R you could use `sprintf()`, but unlike `str_pad()`, `sprintf()` has many other functionalities.

```
# base
sprintf("%30s", "Sriram")
```

```
[1] "                               Sriram"
```

```
sprintf("%-30s", "Sriram")
```

```
[1] "Sriram                               "
```

```
# "both" is not as straightforward
```

```
# stringr
rbind(
  str_pad("Sriram", 30, "left"),
  str_pad("Sriram", 30, "right"),
  str_pad("Sriram", 30, "both")
)
```

```
      [,1]
[1,] "                               Sriram"
[2,] "Sriram                               "
[3,] "          Sriram                               "
```


47.3 `str_trunc()`: Truncate a character string

The `stringr` package provides an easy way to truncate a character string: `str_trunc()`. Base R has no function to do this directly.

```
x <- "This string is moderately long"

# stringr
rbind(
  str_trunc(x, 20, "right"),
  str_trunc(x, 20, "left"),
  str_trunc(x, 20, "center")
)
```

```
      [,1]
[1,] "This string is mo..."
[2,] "...s moderately long"
[3,] "This stri...ely long"
```

47.4 `str_trim()`: Trim whitespace from a string

Similarly, `stringr` provides `str_trim()` to trim whitespace from a string. This is analogous to base R's `trimws()` added in R 3.3.0.

```
# base
trimws(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
trimws("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

```
# stringr
str_trim(" String with trailing and leading white space\t")
```

```
[1] "String with trailing and leading white space"
```

```
str_trim("\n\nString with trailing and leading white space\n\n")
```

```
[1] "String with trailing and leading white space"
```

The stringr function `str_squish()` allows for extra whitespace within a string to be trimmed (in contrast to `str_trim()` which removes whitespace at the beginning and/or end of string). In base R, one might take advantage of `gsub()` to accomplish the same effect.

```
# stringr
str_squish(" String with trailing, middle,   and leading white space\t")
```

```
[1] "String with trailing, middle, and leading white space"
```

```
str_squish("\n\nString with excess, trailing and leading white space\n\n")
```

```
[1] "String with excess, trailing and leading white space"
```

47.5 str_wrap(): Wrap strings into nicely formatted paragraphs

`strwrap()` and `str_wrap()` use different algorithms. `str_wrap()` uses the famous [Knuth-Plass algorithm](#).

```
gettysburg <- "Four score and seven years ago our fathers brought forth on this continent, a  
# base  
cat(strwrap(gettysburg, width = 60), sep = "\n")
```

```
Four score and seven years ago our fathers brought forth on  
this continent, a new nation, conceived in Liberty, and  
dedicated to the proposition that all men are created  
equal.
```

```
# stringr  
cat(str_wrap(gettysburg, width = 60), "\n")
```

```
Four score and seven years ago our fathers brought forth  
on this continent, a new nation, conceived in Liberty, and  
dedicated to the proposition that all men are created equal.
```

Note that `strwrap()` returns a character vector with one element for each line; `str_wrap()` returns a single string containing line breaks.

48 Mutate strings

48.1 `str_replace()`: Replace matched patterns in a string

To replace certain patterns within a string, `stringr` provides the functions `str_replace()` and `str_replace_all()`. The base R equivalents are `sub()` and `gsub()`. Note the difference in default input order again.

```
fruits <- c("apple", "banana", "pear", "pineapple")
```

```
# base  
sub("[aeiou]", "-", fruits)
```

```
[1] "-pple"      "b-nana"     "p-ar"       "p-neapple"
```

```
gsub("[aeiou]", "-", fruits)
```

```
[1] "-ppl-"      "b-n-n-"     "p--r"       "p-n--ppl-"
```

```
# stringr  
str_replace(fruits, "[aeiou]", "-")
```

```
[1] "-pple"      "b-nana"     "p-ar"       "p-neapple"
```

```
str_replace_all(fruits, "[aeiou]", "-")
```

```
[1] "-ppl-"      "b-n-n-"     "p--r"       "p-n--ppl-"
```

48.2 case: Convert case of a string

Both `stringr` and base R have functions to convert to upper and lower case. Title case is also provided in `stringr`.

```
dog <- "The quick brown dog"
```

```
# base  
toupper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
tolower(dog)
```

```
[1] "the quick brown dog"
```

```
tools::toTitleCase(dog)
```

```
[1] "The Quick Brown Dog"
```

```
# stringr  
str_to_upper(dog)
```

```
[1] "THE QUICK BROWN DOG"
```

```
str_to_lower(dog)
```

```
[1] "the quick brown dog"
```

```
str_to_title(dog)
```

```
[1] "The Quick Brown Dog"
```

In stringr we can control the locale, while in base R locale distinctions are controlled with global variables. Therefore, the output of your base R code may vary across different computers with different global settings.

```
# stringr  
str_to_upper("i") # English
```

```
[1] "I"
```

```
str_to_upper("i", locale = "tr") # Turkish
```

```
[1] "i"
```

49 Join and split

49.1 `str_flatten()`: Flatten a string

If we want to take elements of a string vector and collapse them to a single string we can use the `collapse` argument in `paste()` or use `stringr`'s `str_flatten()`.

```
# base
paste0(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

```
# stringr
str_flatten(letters, collapse = "-")
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

The advantage of `str_flatten()` is that it always returns a vector the same length as its input; to predict the return length of `paste()` you must carefully read all arguments.

49.2 `str_dup()`: duplicate strings within a character vector

To duplicate strings within a character vector use `strrep()` (in R 3.3.0 or greater) or `str_dup()`:

```
fruit <- c("apple", "pear", "banana")

# base
strrep(fruit, 2)
```

```
[1] "appleapple"  "pearpear"    "bananabanana"
```

```
strrep(fruit, 1:3)
```

```
[1] "apple"          "pearpear"       "bananabanabanana"
```

```
# stringr  
str_dup(fruit, 2)
```

```
[1] "appleapple"    "pearpear"      "bananabanana"
```

```
str_dup(fruit, 1:3)
```

```
[1] "apple"          "pearpear"       "bananabanabanana"
```

49.3 str_split(): Split up a string into pieces

To split a string into pieces with breaks based on a particular pattern match stringr uses `str_split()` and base R uses `strsplit()`. Unlike most other functions, `strsplit()` starts with the character vector to modify.

```
fruits <- c(  
  "apples and oranges and pears and bananas",  
  "pineapples and mangos and guavas"  
)  
# base  
strsplit(fruits, " and ")
```

```
[[1]]  
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]  
[1] "pineapples" "mangos"    "guavas"
```

```
# stringr  
str_split(fruits, " and ")
```



```
[[1]]
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

The `stringr` package's `str_split()` allows for more control over the split, including restricting the number of possible matches.

```
# stringr
str_split(fruits, " and ", n = 3)
```

```
[[1]]
[1] "apples"          "oranges"          "pears and bananas"
```

```
[[2]]
[1] "pineapples" "mangos"          "guavas"
```

```
str_split(fruits, " and ", n = 2)
```

```
[[1]]
[1] "apples"          "oranges and pears and bananas"
```

```
[[2]]
[1] "pineapples"      "mangos and guavas"
```

49.4 `str_glue()`: Interpolate strings

It's often useful to interpolate varying values into a fixed string. In base R, you can use `sprintf()` for this purpose; `stringr` provides a wrapper for the more general purpose [glue](#) package.

```
name <- "Fred"
age <- 50
anniversary <- as.Date("1991-10-12")

# base
sprintf(
  "My name is %s my age next year is %s and my anniversary is %s.",
```

```
name,  
age + 1,  
format(anniversary, "%A, %B %d, %Y")  
)
```

```
[1] "My name is Fred my age next year is 51 and my anniversary is Saturday, October 12, 1991"
```

```
# stringr  
str_glue(  
  "My name is {name}, ",  
  "my age next year is {age + 1}, ",  
  "and my anniversary is {format(anniversary, '%A, %B %d, %Y')}." )
```

```
My name is Fred, my age next year is 51, and my anniversary is Saturday, October 12, 1991.
```

50 Order strings

50.1 `str_order()`: Order or sort a character vector

Both base R and stringr have separate functions to order and sort strings.

```
# base  
order(letters)
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  
[26] 26
```

```
sort(letters)
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
[20] "t" "u" "v" "w" "x" "y" "z"
```

```
# stringr  
str_order(letters)
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  
[26] 26
```

```
str_sort(letters)
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"  
[20] "t" "u" "v" "w" "x" "y" "z"
```

Some options in `str_order()` and `str_sort()` don't have analogous base R options. For example, the stringr functions have a `locale` argument to control how to order or sort. In base R the locale is a global setting, so the outputs of `sort()` and `order()` may differ across different computers. For example, in the Norwegian alphabet, å comes after z:

```
x <- c("â", "a", "z")
str_sort(x)
```

```
[1] "a" "â" "z"
```

```
str_sort(x, locale = "no")
```

```
[1] "a" "z" "â"
```

The stringr functions also have a **numeric** argument to sort digits numerically instead of treating them as strings.

```
# stringr
x <- c("100a10", "100a5", "2b", "2a")
str_sort(x)
```

```
[1] "100a10" "100a5" "2a" "2b"
```

```
str_sort(x, numeric = TRUE)
```

```
[1] "2a" "2b" "100a5" "100a10"
```

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```

ae <- haven::read_sas("./data/sdtm/ae.sas7bdat")
ae1 <- ae |>
  select (USUBJID, AEDECOD, AEBODSYS)

#find function in sas, str_detect

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HER"))

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "^HIATUS ")) # start of the string

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HERNIA$")) # end of the string

#substr in sas

ae2 <- ae1 |>
  mutate(newvar1=str_sub(AEDECOD, 1, 6),
         newvar2=str_sub(AEDECOD, 2, 6),
         newvar3=str_sub(AEDECOD, -3),
         newvar4=str_length(AEDECOD))

#cat function in sas , Str_c in r

ae2 <- ae1 |>
  mutate(newvar1=str_c(AEDECOD, AEBODSYS, sep="/"),
         newvar2=paste(AEDECOD, AEBODSYS, sep="/"))

# scan function in sas, word in r

ae2 <- ae1 |>
  mutate(newvar1=word(AEDECOD, 1),
         newvar2=word(AEDECOD, 2))

# upper & lower case

ae2 <- ae1 |>
  mutate(newvar1=str_to_lower(AEDECOD),
         newvar2=str_to_upper(AEDECOD),
         newvar3=str_to_title(AEDECOD),
         newvar4=str_to_sentence(AEDECOD))

```

```
#str_trim  
  
a <- "  this is    my String  "  
b <- str_trim(a)  
c <- str_replace_all(a," "," ")  
d <- str_squish(a)
```

51 date and time functions

51.0.1 Date and Time Base Functions in R

Date and time manipulation is a common task in data analysis. R provides several base functions to work with dates and times. Here are some of the most commonly used base R functions for handling date and time data:

```
# Get the current date
current_date <- Sys.Date()
print(current_date)
```

```
[1] "2025-11-28"
```

```
# Get the current date and time
current_datetime <- Sys.time()
print(current_datetime)
```

```
[1] "2025-11-28 17:05:54 UTC"
```

51.0.2 Creating Date and Time Objects

51.0.3 Year Formats

Code	Meaning	Example (<code>format(dt, ...)</code>)
%Y	4-digit year	"2025"
%y	2-digit year (00–99)	"25"
%C	Century (year / 100, 00–99)	"20" (for 2025; OS-dep.)

51.0.4 Month Formats

Code	Meaning	Example
%m	Month number (01–12)	"11"
%b	Abbrev. month name (locale)	"Nov"
%B	Full month name (locale)	"November"
%h	Same as %b (abbrev. month name)	"Nov"

51.0.5 Day Formats

Code	Meaning	Example
%d	Day of month (01–31)	"14"
%e	Day of month (1–31, padded with space)	"14" (or " 7" for 7th; OS-dep.)
%j	Day of year (001–366)	"318"

51.0.6 Weekday Formats

Code	Meaning	Example
%a	Abbrev. weekday name (locale)	"Fri"
%A	Full weekday name (locale)	"Friday"
%w	Weekday number (0–6, 0 = Sunday)	"5"
%u	ISO weekday number (1–7, 1 = Monday) (OS-dep.)	"5" (Friday)

51.0.7 Time Formats

Code	Meaning	Example
%H	Hour (00–23)	"14"
%I	Hour (01–12)	"02"
%M	Minute (00–59)	"30"
%S	Second (00–61, allows for leap)	"05"
%p	AM/PM indicator (locale)	"PM"

###Locale-dependent “full date/time” codes

###These depend on your system locale (language + region):

Code	Meaning	Example
%c	Date and time representation	"Fri Nov 14 16:05:30 2025"
%x	Date representation	"11/14/25" (varies by locale)
%X	Time representation	"16:05:30"

```
# Create a Date object
dt <- as.Date("2025-11-14")
# Create a POSIXct object (date-time)
dt <- as.POSIXct("2025-11-14 16:05:30")
# Format examples
format(dt, "%Y-%m-%d") # e.g. "2025-11-14"
```

```
[1] "2025-11-14"
```

```
format(dt, "%m/%d/%y") # e.g. "11/14/25"
```

```
[1] "11/14/25"
```

```
format(dt, "%B %d, %Y") # e.g. "November 14, 2025"
```

```
[1] "November 14, 2025"
```

```
format(dt, "%c") # e.g. "Fri Nov 14 16:05:30 2025"
```

```
[1] "Fri Nov 14 16:05:30 2025"
```

```
format(dt, "%x") # e.g. "11/14/25" (depends on locale)
```

```
[1] "11/14/25"
```

```
format(dt, "%X") # e.g. "16:05:30"
```

```
[1] "16:05:30"
```

SAS informat	R parsing
ddmmyy10.	as.Date(x, "%d/%m/%Y")
ddmmyyd10.	as.Date(x, "%d-%m-%Y")
ddmmyyp10.	as.Date(x, "%d.%m.%Y")
mmddyy10.	as.Date(x, "%m/%d/%Y")
date9.	as.Date(x, "%d%b%Y")
julian7.	as.Date(strptime(x, "%Y%j"))
time8.	as.POSIXct(x, "%H:%M:%S")
time10.	as.POSIXct(x, "%I:%M:%S%p") (if am/pm)
datetime18.	as.POSIXct(x, "%d%b%Y:%H:%M:%S")
datetime20.	as.POSIXct(x, "%d%b%Y:%I:%M:%S%p")

SAS format	Meaning	R equivalent
worddate18.	Month date, year (e.g., February 23, 2003)	format(d, "%B %d, %Y")
weekdate24.	Weekday, month date, year	format(d, "%A, %B %d, %Y")
year.	numeric year	as.integer(format(d, "%Y"))
month.	month number (1–12)	as.integer(format(d, "%m"))
day.	day of month	as.integer(format(d, "%d"))
weekday.	day of week (1–7)	as.integer(format(d, "%u")) (Mon = 1)

```
# Convert a character string to a Date object
date_string <- "2023-10-01"
date_object <- as.Date(date_string)
print(date_object)
```

```
[1] "2023-10-01"
```

```
# Convert a character string to a POSIXct object (date-time)
datetime_string <- "2023-10-01 12:34:56"
datetime_object <- as.POSIXct(datetime_string)
print(datetime_object)
```

```
[1] "2023-10-01 12:34:56 UTC"
```

```
# Extract components from a Date object
year <- format(date_object, "%Y")
month <- format(date_object, "%m")
day <- format(date_object, "%d")
print(paste("Year:", year, "Month:", month, "Day:", day))
```

```
[1] "Year: 2023 Month: 10 Day: 01"
```

```
# Extract components from a POSIXct object
hour <- format(datetime_object, "%H")
minute <- format(datetime_object, "%M")
second <- format(datetime_object, "%S")
print(paste("Hour:", hour, "Minute:", minute, "Second:", second))
```

```
[1] "Hour: 12 Minute: 34 Second: 56"
```

```
# Perform date arithmetic
tomorrow <- current_date + 1
yesterday <- current_date - 1
print(paste("Tomorrow:", tomorrow, "Yesterday:", yesterday))
```

```
[1] "Tomorrow: 2025-11-29 Yesterday: 2025-11-27"
```

```
# Calculate the difference between two dates
date_diff <- as.numeric(difftime(current_date, date_object, units = "days"))
print(paste("Difference in days:", date_diff))
```

```
[1] "Difference in days: 789"
```

```
# Format a Date object to a specific string format
formatted_date <- format(current_date, "%B %d, %Y")
print(formatted_date)
```

```
[1] "November 28, 2025"
```

```
# Format a POSIXct object to a specific string format
formatted_datetime <- format(current_datetime, "%Y-%m-%d %H:%M:%S")
print(formatted_datetime)
```

```
[1] "2025-11-28 17:05:54"
```

These functions allow you to create, manipulate, and format date and time data in R. For more advanced date and time handling, you might consider using the **lubridate** package, which provides a more user-friendly interface for working with dates and times.

A practical, CDISC-friendly guide to working with **dates/times** and implementing **partial date imputation** in R using **base R** and **lubridate**.

51.1 1. Date & Time Basics in R

51.1.1 1.1 Core classes

- **Date**: calendar dates without time.
- **POSIXct**: seconds since 1970-01-01 (compact, good for storage/arithmetic).
- **POSIXlt**: list of components (year, mon, mday, hour, min, sec) — convenient to extract parts.

51.1.2 1.2 Converting numerics to Date (origins matter)

Different systems use different **origins** for numeric dates: - Excel: 1900-01-01 (beware 1900 leap-year bug in older Excels) - SAS: 1960-01-01 - R: 1970-01-01

```
num_dates <- c(0, 159, 464, 15674)

as.Date(num_dates, origin = "1960-01-01") # SAS origin
```

```
[1] "1960-01-01" "1960-06-08" "1961-04-09" "2002-11-30"
```

```
as.Date(num_dates, origin = "1970-01-01") # R origin
```

```
[1] "1970-01-01" "1970-06-09" "1971-04-10" "2012-11-30"
```

```
as.Date(num_dates, origin = "1900-01-01") # Excel origin
```

```
[1] "1900-01-01" "1900-06-09" "1901-04-10" "1942-12-01"
```

51.1.3 1.3 Custom date formats

```
x1 <- c("01/05/1965", "08/16/1975")
as.Date(x1, "%m/%d/%Y")

x2 <- c("05-01-1965", "16-08-1975")
as.Date(x2, "%d-%m-%Y")

x3 <- c("01MAY1965", "16AUG1975")
as.Date(x3, "%d%b%Y")

# Unknown format? Try multiple patterns
x4 <- c("01MAY1965MORNING", "16AUGUST1975")
as.Date(x4, tryFormats = c("%d%b%Y", "%d%B%Y"))
```

51.1.4 1.4 Datetime strings (ISO 8601 & time zones)

```
x <- c("2015-10-19T10:15", "2018-12-19T23:05")
# Keep the literal 'T' in the format string
as.POSIXct(x, format = "%Y-%m-%dT%H:%M", tz = "UTC")

# Extract parts
xt <- as.POSIXlt(x, format = "%Y-%m-%dT%H:%M")
format(xt, "%Y-%m-%d") # date part
format(xt, "%H:%M")   # time part
```

51.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic

Install & load: `install.packages("lubridate"); library(lubridate)`

51.2.1 2.1 Intuitive parsers

- `ymd()`, `mdy()`, `dmy()` for dates
- `ymd_hms()`, `ymd_hm()`, `ymd_h()` for date-times

```
ymd("20210529")
dmy("29/05/2021")
ymd_hm("2019/01/19T17:15") # delimiter agnostic
```

51.2.2 2.2 Accessors and rounding

```
x <- ymd_hms("2009-08-03 12:01:59")
year(x); month(x); mday(x); wday(x); hour(x); minute(x); second(x)

floor_date(x, "month") # 2009-08-01
ceiling_date(x, "month") # 2009-09-01
round_date(x, "month") # 2009-08-01
rollback(x) # last day of previous month
```

51.2.3 2.3 Timespans

- **Durations:** exact seconds, e.g., `ddays(2)`
- **Periods:** human units, e.g., `months(1)`
- **Intervals:** start-end pair, e.g., `interval(start, end)`

```
start <- ymd("2015-10-19")
end <- ymd("2018-12-19")
int <- interval(start, end)
as.duration(int) # exact seconds
as.period(int) # years/months/days
```

51.2.4 3 ADaM-Style Partial Date Imputation (Start & End)

Imputation rules vary by study; always follow the SAP. A common pattern:

51.2.5 Start date (AESTDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **day** = **01** (first of month)
- YYYY → impute **month** = **01**, **day** = **01** (01-Jan)

51.2.6 End date (AEENDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **last day of that month**
- YYYY → impute **31-Dec**

51.2.7 Censoring by death / last alive date

Additionally, bound AEENDT by death / last alive date (when present):

- CENSOR = coalesce(DTHDT, LASTALVDT)
- AEENDT = min(imputed_end, CENSOR)

51.2.8 Imputation flags

Add flags:

- AESTDTF, AEENDTF:
 - "D" → day imputed
 - "M,D" → month & day imputed
- Add "C" if we had to **censor the end date** at DTHDT / LASTALVDT.

```
library(dplyr, warn.conflicts = FALSE)
library(stringr, warn.conflicts = FALSE)
library(lubridate, warn.conflicts = FALSE)

df <- data.frame(
  USUBJID   = sprintf("01-001-%03d", 1:6),
  AESTDTC   = c("2015-10-19", "2015-10", "2015", "2019-02", "2020-02", "2018-12-31"),
  AEENDTC   = c("2018-12-19", "2018-10", "2017-05-29", "2019-02", "2020-02", "2020"),
  RANDDT    = ymd(c("2015-10-01", "2015-10-01", "2015-01-10", "2019-01-15", "2020-02-10", "2020-02-10")),
  DTHDT     = ymd(c(NA, NA, NA, NA, "2020-08-03", NA)),
  LASTALVDT = ymd(c(NA, NA, NA, NA, NA, "2020-12-15"))
)
```

```

df_imp <- df |>
mutate(
  # Length of partial date strings
  AESTDTC_LEN = str_length(AESTDTC),
  AEENDTC_LEN = str_length(AEENDTC),
  ## --- AE start date imputation (minimum: first possible date) ---
  AESTDT = case_when(
    is.na(AESTDTC) ~ as.Date(NA),
    AESTDTC_LEN == 10 ~ ymd(AESTDTC, quiet = TRUE),
    AESTDTC_LEN == 7 ~ ymd(str_c(AESTDTC, "-01"), quiet = TRUE),
    AESTDTC_LEN == 4 ~ ymd(str_c(AESTDTC, "-01-01"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AESTDTF = case_when(
    is.na(AESTDTC) ~ NA_character_,
    AESTDTC_LEN == 10 ~ "",
    AESTDTC_LEN == 7 ~ "D",
    AESTDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  ),
  ## --- AE end date imputation (maximum: latest possible date) ---
  AEENDT_RAW = case_when(
    is.na(AEENDTC) ~ as.Date(NA),
    AEENDTC_LEN == 10 ~ ymd(AEENDTC, quiet = TRUE),
    AEENDTC_LEN == 7 ~ ymd(str_c(AEENDTC, "-01"), quiet = TRUE),
    AEENDTC_LEN == 4 ~ ymd(str_c(AEENDTC, "-12-31"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AEENDTF_RAW = case_when(
    is.na(AEENDTC) ~ NA_character_,
    AEENDTC_LEN == 10 ~ "",
    AEENDTC_LEN == 7 ~ "D",
    AEENDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  )
) %>%
mutate(
  AEENDT_RAW = if_else(
    AEENDTC_LEN == 7 & !is.na(AEENDT_RAW),
    ceiling_date(AEENDT_RAW, "month") - days(1),
    AEENDT_RAW
  ),

```



```

CENSOR = coalesce(DTHDT, LASTALVDT),
AEENDT = case_when(
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR ~ CENSOR,
  TRUE ~ AEENDT_RAW
),
AEENDTF = case_when(
  is.na(AEENDT) ~ AEENDTF_RAW,
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW == "" ~ "C",
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW != "" ~ str_c(
  TRUE ~ AEENDTF_RAW
)
)

```

51.3 4. Durations & Analysis Variables

51.3.1 4.1 AE duration (AEDUR) in days

52 Using difftime (base) or as.numeric on Date differences

```
AEDUR <- as.integer(out$AEENDT - out$ASTDT)
```

52.0.1 6.2 Handy lubridate parsers (subset)

- Dates: `ymd()`, `mdy()`, `dmy()`, `yq()`
 - Datetimes: `ymd_hms()`, `ymd_hm()`, `ymd_h()`
 - Times: `hms()`, `hm()`, `ms()`
 - Accessors: `year()`, `month()`, `mday()`, `wday()`, `hour()`, `minute()`, `second()`, `date()`
 - Rounding: `floor_date()`, `ceiling_date()`, `round_date()`, `rollback()`
-

53 Reading SAS Datasets (+ Cleaning)

```
library(haven)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(labelled)
```

We try to read a SAS dataset (e.g., SDTM DM). If not present, we **synthesize** an example.

```
dm_path <- "data/sdtm/dm.sas7bdat"

if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:10),
    ARM = rep(c("Placebo", "Active"), length.out=10),
    AGE = c(55, 62, 47, 50, 71, 66, 45, 59, 53, 68),
    SEX = rep(c("M", "F"), length.out=10)
  )
  message("Synthesized `dm` since data/sdtm/dm.sas7bdat was not found.")
}
str(dm)
```

```

tibble [306 x 25] (S3: tbl_df/tbl/data.frame)
 $ STUDYID : chr [1:306] "CDISCPIL0T01" "CDISCPIL0T01" "CDISCPIL0T01" "CDISCPIL0T01" ...
  ..- attr(*, "label")= chr "Study Identifier"
 $ DOMAIN  : chr [1:306] "DM" "DM" "DM" "DM" ...
  ..- attr(*, "label")= chr "Domain Abbreviation"
 $ USUBJID : chr [1:306] "01-701-1015" "01-701-1023" "01-701-1028" "01-701-
1033" ...
  ..- attr(*, "label")= chr "Unique Subject Identifier"
 $ SUBJID  : chr [1:306] "1015" "1023" "1028" "1033" ...
  ..- attr(*, "label")= chr "Subject Identifier for the Study"
 $ RFSTDTC : chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Subject Reference Start Date/Time"
 $ RFENDTC : chr [1:306] "2014-07-02" "2012-09-02" "2014-01-14" "2014-04-
14" ...
  ..- attr(*, "label")= chr "Subject Reference End Date/Time"
 $ RFXSTDTC: chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
  ..- attr(*, "label")= chr "Date/Time of First Study Treatment"
 $ RFXENDTC: chr [1:306] "2014-07-02" "2012-09-01" "2014-01-14" "2014-03-
31" ...
  ..- attr(*, "label")= chr "Date/Time of Last Study Treatment"
 $ RFICDTC : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Informed Consent"
 $ RFPENDTC: chr [1:306] "2014-07-02T11:45" "2013-02-18" "2014-01-14T11:10" "2014-
09-15" ...
  ..- attr(*, "label")= chr "Date/Time of End of Participation"
 $ DTHDTC  : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Date/Time of Death"
 $ DTHFL   : chr [1:306] "" "" "" "" ...
  ..- attr(*, "label")= chr "Subject Death Flag"
 $ SITEID  : chr [1:306] "701" "701" "701" "701" ...
  ..- attr(*, "label")= chr "Study Site Identifier"
 $ AGE      : num [1:306] 63 64 71 74 77 85 59 68 81 84 ...
  ..- attr(*, "label")= chr "Age"
 $ AGEU     : chr [1:306] "YEARS" "YEARS" "YEARS" "YEARS" ...
  ..- attr(*, "label")= chr "Age Units"
 $ SEX      : chr [1:306] "F" "M" "M" "M" ...
  ..- attr(*, "label")= chr "Sex"
 $ RACE     : chr [1:306] "WHITE" "WHITE" "WHITE" "WHITE" ...
  ..- attr(*, "label")= chr "Race"
 $ ETHNIC   : chr [1:306] "HISPANIC OR LATINO" "HISPANIC OR LATINO" "NOT HISPANIC OR LATINO" ...
  ..- attr(*, "label")= chr "Ethnicity"

```

```

$ ARMCD : chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
..- attr(*, "label")= chr "Planned Arm Code"
$ ARM : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
..- attr(*, "label")= chr "Description of Planned Arm"
$ ACTARMCD: chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
..- attr(*, "label")= chr "Actual Arm Code"
$ ACTARM : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
..- attr(*, "label")= chr "Description of Actual Arm"
$ COUNTRY : chr [1:306] "USA" "USA" "USA" "USA" ...
..- attr(*, "label")= chr "Country"
$ DMDTC : chr [1:306] "2013-12-26" "2012-07-22" "2013-07-11" "2014-03-
10" ...
..- attr(*, "label")= chr "Date/Time of Collection"
$ DMDY : num [1:306] -7 -14 -8 -8 -7 -21 NA -9 -13 -7 ...
..- attr(*, "label")= chr "Study Day of Collection"

```

53.1 Handling Labels & Missing

```

# Example: Convert blank strings "" to NA for character columns
convert_blanks_to_na <- function(x) {
  if (is.character(x)) x[x == ""] <- NA_character_
  x
}
dm <- dm |> mutate(across(where(is.character), convert_blanks_to_na))

```

53.2 Labelled to Factor (if needed)

```

if (inherits(dm$SEX, "labelled")) {
  dm <- dm |> mutate(SEX = to_factor(SEX))
}

```

53.3 Common Cleaning

```
dm <- dm |>
  mutate(
    AGEGR1 = cut(AGE, breaks=c(-Inf, 50, 65, Inf),
                 labels=c("<50", "50-65", "65+"))
  )
```

54 Exercises

1. Read another SAS dataset (e.g., `sv.sas7bdat`) if available. If not, create a synthetic tibble.
2. Write a function to trim character whitespace for all character columns.
3. Make a clean factor for `ARM` with levels `Placebo < Active`.

55 Base R Functions & Apply Family

56 Common Utilities

```
x <- 1:10  
sum(x); mean(x); sd(x); var(x); quantile(x)
```

```
[1] 55
```

```
[1] 5.5
```

```
[1] 3.02765
```

```
[1] 9.166667
```

0%	25%	50%	75%	100%
1.00	3.25	5.50	7.75	10.00

```
seq(0, 1, by=0.1); rep(5, times=3)
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
[1] 5 5 5
```

57 Apply Family

```
m <- matrix(1:9, nrow=3)
apply(m, 1, mean) # row means
```

```
[1] 4 5 6
```

```
apply(m, 2, mean) # col means
```

```
[1] 2 5 8
```

```
lst <- list(a=1:3, b=10:12)
sapply(lst, mean) # simplifies result
```

```
  a  b
2 11
```

```
mapply(sum, 1:3, 10:12)
```

```
[1] 11 13 15
```

58 Subsetting Essentials

```
df <- data.frame(id=1:3, val=c(10,20,30))  
df[1, "val"]
```

```
[1] 10
```

```
df[df$val > 10, ]
```

```
  id val  
2  2  20  
3  3  30
```

59 Exercises

1. Use `apply` to get the max per column of a numeric matrix.
2. Write a base R snippet to compute IQR for each column of `mtcars`.
3. Compare `lapply` vs `sapply` in behavior on a list with mixed types.

60 Custom Functions & Validation

61 Function introduction

In R, a **function** is a reusable block of code that performs a specific task. Functions help you:

- Avoid repeating the same code again and again
- Organize code into logical pieces
- Make your code easier to read, test, and maintain

R has many **built-in functions** (e.g., `sum()`, `mean()`, `lm()`), and you can also define your **own functions**.

```
# Example: using a built-in function
x <- c(1, 2, 3, 4, 5)
mean(x)
```

```
[1] 3
```

61.1 Function components

A function has three parts:

- The `formals()`, the list of arguments that control how you call the function.
- The `body()`, the code inside the function.
- The `environment()`, the data structure that determines how the function finds the values associated with the names.

While the `formals` and `body` are specified explicitly when you create a function, the `environment` is specified implicitly, based on *where* you defined the function. The function environment always exists, but it is only printed when the function isn't defined in the global environment.

```
f02 <- function(x, y) {  
  # A comment  
  x + y  
}
```

```
formals(f02)
```

```
$x
```

```
$y
```

```
body(f02)
```

```
{  
  x + y  
}
```

```
environment(f02)
```

```
<environment: R_GlobalEnv>
```

61.2 Signalling conditions

There are three conditions that you can signal in code: errors, warnings, and messages.

- Errors are the most severe; they indicate that there is no way for a function to continue and execution must stop.
- Warnings fall somewhat in between errors and message, and typically indicate that something has gone wrong but the function has been able to at least partially recover.
- Messages are the mildest; they are way of informing users that some action has been performed on their behalf.

There is a final condition that can only be generated interactively: an interrupt, which indicates that the user has interrupted execution by pressing Escape, Ctrl + Break, or Ctrl + C (depending on the platform).

Conditions are usually displayed prominently, in a bold font or coloured red, depending on the R interface. You can tell them apart because errors always start with “Error”, warnings with “Warning” or “Warning message”, and messages with nothing.

```
stop("This is what an error looks like")

warning("This is what a warning looks like")

message("This is what a message looks like")
```

61.3 Ignoring conditions

You can choose to ignore conditions using `try()`, `tryCatch()` or `suppressMessages()`, `suppressWarnings()`.

```
# Example: ignoring a warning
suppressWarnings(log(-1))

suppressWarnings({
  warning("Uhoh!")
  warning("Another warning")
  1
})

suppressMessages({
  message("Hello there")
  2
})

suppressWarnings({
  message("You can still see me")
  3
})

suppressWarnings(library(dplyr))
suppressWarnings(library(ggplot2))
```

61.4 tryCatch function

The `tryCatch()` function allows you to handle conditions (errors, warnings, messages) in a controlled way. You can specify custom behavior for each type of condition.


```

result <- tryCatch({
  # Code that may produce an error
  log(-1)
},
warning = function(w) {
  cat("Caught a warning:", conditionMessage(w), "\n")
  NA # Return NA on warning
},
error = function(e) {
  cat("Caught an error:", conditionMessage(e), "\n")
  NA # Return NA on error
},
finally = {
  cat("Execution completed.\n")
})

```

Caught a warning: NaNs produced
Execution completed.

```
result
```

```
[1] NA
```

61.5 function scope

Functions in R have their own scope, meaning that variables defined inside a function are not accessible outside of it, and vice versa. This helps prevent naming conflicts and keeps functions self-contained.

```

my_function <- function() {
  x <- 10 # x is local to the function
  return(x)
}
my_function()

```

```
[1] 10
```

61.6 Creating Custom Functions

```
# Define a simple function to add two numbers
add_numbers <- function(a, b) {
  return(a + b)
}
# Call the function
add_numbers(3, 5)
```

```
[1] 8
```

61.7 Function Arguments

```
# Function with default argument
greet <- function(name = "Guest") {
  paste("Hello,", name)
}
greet()          # Uses default
```

```
[1] "Hello, Guest"
```

```
greet("Alice")  # Overrides default
```

```
[1] "Hello, Alice"
```

61.8 User-defined Functions in R Programming Language

User-defined functions are the functions that are created by the user. User defines the working, parameters, default parameter, etc. of that user-defined function. They can be only used in that specific code.

1. Single Input Single Output : Now create a function in R that will take a single input and return a single output.
2. Multiple Input Multiple Output : Now create a function in R Language that will take multiple inputs and return multiple outputs.

```

# Single Input Single Output
calculate_bmi <- function(data) {
  # BMI formula: weight (kg) / height (m)^2
  bmi <- data$weight / (data$height ^ 2)
  return(bmi)
}

# Example data
patient_data <- data.frame(
  weight = c(70, 80, 65), # in kg
  height = c(1.75, 1.8, 1.6) # in meters
)

# Calculate BMI
bmi_values <- calculate_bmi(patient_data)
print(bmi_values) # Outputs BMI for each patient

```

```
[1] 22.85714 24.69136 25.39062
```

```

# Multiple Input Multiple Output
calculate_summary_stats <- function(age, weight, height) {
  age_stats <- list(
    mean_age = mean(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    sd_age = sd(age, na.rm = TRUE)
  )

  weight_stats <- list(
    mean_weight = mean(weight, na.rm = TRUE),
    median_weight = median(weight, na.rm = TRUE),
    sd_weight = sd(weight, na.rm = TRUE)
  )

  height_stats <- list(
    mean_height = mean(height, na.rm = TRUE),
    median_height = median(height, na.rm = TRUE),
    sd_height = sd(height, na.rm = TRUE)
  )

  # Return a list containing all summary stats
  return(list(age = age_stats, weight = weight_stats, height = height_stats))
}

```

```

}

# Example data
age <- c(40, 45, 50, 55, 60)
weight <- c(70, 75, 65, 80, 68)
height <- c(1.75, 1.8, 1.6, 1.65, 1.7)

summary_stats <- calculate_summary_stats(age, weight, height)
print(summary_stats) # Outputs summary stats for age, weight, and height

```

```

$age
$age$mean_age
[1] 50

```

```

$age$median_age
[1] 50

```

```

$age$sd_age
[1] 7.905694

```

```

$weight
$weight$mean_weight
[1] 71.6

```

```

$weight$median_weight
[1] 70

```

```

$weight$sd_weight
[1] 5.94138

```

```

$height
$height$mean_height
[1] 1.7

```

```

$height$median_height
[1] 1.7

```

```

$height$sd_height
[1] 0.07905694

```

What is `{{}}` in R?

The `{{}}` syntax is known as curly-curly syntax, and it is part of the tidy evaluation framework in the `{r, lang}` package. It's used to capture variables directly and allows for easy manipulation within functions.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(tidyr)
```

```
stat <- function(data, grp, var) {  
  # Group by specified grouping variables and summarize the statistics  
  summary_stats <- data |> group_by(across({{ grp }})) |> summarise(  
    n = as.character(n_distinct(USUBJID)),  
    mean = as.character(round(mean({{ var }}, na.rm = TRUE), 2)),  
    median = as.character(median({{ var }}, na.rm = TRUE)),  
    sd = as.character(round(sd({{ var }}, na.rm = TRUE), 3)),  
    min = as.character(min({{ var }}, na.rm = TRUE)),  
    max = as.character(max({{ var }}, na.rm = TRUE)),  
    .groups = 'drop' # Ungroup after summarising  
  )  
  
  # Pivot to wider format  
  transposed_data <- summary_stats |>  
  pivot_longer(  
    cols = c(n, mean, median, sd, min, max), # Columns to pivot  
    names_to = "stat", # Name for the new 'stat' column  
    values_to = "value" # Name for the new 'value' column  
  ) |> pivot_wider(  
    names_from = "stat",  
    values_from = "value"  
  )  
}
```

```

    names_from = {{ grp }},      # Treatment groups to become column names
    values_from = value         # Values for each statistic
  )

  return(transposed_data)
}

adsl <- haven::read_sas("./data/adam/adsl.sas7bdat")
age_stat <- stat(data=adsl,grp=c(TRT01A,TRT01AN),var=AGE)
wgt_stat <- stat(adsl,TRT01A,WEIGHTBL)

```

62 difference !,!!,!!!

In R, the symbols `!`, `!!`, and `!!!` are used in different contexts, primarily in logical operations and tidy evaluation. - `!` (Logical NOT): This is a logical operator used to negate a logical value. It takes a single logical value (TRUE or FALSE) and returns the opposite value. For example, `!TRUE` returns FALSE, and `!FALSE` returns TRUE. - `!!` (Unquote): In the context of tidy evaluation (used in packages like dplyr), `!!` is used to unquote a single variable or expression. It allows you to evaluate a variable or expression within a tidy evaluation context. For example, if you have a variable `var_name` that holds the name of a column, you can use `!!var_name` to refer to that column in a dplyr function. - `!!!` (Unquote-splice): This is also used in tidy evaluation and is used to unquote and splice a list of variables or expressions. It allows you to pass multiple arguments to a function by splicing a list. For example, if you have a list of column names, you can use `!!!` to pass them as individual arguments to a dplyr function.

```
stat_simple <- function(data, grp, var, id_var = "USUBJID") {  
  # Convert strings to symbols  
  grp_sym <- sym(grp)  
  var_sym <- sym(var)  
  id_var_sym <- sym(id_var)  
  
  # Group by specified grouping variables and summarize the statistics  
  summary_stats <- data |>  
    group_by(!!grp_sym) |>  
    summarise(  
      n = as.character(n_distinct(!!id_var_sym)),  
      mean = as.character(round(mean(!!var_sym, na.rm = TRUE), 2)),  
      median = as.character(round(median(!!var_sym, na.rm = TRUE), 2)),  
      sd = as.character(round(sd(!!var_sym, na.rm = TRUE), 3)),  
      min = as.character(round(min(!!var_sym, na.rm = TRUE), 2)),  
      max = as.character(round(max(!!var_sym, na.rm = TRUE), 2)),  
      .groups = 'drop'  
    )  
  
  # Pivot to longer format then wider  
  transposed_data <- summary_stats |>  
    pivot_longer(  

```

```

      cols = c(n, mean, median, sd, min, max),
      names_to = "stat",
      values_to = "value"
    ) |>
    pivot_wider(
      names_from = !!grp_sym,
      values_from = value
    ) |>
    arrange(factor(stat, levels = c("n", "mean", "median", "sd", "min", "max")))

  return(transposed_data)
}

adsl1 <- haven::read_sas("../data/adam/adsl.sas7bdat")
age_stat <- stat_simple(data=adsl1,grp="TRT01A",var="AGE")

```

```

#where function
adsl1 <- adsl |>
  dplyr::select(dplyr::where(is.character))

adsl2 <- adsl |>
  dplyr::select(dplyr::where(is.numeric))

# across functions structure and explanation

#across(.cols, .fns, ..., .names = NULL, .unpack = FALSE)

df <- data.frame(
  id = 1:6,
  age = c(21, 34, 28, NA, 45, 30),
  wt = c(70, 80, 65, 72, NA, 68),
  sex = c("M","F","F","M","F","M")
)

# 2) Summaries grouped by sex
df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=dplyr::where(is.numeric),
    .fns=list( mean = ~mean(., na.rm=TRUE),
               sd   = ~sd(., na.rm=TRUE)
             ),
    .names = "{.col}_{.fn}")

```



```

)

df_qc <- df |>
  group_by(sex) |>
  summarize(mm=mean(age,na.rm=TRUE),
            mn1=mean(wt,na.rm=TRUE))

df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=c(age, wt),
                    .fns=list(nrecs = ~dplyr::n(),
                              mean = ~mean(., na.rm=TRUE),
                              sd   = ~sd(., na.rm=TRUE)
                    ),
            .names = "{.col}_{.fn}")
)

df2 <- df |>
  group_by(sex) |>
  summarise(across(.cols=c(age, wt),
                    .fns= ~mean(., na.rm=TRUE))
)

a <- c(1, 2, 3, 4, 5, 9909)
b <- data.frame(
  var1 = c(1, 2, 3, 4, 5, 9909),
  var2 = c(5, 6, 7, 8, 9, 9909)
)
c <- list(a,b)

# Numeric: replace sentinel 9909 with 0
numericdata <- tibble::tribble(
  ~var1, ~var2, ~var3, ~var4, ~var5,
  9909, 1, 2, 3, 4,
  5, 9909, 6, 7, 8,
  9, 3, 9909, 2, 7
)

dm <- haven::read_sas("./data/sdtm/dm.sas7bdat") |>
  dplyr::mutate(dplyr::across(where(is.character),~ dplyr::na_if(.,"")))

numericdata01 <- numericdata |>

```

```
dplyr::mutate(dplyr::across(.cols=where(is.numeric), .fns=~ dplyr::if_else(.x == 9909, 0,

# multiply all numeric variables by 2
adsl3 <- adsl |>
  dplyr::mutate(
    dplyr::across(.cols = where(is.numeric),
                  .fns = ~ .x * 2)
  )
```

63 Error Handling

```
robust_divide <- function(a, b) {  
  tryCatch(a / b,  
    warning = function(w) NA_real_,  
    error   = function(e) NA_real_)  
}  
robust_divide(10, 0)
```

```
[1] Inf
```

64 Unit Testing with testthat

Install once: `install.packages(c("testthat","devtools","usethis","roxygen2"))`

```
usethis::use_testthat()
usethis::use_test("safe_mean")
```

Create `tests/testthat/test-safe_mean.R`:

```
testthat::test_that("safe_mean works", {
  x <- c(1,2,NA)
  testthat::expect_equal(safe_mean(x), 1.5)
  testthat::expect_error(safe_mean("oops"))
})
```

65 Document with roxygen2

```
#' Compute a safe mean
#' @param x Numeric vector
#' @param na.rm Logical; remove NAs
#' @return Numeric scalar
#' @examples
#' safe_mean(c(1,2,NA))
#' @export
safe_mean <- function(x, na.rm = TRUE) {
  stopifnot(is.numeric(x))
  mean(x, na.rm = na.rm)
}
```

Run:

```
devtools::document()
```

66 Exercises

1. Write `winsorize(x, probs=c(0.05,0.95))` and test it.
2. Create `validate_columns(df, required=c("USUBJID","AGE"))` and add tests.
3. Add roxygen docs and build help pages.

67 R Package Development

A practical, end-to-end cookbook for building robust **R packages** with **devtools**, **roxygen2**, and **testthat**. Written for day-to-day use, from scaffolding to checks and tests.

67.1 1. Prerequisites

- R (4.1 recommended so you have the base pipe |>)
- RTools (Windows) / Xcode command line tools (macOS) for compilation
- Recommended packages:
 - `install.packages(c("devtools", "roxygen2", "usethis", "testthat", "pkgdown"))`

```
# Load helpers  
library(devtools)
```

Loading required package: usethis

```
library(usethis)
```

67.2 2. Create the Package Skeleton

67.2.1 2.1 Create a new package directory

```
# Creates simutils with DESCRIPTION, NAMESPACE placeholder, R/ etc.  
#usethis::create_package("simutils")  
# Or from an existing project folder  
# usethis::create_package(getwd())
```

67.2.2 2.2 Initialize Git and a README

```
usethis::use_git()
usethis::use_readme_rmd() # README.Rmd + badge placeholders
```

67.2.3 2.3 The minimal structure (what you get)

```
simutils/
DESCRIPTION
NAMESPACE
R/
man/
```

Tip: Keep all your R code inside R/. Avoid subfolders there. Group related functions in sensible files (e.g., `na.R`, `sample.R`).

67.3 3. DESCRIPTION: Package Metadata

Run helpers and then edit as needed.

```
usethis::use_description(fields = list(
  Title = "Simulation Utilities",
  Description = "Convenience functions for sampling and NA counting.",
  `Authors@R` = 'person(given = "Your", family = "Name", role = c("aut","cre"), email = "you@your.org")',
  License = "MIT + file LICENSE"
))

usethis::use_mit_license("Your Name")
```

Key fields you usually set:

- Package, Title, Version, Description
- Authors@R (with aut, cre roles)
- License (e.g., MIT)
- Depends, Imports, Suggests (use helpers below)

67.4 4. NAMESPACE: Exports and Imports (auto-generated)

You generally **do not** edit **NAMESPACE** by hand. It's generated from your roxygen tags in code.

- `@export` marks a function for users.
- `@import`, `@importFrom` declare your dependencies at function level.

You'll generate it with `devtools::document()`.

67.5 5. Write Functions + roxygen2 Headers

Create your first R file:

```
# File: R/na.R
#' Count NAs in a vector
#'
#' Count number of missing values (NA) in a numeric/logical/character vector.
#'
#' @param x A vector.
#' @return Integer count of NAs in `x`.
#' @examples
#' sum_na(c(1, NA, 3))
#' sum_na(airquality$Ozone)
#' @export
sum_na <- function(x) {
  sum(is.na(x))
}
```

Add a second exported function that works on data frames/matrices:

```
# File: R/na_counter.R
#' Column-wise NA counts
#'
#' Returns the number of NAs per column for a data frame or matrix.
#'
#' @param x A data frame or matrix.
#' @return A named integer vector of NA counts per column.
#' @examples
#' na_counter(airquality)
#' @export
```

```
na_counter <- function(x) {
  if (!is.data.frame(x) && !is.matrix(x)) stop("x must be a data.frame or matrix")
  colSums(is.na(x))
}
```

A simple sampler utility with a documented argument:

```
# File: R/sample_from_data.R
#' Sample rows from a data frame
#'
#' Draw a row-wise sample with optional replacement.
#'
#' @param data A data frame.
#' @param size Number of rows to sample.
#' @param replace Logical; sample with replacement?
#' @return A data frame containing the sampled rows.
#' @examples
#' sample_from_data(airquality, 10)
#' sample_from_data(airquality, 10, replace = TRUE)
#' @export
sample_from_data <- function(data, size, replace = FALSE) {
  stopifnot(is.data.frame(data))
  idx <- sample.int(nrow(data), size, replace = replace)
  data[idx, , drop = FALSE]
}
```

67.6 6. Generate Documentation

```
# Parses roxygen headers -> creates/updates NAMESPACE and man/*.Rd
devtools::document()
```

This will populate `man/` with help files and refresh `NAMESPACE`.

67.7 7. Add Data, Vignettes, Tests (Optional but Recommended)

67.7.1 7.1 Package data

```
# Create example data
sim_dat <- data.frame(
  ID = 1:10,
  Value = sample(1:11, 10),
  Apples = sample(c(TRUE, FALSE), 10, replace = TRUE)
)

usethis::use_data(sim_dat, overwrite = TRUE)
```

67.7.2 7.2 Vignettes

```
usethis::use_vignette("getting-started")
# Edit vignettes/getting-started.Rmd, then:
devtools::build_vignettes()
```

67.7.3 7.3 Tests with testthat

```
usethis::use_testthat()
# Creates tests/testthat.R and tests/testthat/ directory

usethis::use_test("na_counter")
# -> tests/testthat/test-na_counter.R (edit and add tests)
```

Example tests:

```
# File: tests/testthat/test-na_counter.R

library(testthat)

test_that("na_counter counts NAs correctly for data.frame and matrix", {
  test_matrix <- matrix(c(NA, 1, 4, NA, 5, 6), nrow = 2)
  air_expected <- c(Ozone = 37, Solar.R = 7, Wind = 0, Temp = 0, Month = 0, Day = 0)
  mat_expected <- c(V1 = 1, V2 = 1, V3 = 0)
```

```

expect_equal(na_counter(airquality), air_expected)
expect_equal(na_counter(test_matrix), mat_expected)
})

# Non-exported functions can be called with ::: if truly needed
# expect_equal(simutils:::sum_na(airquality$Ozone), 37)

```

Common expectations:

- `expect_identical()`, `expect_equal(tolerance = ...)`, `expect_equivalent()`
- `expect_error()`, `expect_warning()`
- `expect_output()` / `expect_output_file()` for console text

Run the test suite:

```
devtools::test()
```

67.8 8. Manage Dependencies

Use helpers to declare dependencies in DESCRIPTION (rather than editing by hand):

```

usethis::use_package("dplyr")           # add to Imports
usethis::use_package("ggplot2", type = "Suggests")

```

Guidelines:

- **Imports:** packages needed at runtime for your functions.
- **Suggests:** optional packages used in examples/tests/vignettes.
- **Depends:** rarely needed; if used, it attaches packages for users (discouraged).

67.9 9. Check Your Package Thoroughly

```

# Full check like CRAN does; read 00check.log for details
rc <- devtools::check()
rc

```

Typical findings & quick fixes:

- **Missing fields** (e.g., `License`) → add via `use_mit_license()` or set in `DESCRIPTION`.
- **Undocumented args** → ensure every parameter appears in `@param` and usage matches.
- **Broken examples** → run your examples; fix typos (e.g., `airquality`, not `airuality`).
- **Missing system tools** for building PDF manual/vignettes → install LaTeX if you need PDFs.
- **Failing tests** → run `devtools::test()`; update your code or expectations.

67.10 10. Build & Install Locally

```
# Build source tarball
devtools::build()

# Install the package from source
devtools::install()

# Try it out
#library(simutils)
na_counter(airquality)
```

67.11 11. Continuous Integration (optional)

You can wire up CI so checks run automatically on each commit. (Any provider works; pick what your org supports.)

```
# Example: set up a CI service config (adjust per provider)
# usethis::use_github_action_check_standard()
# or legacy: usethis::use_travis()
```

67.12 12. Package Website (optional)

```
usethis::use_pkgdown()
pkgdown::build_site()
```

67.13 13. Release Checklist

- ☐ R CMD check passes locally with **0 errors, 0 warnings, 0 notes**
 - ☐ All exported functions documented with examples
 - ☐ Vignettes render without errors
 - ☐ Tests pass on your platforms (and CI)
 - ☐ NEWS.md updated; version bumped in DESCRIPTION
 - ☐ License added and included
-

67.13.1 Appendix A — Common Files (at a glance)

- DESCRIPTION: metadata & dependencies
- NAMESPACE: auto-generated exports/imports
- R/: function source files
- man/: help files (.Rd) generated by roxygen2
- tests/: unit tests (testthat)
- vignettes/: long-form docs
- data/ & R/sysdata.rda: package data

67.13.2 Appendix B — Troubleshooting Snippets

```
# 1) Missing License in check
usethis::use_mit_license("Your Name")

# 2) Undocumented argument in check
# -> Make sure @param includes it and re-run document()

devtools::document(); devtools::check()

# 3) Examples fail in check (typo)
# -> Fix the example code, rerun check

# 4) pdflatex not found during check
# -> Install a LaTeX distribution or run check without manual building

# 5) Dependency not available
# -> Ensure it's in Imports/Suggests and installed in your environment
```

Turn this file into your team's template. Duplicate it, search/replace `simutils`, and you're ready to scaffold a new package in minutes.

68 Git in RStudio (Setup & Auth)

68.1 One-Time Setup

- Install Git and ensure `git --version` works.
- In R:

```
usethis::use_git_config(user.name = "Your Name", user.email = "you@example.com")
```

68.2 Initialize Git for the Current Project

```
usethis::use_git()
```

68.3 Connect to GitHub

- Create a GitHub account.
- In R:

```
usethis::create_github_token()
gitcreds::gitcreds_set() # paste token when prompted
usethis::use_github(protocol = "https")
```

Or set up SSH keys via RStudio (Tools > Global Options > Git/SVN).

68.4 Typical Workflow

1. Stage changes (Git pane in RStudio).
2. Commit with a clear message.
3. Push to origin (GitHub).

68.5 Remove Git from a Project (macOS/RStudio)

- In Finder/Terminal, delete the hidden `.git` folder in the project root (careful!).
- Or from Terminal at project root:

```
rm -rf .git
```

- Reopen project in RStudio; Git pane will disappear.

Exercises - Create a new repo for this Quarto course and push it. - Branch, make a change, open a Pull Request on GitHub.

69 Creating ADaM: ADSL from SDTM-like Inputs

```
####-----
# Program Name : adsl
# Protocol Name:
# Author      :
# Creation Date: `<DDMMYYYY>` by <initials>
# Purpose      : Create ADSL dataset from SDTM datasets
# Modification : <DDMMYYYY> by <initials>: <summary of changes>
####-----

library(tidyverse)
library(lubridate)
library(xportr)
library(haven)
library(stringr)

##-----
## 0. Helper functions
##-----

# Function to group together missing strings and NAs
str_missing <- function(x) {
  is.na(x) | str_trim(as.character(x)) == ""
}

# Function to check dataframe metadata
contents <- function(dat) {

  row_contents <- function(nm, dat) {
    var <- dat[[nm]]
    data.frame(
      Variable = nm,
      Class    = class(var)[1],

```

```

    Label    = ifelse(is.null(attr(var, "label")), "", attr(var, "label")),
    Format    = ifelse(is.null(attr(var, "SASformat")), "", attr(var, "SASformat")),
    stringsAsFactors = FALSE
  )
}

purrr::map_dfr(names(dat), row_contents, dat = dat)
}

##-----
## 1. Load SDTM from PHUSE Test Data Factory
##-----

sdtm <- "D:/R2SAS/r4sas/data/sdtm"

dm <- read_sas(paste0(sdtm, "/dm.sas7bdat"))
ex <- read_sas(paste0(sdtm, "/ex.sas7bdat"))
vs <- read_sas(paste0(sdtm, "/vs.sas7bdat"))
ds <- read_sas(paste0(sdtm, "/ds.sas7bdat"))

##-----
## 2. Derive DM-based variables (demo)
##-----

demo <- dm |>
  # Keep only necessary columns
  select(
    STUDYID, USUBJID, SUBJID, SITEID,
    AGE, AGEU, SEX, RACE, ETHNIC,
    DTHDTC, ARM, ACTARM
  ) |>
  mutate(
    # Age groups
    AGEGR1 = case_when(
      AGE < 65 ~ "<65",
      AGE >= 65 & AGE <= 70 ~ "65-70",
      AGE > 70 ~ ">70",
      TRUE ~ NA_character_
    ),
    AGEGR1N = case_when(
      AGEGR1 == "<65" ~ 1,
      AGEGR1 == "65-70" ~ 2,

```

```

    AGEGR1 == ">70" ~ 3,
    TRUE ~ NA_real_
  ),

  # Race numeric
  RACEN = case_when(
    RACE == "WHITE" ~ 1,
    RACE == "BLACK OR AFRICAN AMERICAN" ~ 2,
    RACE == "ASIAN" ~ 3,
    RACE == "AMERICAN INDIAN OR ALASKA NATIVE" ~ 4,
    RACE == "NATIVE HAWAIIAN OR OTHER PACIFIC ISLANDER" ~ 5,
    RACE == "UNKNOWN" ~ 6,
    TRUE ~ NA_real_
  ),

  # Planned treatment
  TRT01P = ARM,
  TRT01PN = case_when(
    TRT01P == "Placebo" ~ 0,
    TRT01P == "Xanomeline Low Dose" ~ 6,
    TRT01P == "Xanomeline High Dose" ~ 9,
    TRUE ~ NA_real_
  ),

  # Actual treatment
  TRT01A = ACTARM,
  TRT01AN = case_when(
    TRT01A == "Placebo" ~ 0,
    TRT01A == "Xanomeline Low Dose" ~ 6,
    TRT01A == "Xanomeline High Dose" ~ 9,
    TRUE ~ NA_real_
  ),

  # ITT flag: Y if ARM not missing
  ITTFL = if_else(!str_missing(ARM), "Y", "N"),

  # Death date
  DTHDT = as_date(DTHDTC)
) |>
select(-DTHDTC)

```

##-----

```

## 3. EX-based variables: treatment dates, duration, dose
##-----

# Treatment dates
treat <- ex |>
  select(USUBJID, EXSTDTC, EXENDTC) |>
  mutate(
    across(ends_with("DTC"), as_date)
  ) |>
  group_by(USUBJID) |>
  summarise(
    TRTSDT = suppressWarnings(min(EXSTDTC, na.rm = TRUE)),
    TRTEDT = suppressWarnings(max(EXENDTC, na.rm = TRUE)),
    .groups = "drop"
  ) |>
  mutate(
    # Handle Inf from min/max when all values are NA
    TRTSDT = replace(TRTSDT, is.infinite(TRTSDT), NA_Date_),
    TRTEDT = replace(TRTEDT, is.infinite(TRTEDT), NA_Date_),
    # Treatment duration (days)
    TRTDURD = as.numeric(TRTEDT - TRTSDT + (TRTEDT >= TRTSDT))
  )

# Dosing information
dose <- ex |>
  group_by(USUBJID) |>
  summarise(
    DOSE01A = sum(EXDOSE, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(
    DOSE01U = "mg"
  ) |>
  select(USUBJID, DOSE01A, DOSE01U)

# Merge DM- and EX-based info
demo1 <- reduce(
  list(demo, treat, dose),
  .f = left_join,
  by = "USUBJID"
) |>
  mutate(

```

```

# Safety flag: Y if ITTFL = Y and TRTSDT not missing
SAFFL = if_else(ITTFL == "Y" & !str_missing(TRTSDT), "Y", "N")
)

##-----
## 4. DS-based variables: EOS, discontinuation, screen failure
##-----

demo2 <- demo1 |>
  left_join(
    ds |>
      filter(DSCAT == "DISPOSITION EVENT") |>
      select(USUBJID, DSCAT, DSDECOD, DSTERM),
    by = "USUBJID"
  ) |>
  mutate(
    # End of study status
    EOSSTT = case_when(
      str_missing(DSDECOD) ~ "ONGOING",
      DSDECOD == "COMPLETED" ~ "COMPLETED",
      TRUE ~ "DISCONTINUED"
    ),
    # Discontinuation reason and text
    DCSREAS = if_else(
      DSDECOD != "COMPLETED" & !str_missing(DSDECOD),
      DSDECOD,
      "",
      missing = ""
    ),
    DCSREASP = if_else(
      DSDECOD != "COMPLETED" & !str_missing(DSTERM),
      DSTERM,
      "",
      missing = ""
    ),
    # Screen failure flag
    SCRNFL = if_else(
      DSDECOD == "SCREEN FAILURE",
      "Y",
      "N",
      missing = "N"
    )
  )

```

```

) |>
select(-c(DSCAT, DSDECOD, DSTERM))

##-----
## 5. VS-based variables: weight, height, BMI
##-----

# Baseline weight (visit 3)
weight <- vs |>
  filter(VSTESTCD == "WEIGHT", VISITNUM == 3) |>
  select(USUBJID, WEIGHTBL = VSSTRESN)

# Baseline height (visit 1)
height <- vs |>
  filter(VSTESTCD == "HEIGHT", VISITNUM == 1) |>
  select(USUBJID, HEIGHTBL = VSSTRESN)

demo3 <- reduce(
  list(demo2, weight, height),
  .f = left_join,
  by = "USUBJID"
) |>
  mutate(
    BMIBL = (WEIGHTBL / HEIGHTBL / HEIGHTBL) * 10000,
    BMIGR1 = case_when(
      BMIBL < 25 ~ "<25",
      BMIBL >= 25 ~ ">=25",
      TRUE ~ NA_character_
    )
  )

##-----
## 6. Metadata (dataset + variable level) and final ADSL
##-----

# Dataset-level metadata
dlm <- tribble(
  ~dataset, ~label,
  "adsl", "Subject-Level Analysis Dataset"
)

# Variable-level metadata

```

```

vlm <- tribble(
  ~dataset, ~variable, ~label, ~type, ~format,
  "adsl", "STUDYID", "Study Identifier", "character", NA_character_,
  "adsl", "USUBJID", "Unique Subject Identifier", "character", NA_character_,
  "adsl", "SUBJID", "Subject Identifier for the Study", "character", NA_character_,
  "adsl", "SITEID", "Study Site Identifier", "character", NA_character_,
  "adsl", "AGE", "Age", "numeric", NA_character_,
  "adsl", "AGEU", "Age Units", "character", NA_character_,
  "adsl", "AGEGR1", "Pooled Age Group 1", "character", NA_character_,
  "adsl", "AGEGR1N", "Pooled Age Group 1 (N)", "numeric", NA_character_,
  "adsl", "SEX", "Sex", "character", NA_character_,
  "adsl", "RACE", "Race", "character", NA_character_,
  "adsl", "RACEN", "Race (N)", "numeric", NA_character_,
  "adsl", "ETHNIC", "Ethnicity", "character", NA_character_,
  "adsl", "SAFFL", "Safety Population Flag", "character", NA_character_,
  "adsl", "ITTFL", "Intent-To-Treat Population Flag", "character", NA_character_,
  "adsl", "SCRNFL", "Screen Failure Population Flag", "character", NA_character_,
  "adsl", "ARM", "Description of Planned Arm", "character", NA_character_,
  "adsl", "ACTARM", "Description of Actual Arm", "character", NA_character_,
  "adsl", "TRT01P", "Planned Treatment for Period 01", "character", NA_character_,
  "adsl", "TRT01PN", "Planned Treatment for Period 01 (N)", "numeric", NA_character_,
  "adsl", "TRT01A", "Actual Treatment for Period 01", "character", NA_character_,
  "adsl", "TRT01AN", "Actual Treatment for Period 01 (N)", "numeric", NA_character_,
  "adsl", "DOSE01A", "Actual Treatment Dose for Period 01", "numeric", NA_character_,
  "adsl", "DOSE01U", "Units for Dose for Period 01", "character", NA_character_,
  "adsl", "TRTSDT", "Date of First Exposure to Treatment", "Date", "DATE9.",
  "adsl", "TRTEDT", "Date of Last Exposure to Treatment", "Date", "DATE9.",
  "adsl", "TRTDURD", "Total Treatment Duration (Days)", "numeric", NA_character_,
  "adsl", "EOSSTT", "End of Study Status", "character", NA_character_,
  "adsl", "DCSREAS", "Reason for Discontinuation from Study", "character", NA_character_,
  "adsl", "DCSREASP", "Reason Spec for Discont from Study", "character", NA_character_,
  "adsl", "DTHDT", "Date of Death", "Date", "DATE9.",
  "adsl", "WEIGHTBL", "Weight (kg) at Baseline", "numeric", NA_character_,
  "adsl", "HEIGHTBL", "Height (cm) at Baseline", "numeric", NA_character_,
  "adsl", "BMIBL", "Body Mass Index (kg/m2) at Baseline", "numeric", NA_character_,
  "adsl", "BMIGR1", "Pooled BMI Group 1", "character", NA_character_
)

adsl <- demo3 |>
  # Order variables
  select(
    STUDYID, USUBJID, SUBJID, SITEID,

```



```

    AGE, AGEU, AGEGR1, AGEGR1N, SEX, RACE, RACEN, ETHNIC,
    SAFFL, ITTFL, SCRNFL,
    ARM, ACTARM,
    TRT01P, TRT01PN, TRT01A, TRT01AN,
    DOSE01A, DOSE01U,
    TRTSDT, TRTEDT, TRTDURD,
    EOSSTT, DCSREAS, DCSREASP,
    DTHDT,
    WEIGHTBL, HEIGHTBL, BMIBL, BMIGR1
  ) |>
  arrange(USUBJID) |>
  # Apply labels / types / formats
  xportr_df_label(dlm, domain = "adsl") |>
  xportr_label(vlm, domain = "adsl") |>
  xportr_type(vlm, domain = "adsl") |>
  xportr_format(vlm, domain = "adsl") |>
  # Replace character NA with ""
  mutate(across(where(is.character), ~ replace(., is.na(.), "")))

##-----
## 7. QC: label + metadata + export
##-----

# Check dataframe label
attr(adsl, "label")

# Check metadata
adsl |>
  contents()

# Output to specified path
xportr_write(adsl, "./adsl.xpt")

```

Note: Real ADSL creation must follow **ADaM IG** (derive flags, dates, imputations, populations). This example is educational only.

Exercises 1. Add analysis populations (e.g., SAFFL, FASFL) based on simple rules. 2. Derive AGEGR1 as <65 / 65 and use ordered factor. 3. Add a treatment end date TRTEDT and compute treatment duration.

70 TLFs: Table, Figure, Listing

70.1 Create demographic table from ADSL

```
library(r2rtf)
library(dplyr)
library(tidyr)
library(haven)

adsl <- haven::read_sas("data/adam/adsl.sas7bdat")

bs_count <- function(data, grp, var,
                      var_label = var,
                      decimal = 1,
                      total = TRUE,
                      blank_row = FALSE) {
  data <- data |>
    dplyr::rename(grp = !!grp, var = !!var)

  coding <- levels(factor(data$grp))

  data <- data |>
    dplyr::mutate(grp = as.numeric(factor(grp)))

  res <- with(data, table(var, grp)) |>
    as.data.frame() |>
    dplyr::mutate(grp = as.numeric(grp))

  if (total) {
    res_tot <- with(data, table(var)) |>
      as.data.frame() |>
      dplyr::mutate(grp = 9999)
    res <- dplyr::bind_rows(res, res_tot)
  }
}
```

```

res <- res |>
  rename(n = Freq)

res <- res |>
  mutate(pct = formatC(n / sum(n) * 100, digits = decimal, format = "f", flag = "0"),
    pct1= paste0(n, "(", pct, "%)")

res$n <- as.character(res$n)
res <- res |>
  dplyr::select(-n,-pct)
res <- res |>
  pivot_longer(cols = c(pct1), names_to = "key", values_to = "value") |>
  unite(keys, grp, key) |>
  pivot_wider(names_from = keys, values_from = value) |>
  mutate(var_label = var_label) |>
  mutate(var = as.character(var))

names(res) <- gsub("_n", "", names(res), fixed = TRUE)
attr(res, "coding") <- coding
if (blank_row) {
  res <- bind_rows(tibble(var_label = var_label), res)
}
res
}

bs_continuous <- function(data, grp, var,
                          var_label = var,
                          decimal = 1,
                          total = TRUE,
                          blank_row = FALSE) {

  data <- data |>
    rename(grp = !!grp, var = !!var)
  coding <- levels(factor(data$grp))
  data <- data |> mutate(grp = as.numeric(factor(grp)))

  res <- data |>
    select(grp, var) |>
    na.omit() |>
    group_by(grp) |>
    summarise(
      n= n(),
      Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),

```

```

    SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
    Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
    Range = paste(range(var), collapse = " to ")
  )

  if (total) {
    res_tot <- data |>
      select(grp, var) |>
      na.omit() |>
      summarise(
        n = n(),
        Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),
        SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
        Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
        Range = paste(range(var), collapse = " to ")
      ) |>
      mutate(grp = 9999)
    res <- bind_rows(res, res_tot)
  }

  res$"n" <- as.character(res$"n")

  res <- res |>
    pivot_longer(cols = -grp, names_to = "key", values_to = "value") |>
    mutate(key = factor(key, levels = c("n", "Mean", "SD", "Median", "Range"))) |>
    pivot_wider(names_from = grp, values_from = value) %>%
    mutate(var_label = var_label) |>
    mutate(key = as.character(key)) |>
    rename(var = key)

  if (blank_row) {
    res <- bind_rows(tibble(var_label = var_label), res)
  }

  res
}

# The code above define two utility function for baseline characteristic tables.

# Analysis Set

```

```

ana <- adsl |>
  subset(ITTFL == "Y")

ana <- ana |>
  mutate(
    RACE = factor(
      RACE,
      c("WHITE", "BLACK OR AFRICAN AMERICAN", "AMERICAN INDIAN OR ALASKA NATIVE"),
      c("White", "Black", "Other")
    ),
    SEX = factor(
      SEX,
      c("F", "M"),
      c("Female", "Male")
    ),
    AGEGR1 = factor(
      AGEGR1,
      levels = c("<65", "65-80", ">80")
    )
  )

age_1 <- bs_continuous(ana, "TRT01AN", "AGE", "Age (Years)", blank_row = TRUE)
names(age_1) <- c("var_label", "var", "1_pct1", "2_pct1", "3_pct1", "9999_pct1")
# Build Data for r2rtf
bs_tb <- bind_rows(
  bs_count(ana, "TRT01AN", "SEX", "Gender", blank_row = TRUE),
  bs_count(ana, "TRT01AN", "AGEGR1", "Age (Years)", blank_row = TRUE),
  age_1,
  bs_count(ana, "TRT01AN", "RACE", "Race", blank_row = TRUE)
) |>
  mutate(
    var_label = factor(var_label, levels = c("Gender", "Age (Years)", "Race"))
  )
bs_tb <- bs_tb |>
  dplyr::mutate(lab1 = if_else(is.na(var), var_label, paste0(" ", var))) |>
  dplyr::select(lab1, `1_pct1`, `2_pct1`, `3_pct1`, `9999_pct1`)

bs_tb[is.na(bs_tb)] <- "" # convert NA to blank string.

# reporting
bs_rtf <- bs_tb |>

```

```

rtf_page(width = 9.5,
         border_first = "single",
         border_last = "single") |>
rtf_title("Demographic and Anthropometric Characteristics", "ITT Subjects") |>
rtf_colheader(" | Placebo | Drug Low Dose | Drug High Dose | Total ",
              col_rel_width = c(3, rep(2, 4)),
              border_top = rep("",5),
              border_right = rep("",5),
              border_left =rep("",5)
) |>
rtf_colheader(" | n (%) | n (%) | n (%) | n (%) ",
              col_rel_width =c(3, rep(2, 4)),
              border_top = rep("",5),
              border_right = rep("",5),
              border_left =rep("",5)
) |>
rtf_body(
  #page_by = "var_label",
  col_rel_width = c(3, rep(2, 4)),
  text_justification = c("l", rep("c", 4)),
  #text_format = c(rep("", 5), "b"),
  border_top = rep("",5),
  border_right = rep("",5),
  border_left =rep("",5)
) |>
rtf_footnote("This is a footnote",
             border_left = "",
             border_right = "",
             border_bottom = "")

# Output .rtf file
bs_rtf %>%
  rtf_encode() %>%
  write_rtf("bs_example.rtf")

```

Demographic and Anthropometric Characteristics
ITT Subjects

	Placebo n (%)	Drug Low Dose n (%)	Drug High Dose n (%)	Total n (%)
Gender				
Female	53(10.4%)	50(9.8%)	40(7.9%)	143(28.1%)
Male	33(6.5%)	34(6.7%)	44(8.7%)	111(21.9%)
Age (Years)				
<65	14(2.8%)	8(1.6%)	11(2.2%)	33(6.5%)
65-80	42(8.3%)	47(9.3%)	55(10.8%)	144(28.3%)
>80	30(5.9%)	29(5.7%)	18(3.5%)	77(15.2%)
Age (Years)				
n	86	84	84	254
Mean	75.2	75.7	74.4	75.1
SD	8.6	8.3	7.9	8.2
Median	76.0	77.5	76.0	77.0
Range	52 to 89	51 to 88	56 to 88	51 to 89
Race				
White	78(15.4%)	78(15.4%)	74(14.6%)	230(45.3%)
Black	8(1.6%)	6(1.2%)	9(1.8%)	23(4.5%)
Other	0(0.0%)	0(0.0%)	1(0.2%)	1(0.2%)
This is a footnote				

70.2 Create adverse event table from ADAE

```
library(r2rtf)
library(dplyr)
library(tidyr)

####Read SAS Data####
#keep the variable
adsl <- haven::read_sas("./data/adam/adsl.sas7bdat")
adae <- haven::read_sas("./data/adam/adae.sas7bdat")

adae <- adae |>
  dplyr::mutate(AEDECOD=stringr::str_to_title(AEDECOD))

ae_t1 <- adae %>%
  group_by(TRTA) %>%
  mutate(n_subj = n_distinct(USUBJID)) %>%
  group_by(TRTA, AEDECOD) %>%
  summarise(
    n_ae = n_distinct(USUBJID),
    pct = round(n_ae / unique(n_subj) * 100, 2)
  ) %>%
  ungroup() %>%
  dplyr::mutate(pct = paste0(n_ae,"(",pct, "%)")) %>%
  dplyr::filter(n_ae > 5) %>%
  # only show AE terms with at least 5 subjects in one treatment group.
  pivot_longer(cols = c( pct), names_to = "var", values_to = "value") %>%
  unite(temp, TRTA, var) %>%
  pivot_wider(names_from = temp, values_from = value, values_fill = "0(0.00%)") |>
  select(-n_ae)

ae_tbl <- ae_t1 %>%
  rtf_page(orientation = "landscape",
    border_first = "single",
    border_last = "single") %>%
  rtf_title(
    "Analysis of Subjects With Specific Adverse Events",
    c(
      "(Incidence > 5 Subjects in One or More Treatment Groups)",
```



```

      "ASaT"
    )
  ) %>%
  rtf_colheader(" | Placebo | Drug High Dose | Drug Low Dose",
    col_rel_width = c(4, rep(2, 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)
  ) %>%
  rtf_colheader(" | n (%) | n (%) | n (%)",
    col_rel_width = c(4, rep(2, 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)

  ) %>%
  rtf_body(
    col_rel_width = c(4, rep(2, 3)),
    text_justification = c("l", rep("c", 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)
  ) %>%
  rtf_footnote(c("{^\\dagger}This is footnote 1", "This is footnote 2"),
    border_left = "",
    border_right = "",
    border_bottom = "")

# Output .rtf file
ae_tbl %>%
  rtf_encode(page_title = "all",
    page_footnote = "all") %>%
  write_rtf("ae_example.rtf")

```

Analysis of Subjects With Specific Adverse Events
(Incidence > 5 Subjects in One or More Treatment Groups)
ASaT

	Placebo n (%)	Drug High Dose n (%)	Drug Low Dose n (%)
Application Site Pruritus	6(8.7%)	0(0.00%)	0(0.00%)
Diarrhoea	9(13.04%)	0(0.00%)	0(0.00%)
Erythema	9(13.04%)	0(0.00%)	0(0.00%)
Headache	7(10.14%)	0(0.00%)	0(0.00%)
Pruritus	8(11.59%)	0(0.00%)	0(0.00%)
Upper Respiratory Tract Infection	6(8.7%)	0(0.00%)	0(0.00%)
Application Site Dermatitis	0(0.00%)	7(8.86%)	0(0.00%)
Application Site Erythema	0(0.00%)	15(18.99%)	0(0.00%)
Application Site Irritation	0(0.00%)	9(11.39%)	9(11.69%)
Application Site Pruritus	0(0.00%)	22(27.85%)	22(28.57%)
Application Site Vesicles	0(0.00%)	6(7.59%)	0(0.00%)
Dizziness	0(0.00%)	12(15.19%)	0(0.00%)
Erythema	0(0.00%)	14(17.72%)	0(0.00%)
Headache	0(0.00%)	6(7.59%)	0(0.00%)
Hyperhidrosis	0(0.00%)	8(10.13%)	0(0.00%)
Nasopharyngitis	0(0.00%)	6(7.59%)	0(0.00%)
Nausea	0(0.00%)	6(7.59%)	0(0.00%)

*This is footnote 1
This is footnote 2

Exercises 1. Format Table 1 to N ($\text{mean} \pm \text{SD}$) for age. 2. Add risk table to the KM plot (use an extension like survminer outside of this minimal example). 3. Create a listing that includes population flags once you derive them.

71 ggplot2 in R with CDISC ADaM Examples

72 Introduction

This tutorial is a **step-by-step guide to ggplot2** in R, designed to take you from **beginner to advanced**, using **CDISC ADaM-style datasets** throughout.

- ADSL: Subject-level data
- ADLB: Laboratory data
- ADVS: Vital signs (optional example)
- ADTTE: Time-to-event data (e.g. PFS)

These are **simulated examples** with ADaM-like structures and variable names. In a real project you would replace them with your actual ADSL/ADLB/ADTTE datasets.

We will cover:

- Basic **Grammar of Graphics** concepts
 - Building plots layer by layer with `ggplot()`
 - Common geoms: points, lines, bars, histograms, boxplots
 - Aesthetics: color, shape, size, linetype
 - Faceting, scales, themes, coordinates
 - Combining `dplyr` + `ggplot2` in a tidy ADaM workflow
 - Advanced: annotations, secondary axes, simple KM plot from ADTTE
 - Saving plots for reports/presentations
-

73 Creating Simulated ADaM-Style Data

In this section we create small ADaM-like datasets entirely in R so the tutorial is self-contained.

73.1 ADSL: Subject-Level Data

```
ADSL <- haven::read_sas("./data/adam/adsl.sas7bdat")  
  
ADSL <- ADSL |>  
  dplyr::mutate(TRTA=ARM)
```

73.2 ADLB: Laboratory Data (e.g. ALT)

We simulate longitudinal ALT values (PARAMCD = "ALT") at multiple visits:

```
ADLB <- haven::read_sas("./data/adam/adlbhy.sas7bdat") |>  
  dplyr::filter(PARAMCD == "ALT")
```

73.3 ADTTE: Time-to-Event Data (e.g. PFS)

We simulate a simple ADTTE:

- PARAMCD = "PFS"
- AVAL: time (months)
- CNSR: censor flag (0 = event, 1 = censored)

```

ADTTE <- ADSL %>%
  select(STUDYID, USUBJID, ARM, TRTA, SEX, AGE) %>%
  mutate(
    PARAMCD = "PFS",
    PARAM    = "Progression-Free Survival (Months)",
    # Arm-specific exponential rates
    rate = case_when(
      ARM == "Placebo" ~ 0.12,
      ARM == "Dose 1"  ~ 0.09,
      TRUE             ~ 0.07
    ),
    AVAL = rexp(n(), rate = rate),
    AVAL = pmin(AVAL, 36), # administrative censoring at 36 months
    CNSR = if_else(AVAL >= 36, 1, 0)
  ) %>%
  select(-rate)

ADTTE %>% head()

```

```

# A tibble: 6 x 10
  STUDYID    USUBJID    ARM    TRTA  SEX    AGE PARAMCD PARAM    AVAL  CNSR
  <chr>      <chr>      <chr>  <chr> <chr> <dbl> <chr>   <chr>  <dbl> <dbl>
1 CDISCPIL0T01 01-701-1015 Placebo Plac~ F      63 PFS     Prog~  7.03     0
2 CDISCPIL0T01 01-701-1023 Placebo Plac~ M      64 PFS     Prog~  4.81     0
3 CDISCPIL0T01 01-701-1028 Xanomel~ Xano~ M      71 PFS     Prog~ 19.0     0
4 CDISCPIL0T01 01-701-1033 Xanomel~ Xano~ M      74 PFS     Prog~  0.451    0
5 CDISCPIL0T01 01-701-1034 Xanomel~ Xano~ F      77 PFS     Prog~  0.803    0
6 CDISCPIL0T01 01-701-1047 Placebo Plac~ F      85 PFS     Prog~  2.64     0

```

With these datasets we can now explore ggplot2 concepts **in an ADaM context**.

74 Grammar of Graphics with ADaM

The basic idea of ggplot2 (Grammar of Graphics):

- **Data:** ADSL / ADLB / ADTTE
- **Aesthetics** (`aes`): how variables map to x, y, color, etc.
- **Geoms:** geometric objects (points, lines, bars, etc.)
- **Scales, facets, coordinates, theme:** control look and structure

The general pattern:

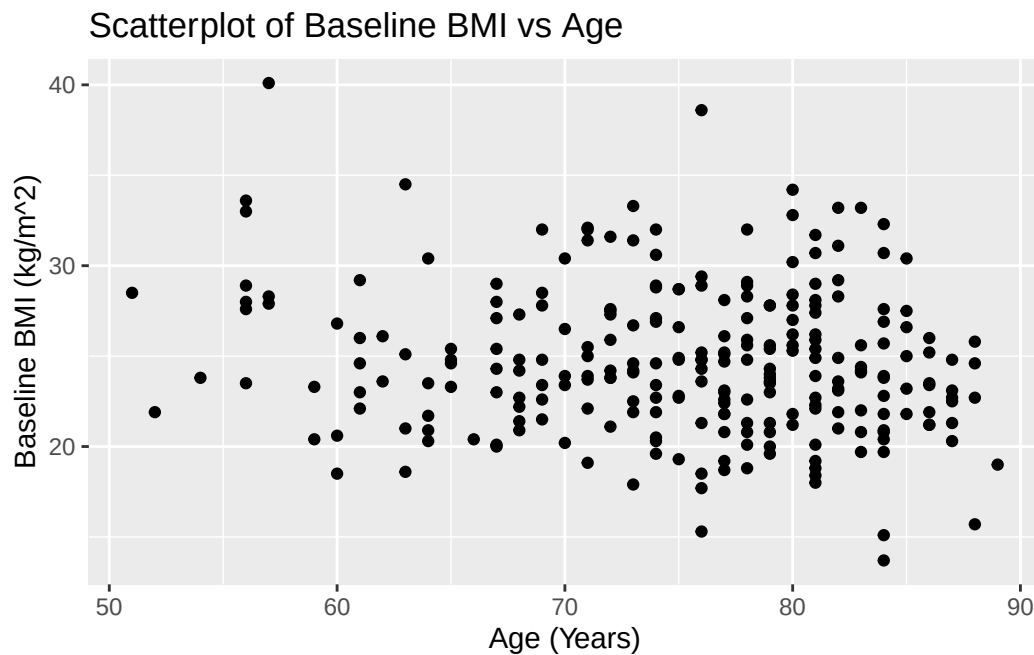
```
ADSL <- haven::read_sas("./data/adam/adsl.sas7bdat")
ggplot(data = ADSL, aes(x = AGE, y = BMIBL)) +
  geom_point()
```

We will start with subject-level plots from **ADSL**, then move to longitudinal labs from **ADLB**, and finally to **ADTTE**.

75 Basic Plots from ADSL (Beginner Level)

75.1 Scatterplot: AGE vs BMIBL

```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point() +  
  labs(  
    title = "Scatterplot of Baseline BMI vs Age",  
    x = "Age (Years)",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



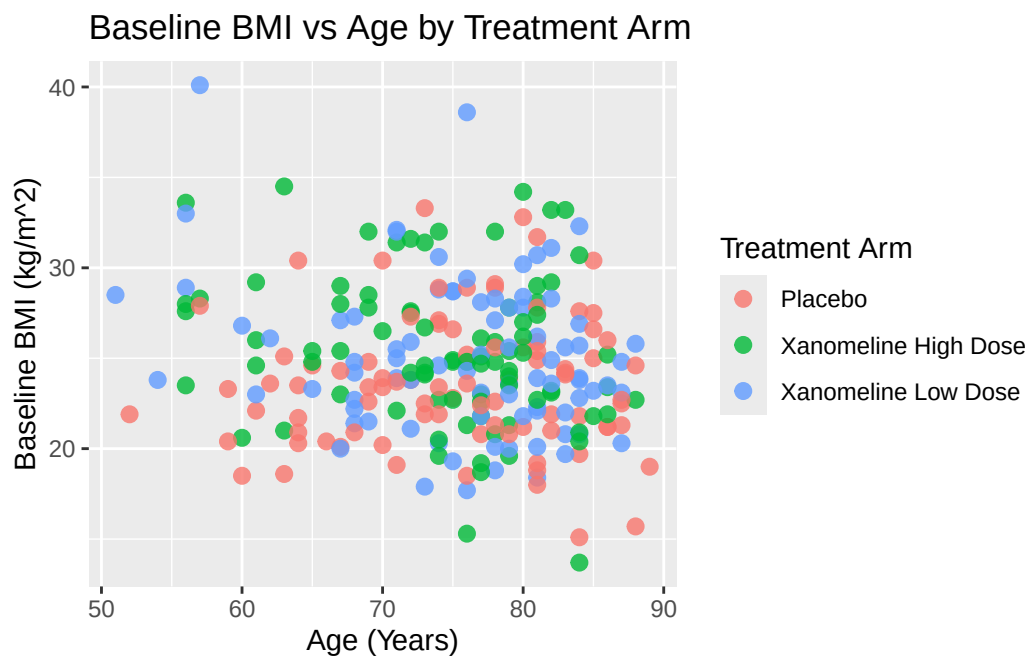
75.1.1 Mapping vs Setting Aesthetics

- **Mapping** (inside `aes()`): color/shape/size depend on data (e.g. ARM).

- **Setting** (outside `aes()`): fixed values.

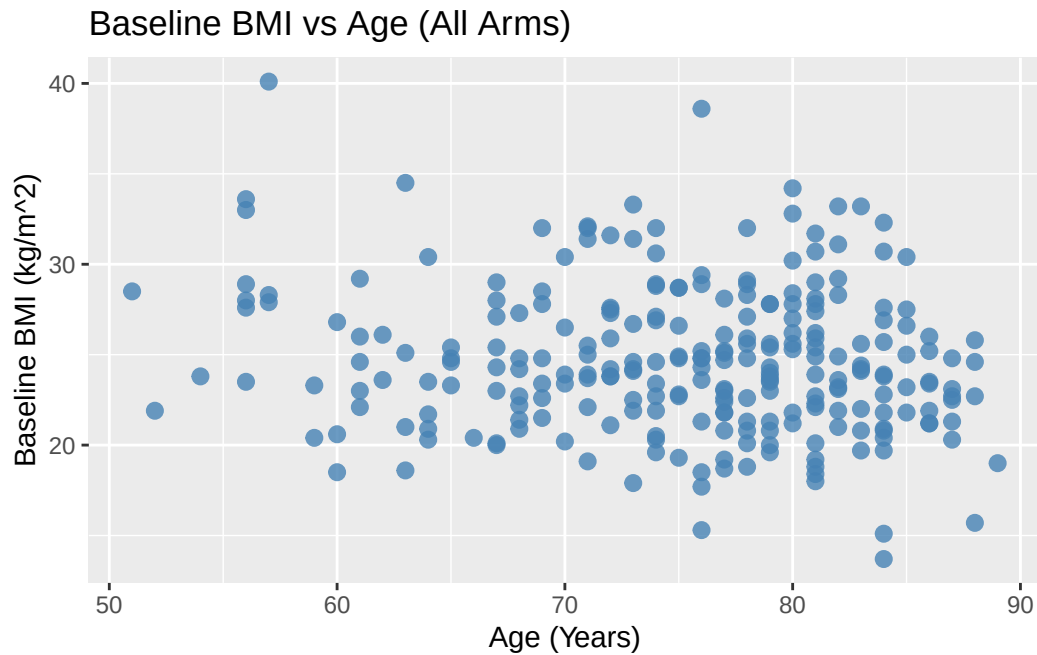
Map ARM to color:

```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(
    title = "Baseline BMI vs Age by Treatment Arm",
    color = "Treatment Arm",
    x = "Age (Years)",
    y = "Baseline BMI (kg/m^2)"
  )
```



Set color manually (not based on data):

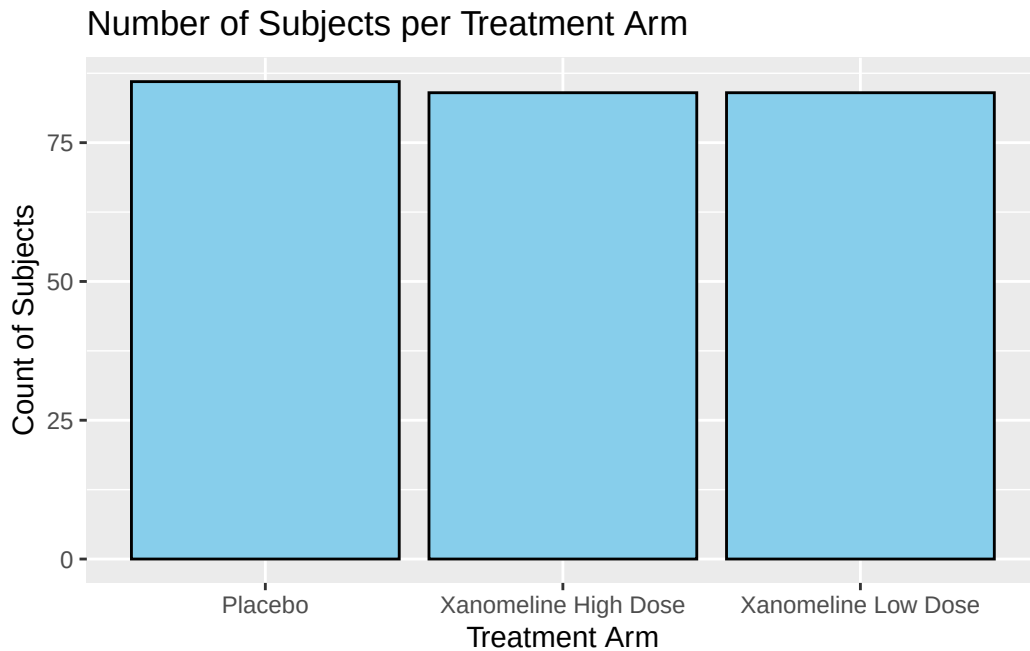
```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +
  geom_point(color = "steelblue", size = 2.5, alpha = 0.8) +
  labs(
    title = "Baseline BMI vs Age (All Arms)",
    x = "Age (Years)",
    y = "Baseline BMI (kg/m^2)"
  )
```



75.2 Bar Plot: Subject Counts per Arm

Using ADSL, we can show how many subjects per arm:

```
ggplot(ADSL, aes(x = TRT01A)) +  
  geom_bar(fill = "skyblue", color = "black") +  
  labs(  
    title = "Number of Subjects per Treatment Arm",  
    x = "Treatment Arm",  
    y = "Count of Subjects"  
  )
```



If you pre-compute counts (e.g. for TLG), use `geom_col()`:

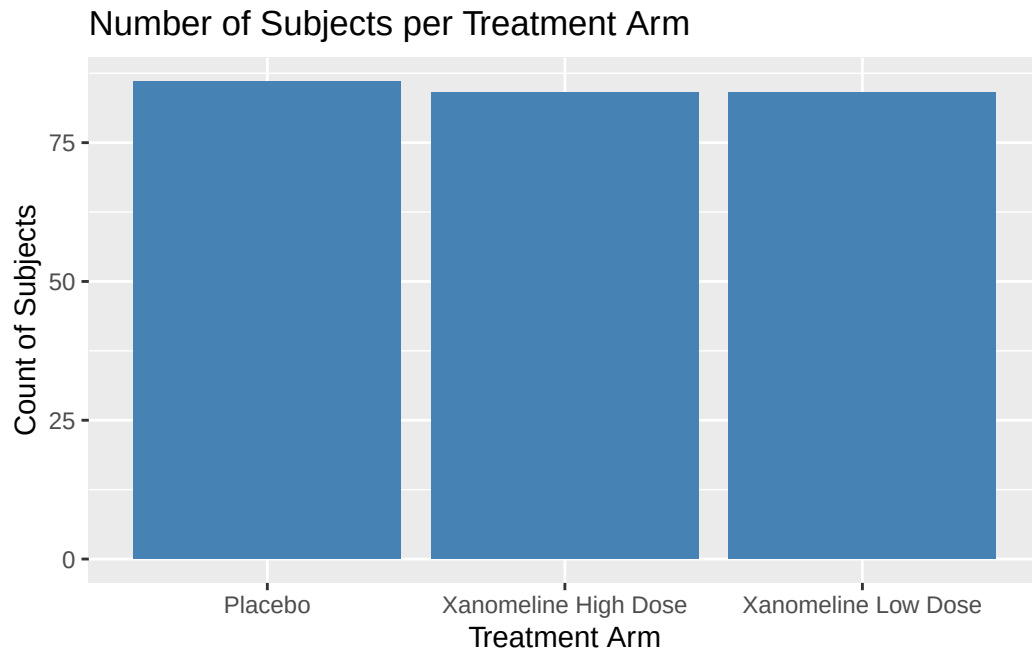
```
adsl_counts <- ADSL %>%  
  count(TRT01A, name = "n")
```

```
adsl_counts
```

```
# A tibble: 3 x 2
```

TRT01A	n
<chr>	<int>
1 Placebo	86
2 Xanomeline High Dose	84
3 Xanomeline Low Dose	84

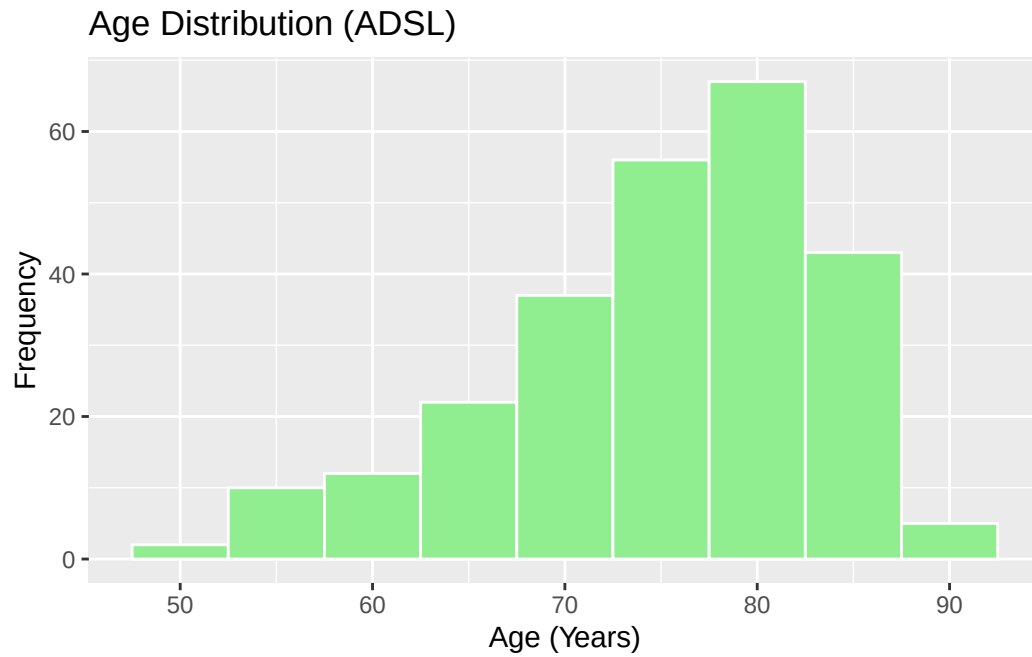
```
ggplot(adsl_counts, aes(x = TRT01A, y = n)) +  
  geom_col(fill = "steelblue") +  
  labs(  
    title = "Number of Subjects per Treatment Arm",  
    x = "Treatment Arm",  
    y = "Count of Subjects"  
  )
```



75.3 Histogram and Density: Age Distribution

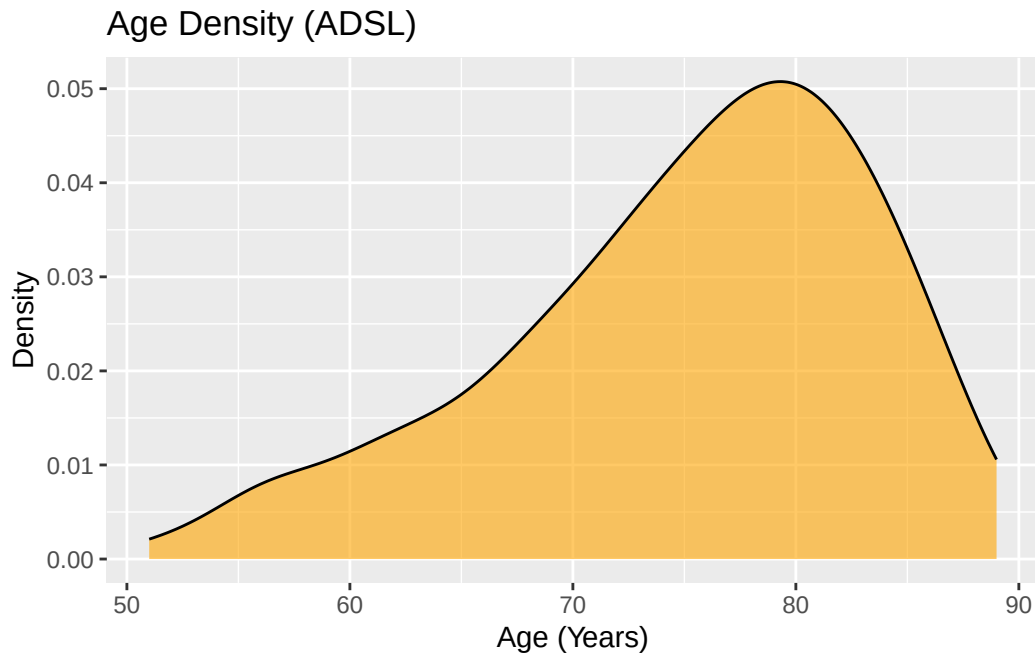
Histogram of AGE:

```
ggplot(ADSL, aes(x = AGE)) +  
  geom_histogram(binwidth = 5, fill = "lightgreen", color = "white") +  
  labs(  
    title = "Age Distribution (ADSL)",  
    x = "Age (Years)",  
    y = "Frequency"  
  )
```



Density plot:

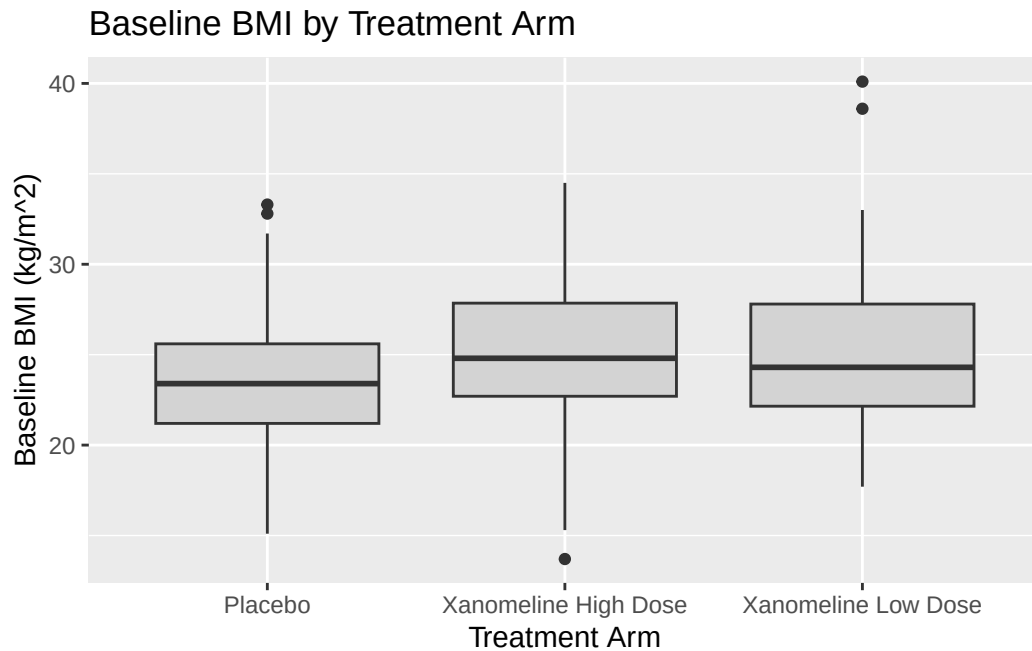
```
ggplot(ADSL, aes(x = AGE)) +  
  geom_density(fill = "orange", alpha = 0.6) +  
  labs(  
    title = "Age Density (ADSL)",  
    x = "Age (Years)",  
    y = "Density"  
  )
```



75.4 Boxplot: BMIBL by ARM

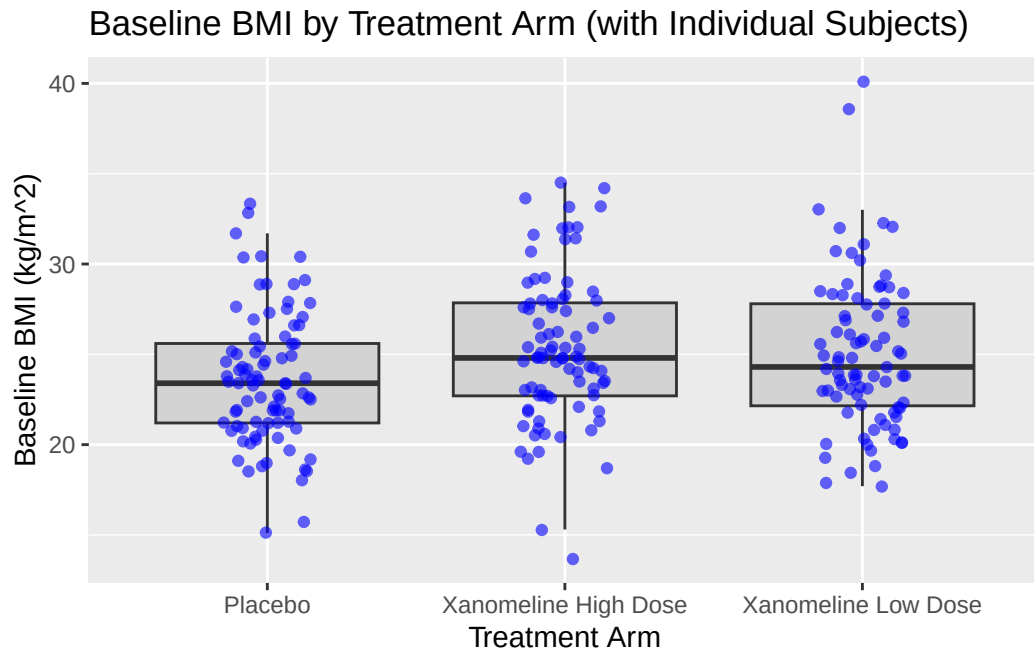
Boxplots are useful in CSR or listings to compare distributions across arms.

```
ggplot(ADSL, aes(x = TRT01A, y = BMIBL)) +  
  geom_boxplot(fill = "lightgray") +  
  labs(  
    title = "Baseline BMI by Treatment Arm",  
    x = "Treatment Arm",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



You can combine boxplot with jittered points:

```
ggplot(ADSL, aes(x = TRT01A, y = BMIBL)) +  
  geom_boxplot(fill = "lightgray", outlier.shape = NA) +  
  geom_jitter(width = 0.15, alpha = 0.6, color = "blue") +  
  labs(  
    title = "Baseline BMI by Treatment Arm (with Individual Subjects)",  
    x = "Treatment Arm",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



76 Longitudinal Plots from ADLB (Intermediate)

Now we use **ADLB** to illustrate time course plots, which are common in clinical reports.

76.1 Line Plot: Mean ALT Over Time by ARM

First summarise ADLB:

```
adlb_alt_summary <- ADLB %>%  
  group_by(TRTA, AVISITN, AVISIT) %>%  
  summarise(  
    mean_ALT = mean(AVAL, na.rm=TRUE),  
    sd_ALT   = sd(AVAL, na.rm=TRUE),  
    n        = n(),  
    se_ALT   = sd_ALT / sqrt(n),  
    .groups = "drop"  
  )  
  
adlb_alt_summary$AVISIT <- factor(adlb_alt_summary$AVISIT, levels=unique(adlb_alt_summary$AVISIT))  
adlb_alt_summary
```

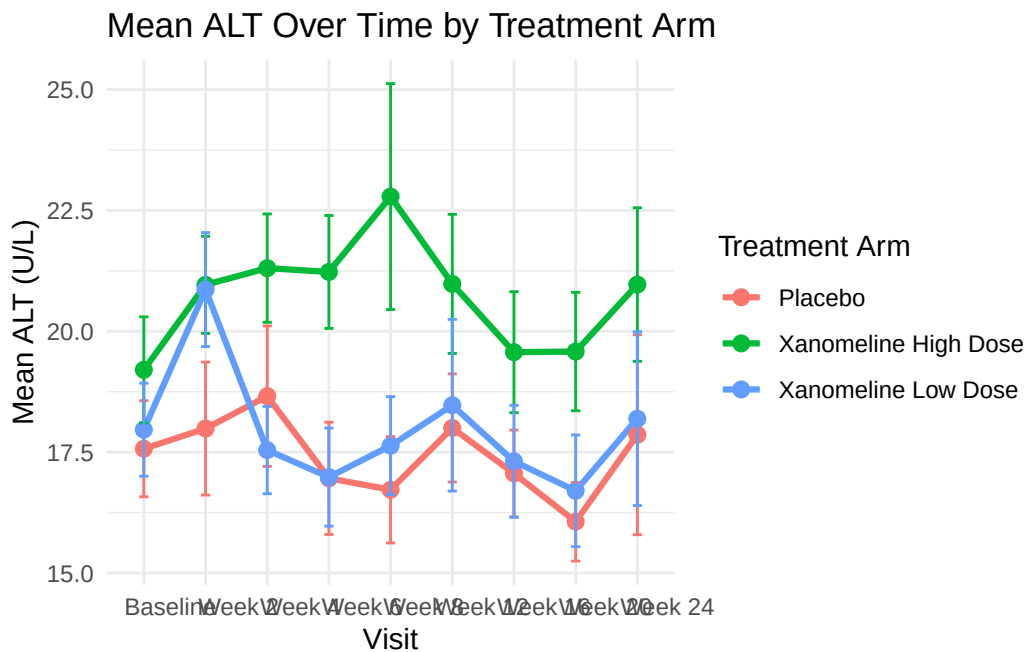
A tibble: 27 x 7

TRTA	AVISITN	AVISIT	mean_ALT	sd_ALT	n	se_ALT
<chr>	<dbl>	<fct>	<dbl>	<dbl>	<int>	<dbl>
1 Placebo	0	"Baseline"	17.6	9.22	86	0.994
2 Placebo	2	"Week 2"	18.0	12.5	83	1.38
3 Placebo	4	"Week 4"	18.7	12.9	79	1.45
4 Placebo	6	"Week 6"	17.0	9.92	73	1.16
5 Placebo	8	"Week 8"	16.7	9.34	72	1.10
6 Placebo	12	"Week 12"	18	9.16	67	1.12
7 Placebo	16	"Week 16"	17.1	7.39	68	0.897
8 Placebo	20	"Week 20"	16.1	6.56	65	0.814
9 Placebo	24	"Week 24"	17.9	15.6	57	2.07

```
10 Xanomeline High Dose      0 "      Baseline"      19.2  10.0      84  1.10
# i 17 more rows
```

Line plot with error bars:

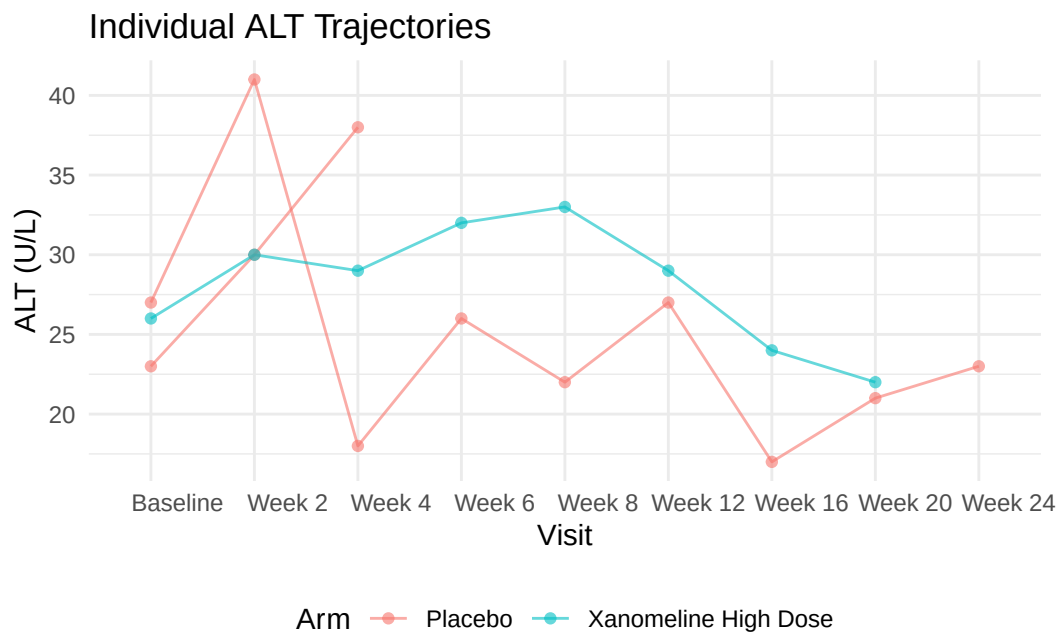
```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  geom_errorbar(
    aes(ymin = mean_ALT - se_ALT, ymax = mean_ALT + se_ALT),
    width = 0.15
  ) +
  labs(
    title = "Mean ALT Over Time by Treatment Arm",
    x = "Visit",
    y = "Mean ALT (U/L)",
    color = "Treatment Arm"
  ) +
  theme_minimal()
```



76.2 Individual Profiles (Spaghetti Plot)

To see variability, we can plot individual patients (subset to avoid overcrowding):

```
ADLB_sub <- ADLB |>
  head(20)
ADLB_sub$AVISIT <- factor(ADLB_sub$AVISIT, levels=unique(ADLB_sub$AVISIT[order(ADLB_sub$AVISIT)]))
ggplot(ADLB_sub,
  aes(x = AVISIT, y = AVAL, group = USUBJID, color = TRTA)) +
  geom_line(alpha = 0.6) +
  geom_point(alpha = 0.6, size = 1.5) +
  labs(
    title = "Individual ALT Trajectories ",
    x = "Visit",
    y = "ALT (U/L)",
    color = "Arm"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



77 Faceting with ADaM Data

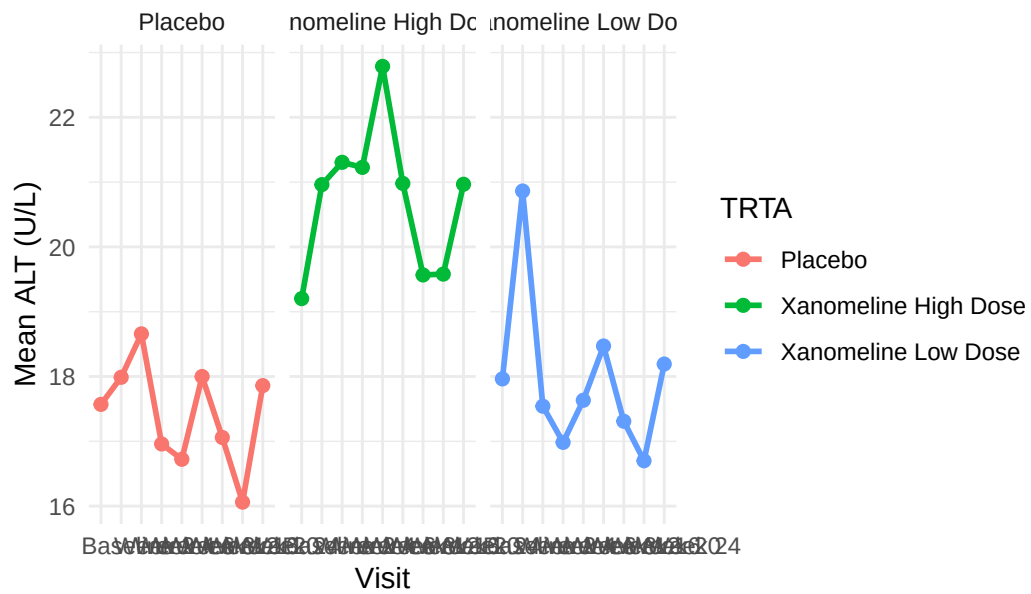
Faceting is useful when splitting results by subgroups, lab parameters, or regions.

Here we only have PARAMCD = "ALT", but we can pretend we have multiple parameters.

77.1 Example: Facet ALT by Sex

```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  facet_wrap(~ TRTA) +
  labs(
    title = "Mean ALT Over Time - Faceted by Treatment TRTA",
    x = "Visit",
    y = "Mean ALT (U/L)",
    color = "TRTA"
  ) +
  theme_minimal()
```

Mean ALT Over Time – Faceted by Treatment TRTA

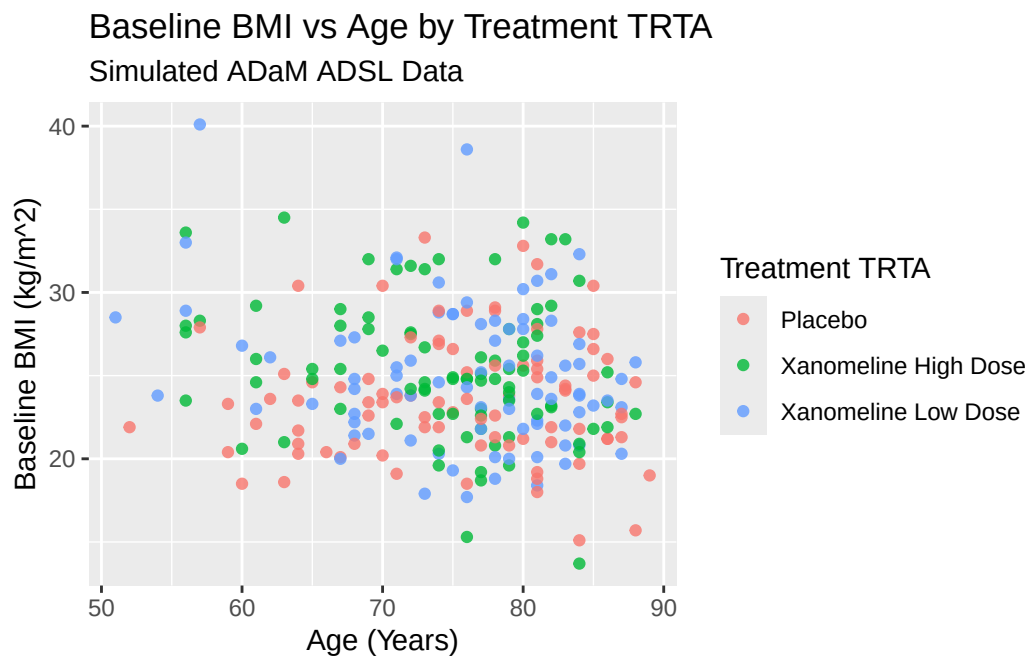


You can similarly facet by **SEX**, **RACE**, or any other ADSL variable once merged into ADLB.

78 Scales, Labels, and Themes in ADaM Context

78.1 Changing Axis Labels and Titles

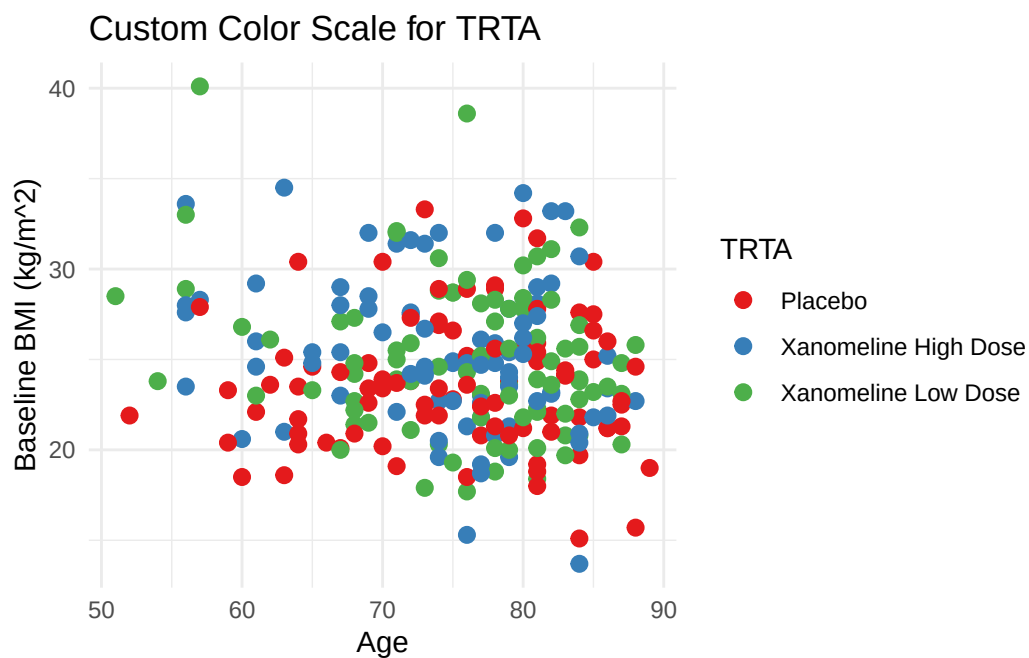
```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(alpha = 0.8) +  
  labs(  
    title = "Baseline BMI vs Age by Treatment TRTA",  
    subtitle = "Simulated ADaM ADSL Data",  
    x = "Age (Years)",  
    y = "Baseline BMI (kg/m^2)",  
    color = "Treatment TRTA"  
  )
```



78.2 Controlling Color Scales

For discrete TRTA:

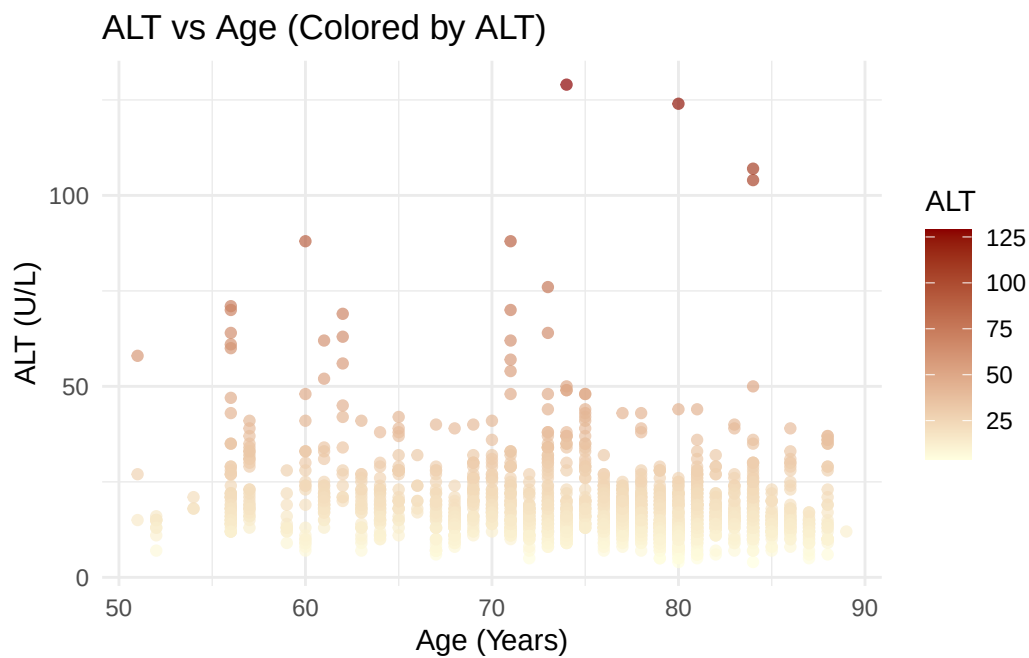
```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(size = 2.5) +  
  scale_color_brewer(palette = "Set1") +  
  labs(  
    title = "Custom Color Scale for TRTA",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```



For continuous values (e.g. ALT):

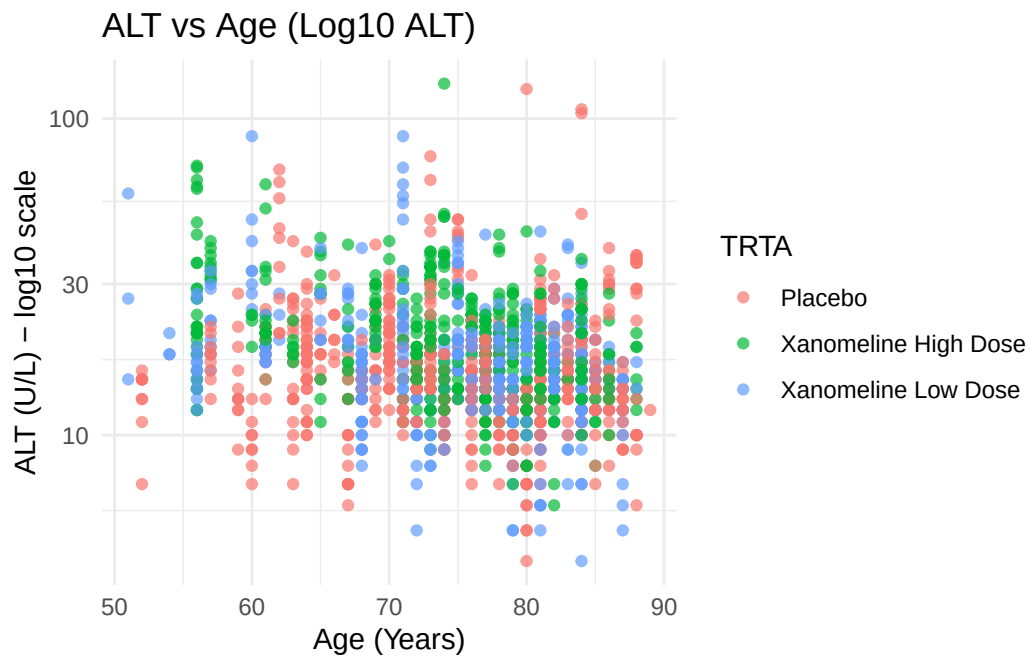
```
ggplot(ADLB, aes(x = AGE, y = AVAL, color = AVAL)) +  
  geom_point(alpha = 0.7) +  
  scale_color_gradient(low = "lightyellow", high = "darkred") +  
  labs(  
    title = "ALT vs Age (Colored by ALT)",  
    x = "Age (Years)",  
    y = "ALT (U/L)",  
    color = "ALT"  
  )
```

```
) +  
theme_minimal()
```



78.3 Transforming Scales (e.g., log10 ALT)

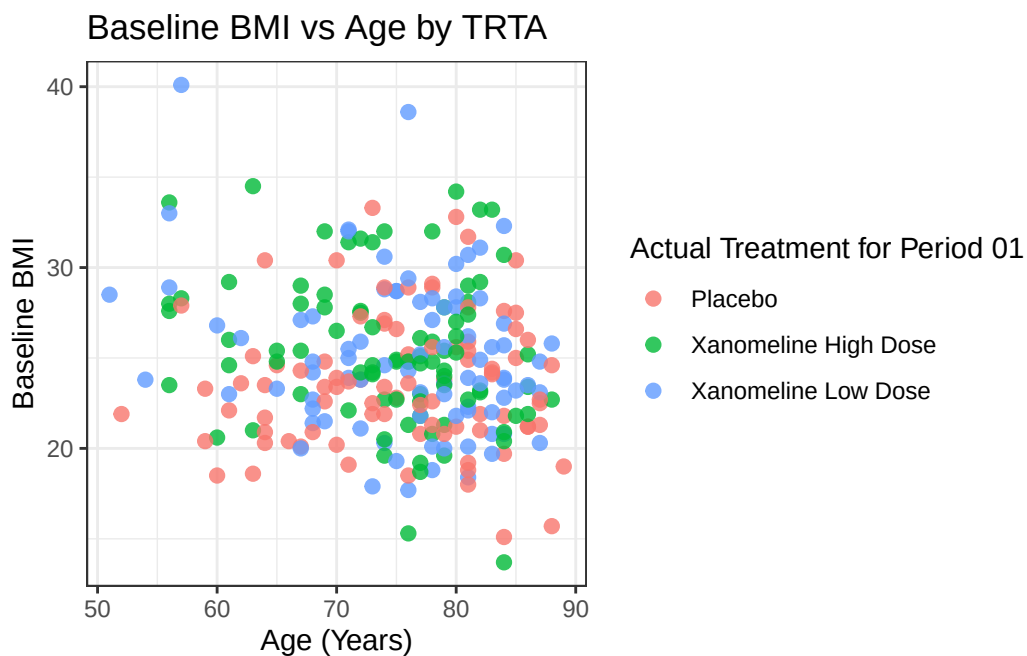
```
ggplot(ADLB, aes(x = AGE, y = AVAL, color = TRTA)) +  
  geom_point(alpha = 0.7) +  
  scale_y_log10() +  
  labs(  
    title = "ALT vs Age (Log10 ALT)",  
    x = "Age (Years)",  
    y = "ALT (U/L) - log10 scale",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```

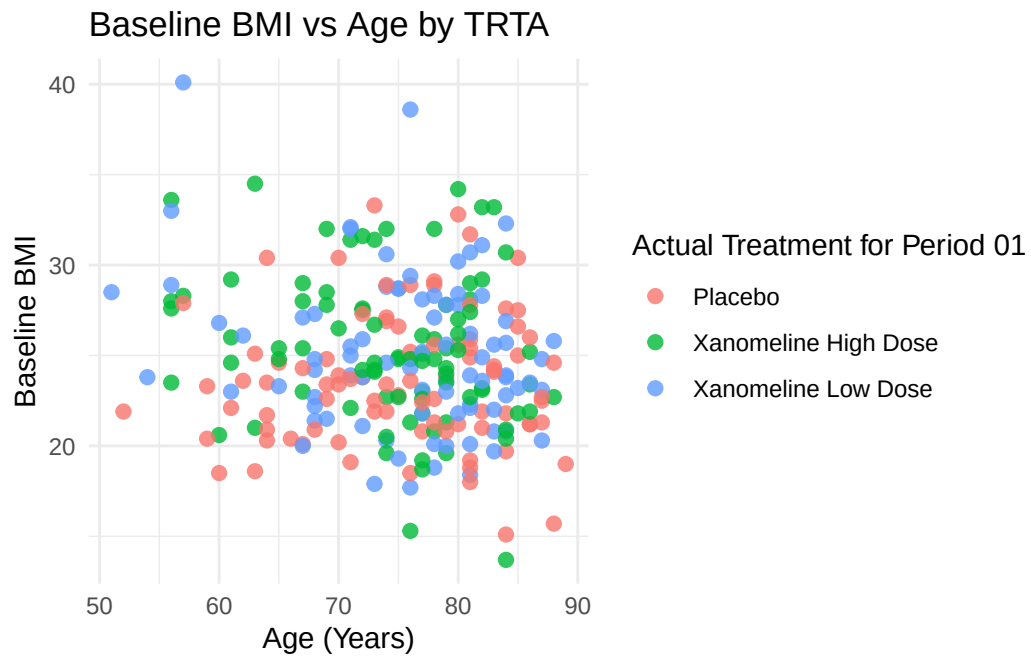
79 Themes and Publication-Style Graphics

79.1 Using Built-In Themes

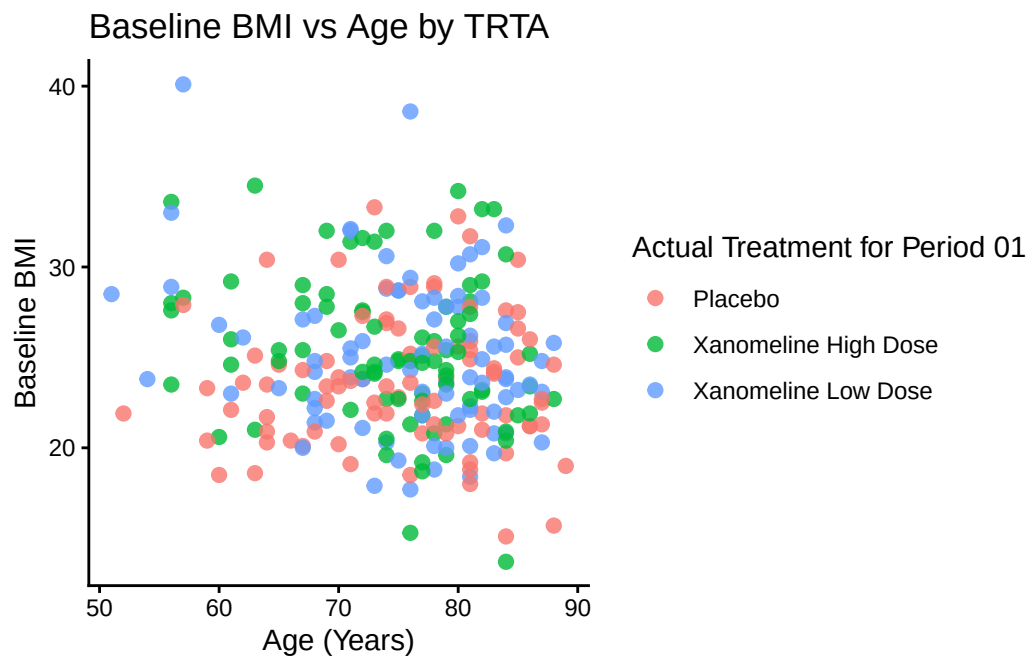
```
p_base <- ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(size = 2.2, alpha = 0.8) +  
  labs(  
    title = "Baseline BMI vs Age by TRTA",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  )  
  
p_base + theme_bw()
```



```
p_base + theme_minimal()
```

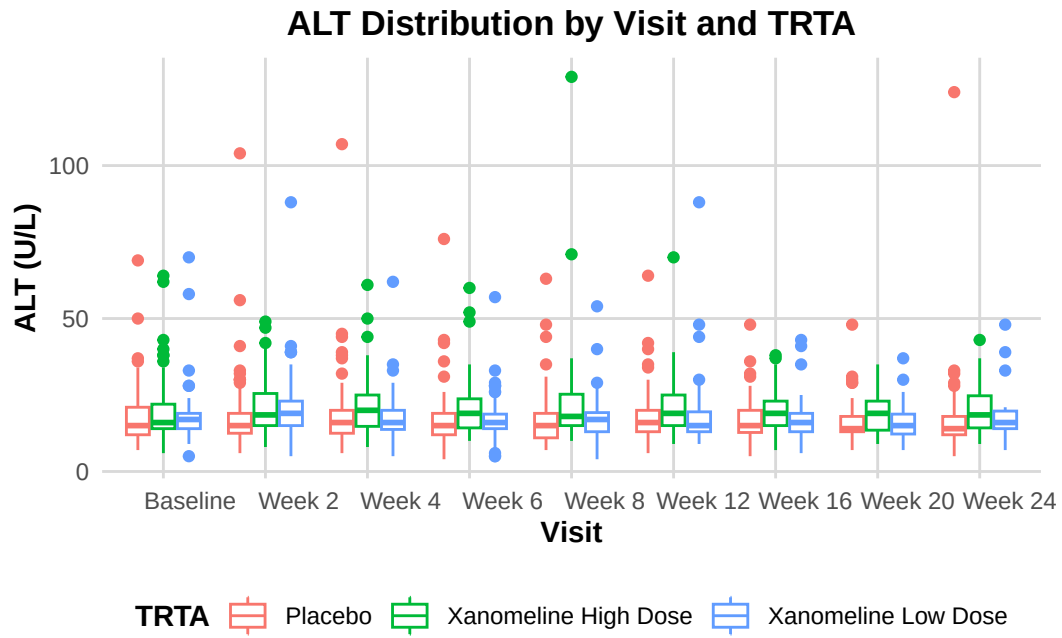


```
p_base + theme_classic()
```



79.2 Custom Theme for Clinical Reports

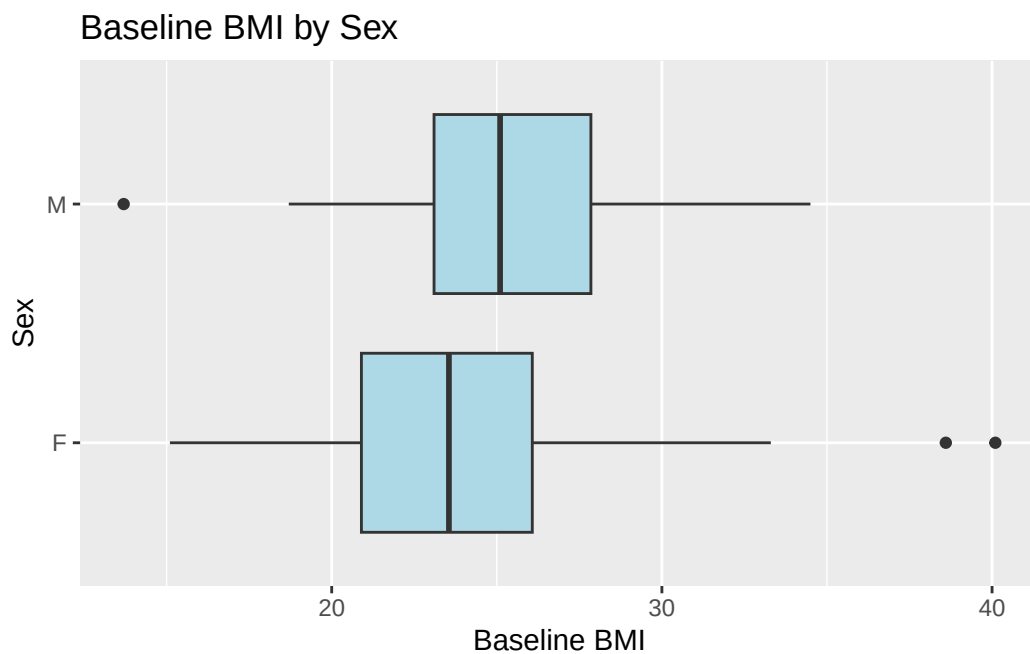
```
theme_adam_pub <- function() {  
  theme_minimal(base_size = 11) +  
    theme(  
      plot.title = element_text(face = "bold", hjust = 0.5, size = 13),  
      axis.title = element_text(face = "bold"),  
      panel.grid.major = element_line(color = "grey85"),  
      panel.grid.minor = element_blank(),  
      legend.position = "bottom",  
      legend.title = element_text(face = "bold")  
    )  
}  
  
ADLB$AVISIT <- factor(ADLB$AVISIT, levels=unique(ADLB$AVISIT[order(ADLB$AVISITN)]))  
ggplot(ADLB, aes(x = AVISIT, y = AVAL, color = TRTA)) +  
  geom_boxplot() +  
  labs(  
    title = "ALT Distribution by Visit and TRTA",  
    x = "Visit",  
    y = "ALT (U/L)",  
    color = "TRTA"  
  ) +  
  theme_adam_pub()
```



80 Coordinates and Positions

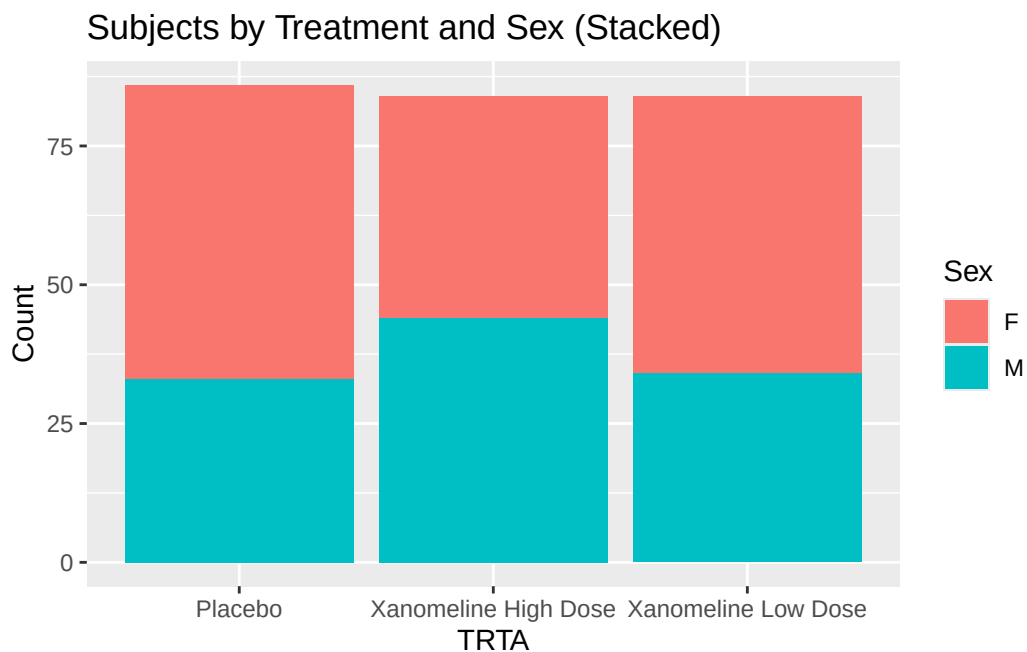
80.1 Flipping Coordinates: Boxplot of BMI by Sex

```
ggplot(ADSL, aes(x = SEX, y = BMIBL)) +  
  geom_boxplot(fill = "lightblue") +  
  coord_flip() +  
  labs(  
    title = "Baseline BMI by Sex",  
    x = "Sex",  
    y = "Baseline BMI"  
  )
```

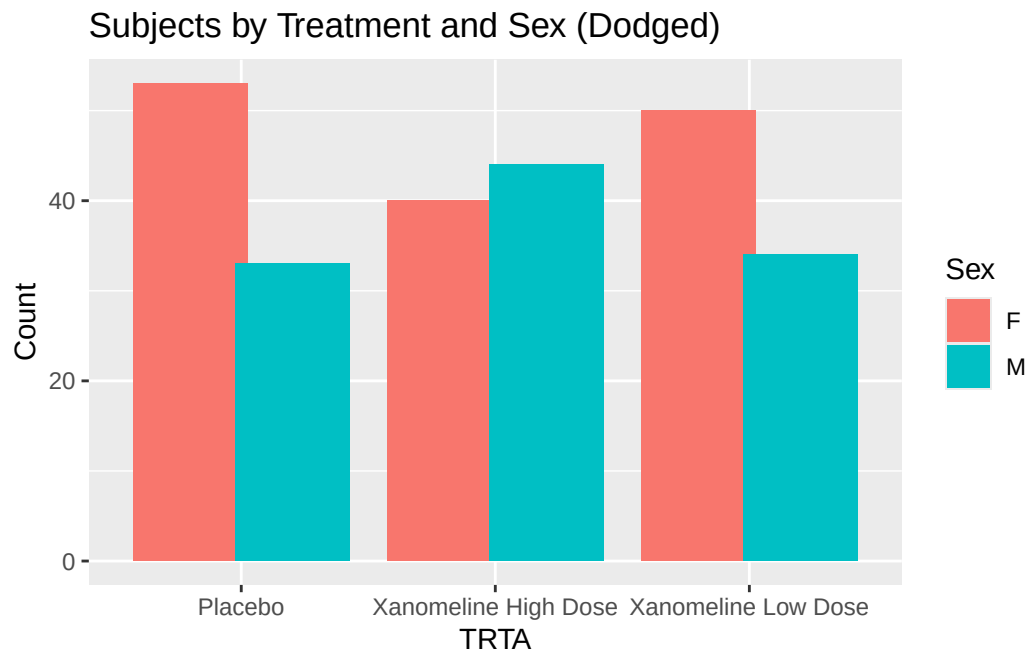


80.2 Stacked vs Dodged Bar Plots: TRTA by SEX

```
ggplot(ADSL, aes(x = TRTA, fill = SEX)) +  
  geom_bar(position = "stack") +  
  labs(  
    title = "Subjects by Treatment and Sex (Stacked)",  
    x = "TRTA",  
    y = "Count",  
    fill = "Sex"  
  )
```



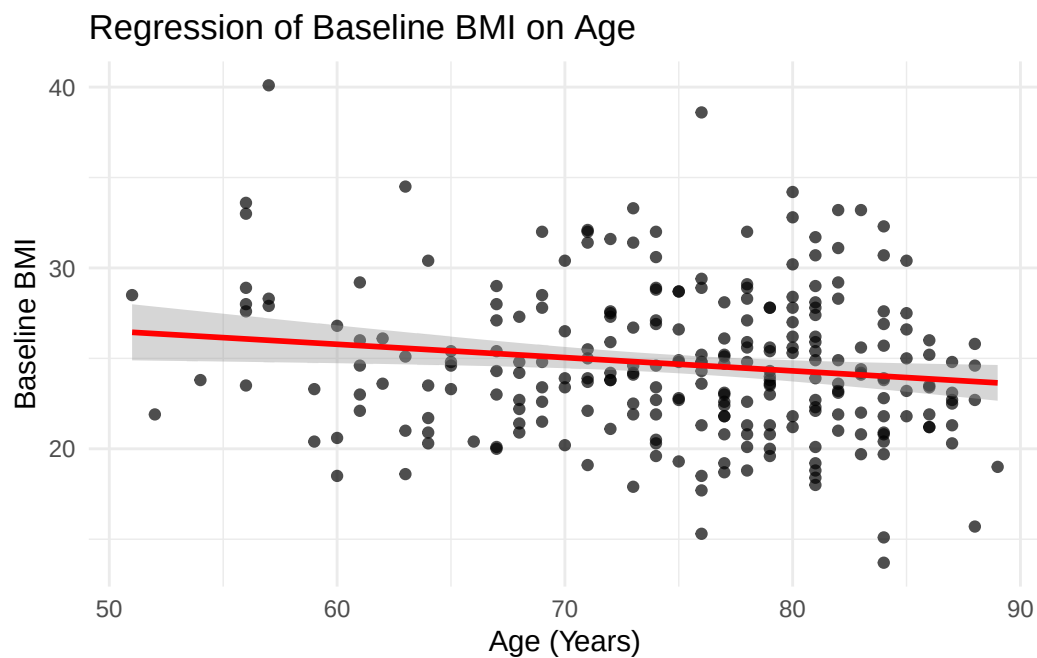
```
ggplot(ADSL, aes(x = TRTA, fill = SEX)) +  
  geom_bar(position = position_dodge(width = 0.8)) +  
  labs(  
    title = "Subjects by Treatment and Sex (Dodged)",  
    x = "TRTA",  
    y = "Count",  
    fill = "Sex"  
  )
```



81 Adding Statistical Layers

81.1 Linear Regression: BMIBL vs AGE

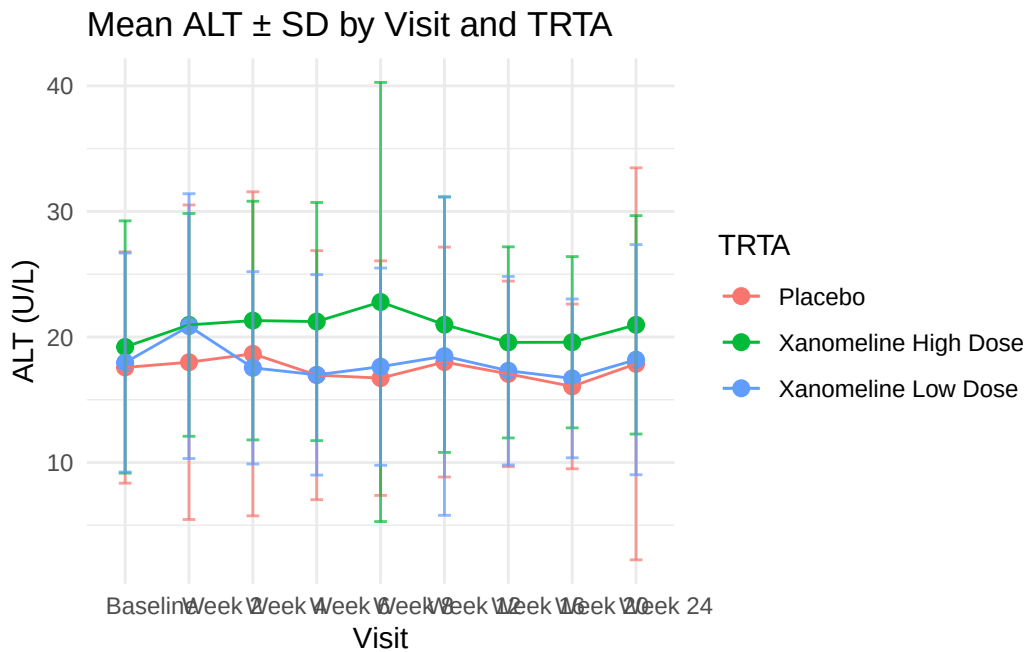
```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point(alpha = 0.7) +  
  stat_smooth(method = "lm", se = TRUE, color = "red") +  
  labs(  
    title = "Regression of Baseline BMI on Age",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_minimal()
```



81.2 Summaries: Mean ALT $\pm$ SD per Visit and TRTA

We already computed `adlb_alt_summary`. Plot mean $\pm$ SD:

```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, color = TRTA, group = TRTA)) +
  geom_point(size = 2.5) +
  geom_line() +
  geom_errorbar(
    aes(ymin = mean_ALT - sd_ALT, ymax = mean_ALT + sd_ALT),
    width = 0.2, alpha = 0.7
  ) +
  labs(
    title = "Mean ALT  $\pm$  SD by Visit and TRTA",
    x = "Visit",
    y = "ALT (U/L)",
    color = "TRTA"
  ) +
  theme_minimal()
```



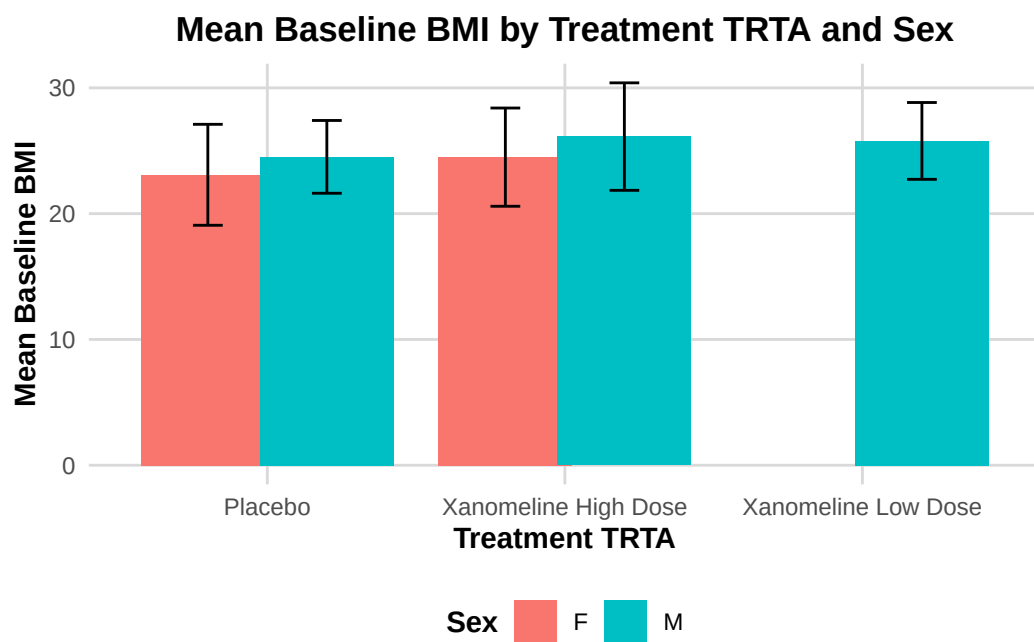
82 Tidy Workflow: dplyr + ggplot2 with ADaM

A very common pattern in clinical reporting:

1. Start from ADaM dataset
2. Filter / summarise with `dplyr`
3. Plot with `ggplot2`

Example: Mean BMI by TRTA and SEX:

```
ADSL %>%
  group_by(TRTA, SEX) %>%
  summarise(
    mean_BMIBL = mean(BMIBL),
    sd_BMIBL   = sd(BMIBL),
    n          = n(),
    .groups = "drop"
  ) %>%
  ggplot(aes(x = TRTA, y = mean_BMIBL, fill = SEX)) +
  geom_col(position = position_dodge(width = 0.8)) +
  geom_errorbar(
    aes(ymin = mean_BMIBL - sd_BMIBL, ymax = mean_BMIBL + sd_BMIBL),
    position = position_dodge(width = 0.8),
    width = 0.2
  ) +
  labs(
    title = "Mean Baseline BMI by Treatment TRTA and Sex",
    x = "Treatment TRTA",
    y = "Mean Baseline BMI",
    fill = "Sex"
  ) +
  theme_adam_pub()
```



83 Advanced: Simple Kaplan–Meier Plot from ADTTE

In ADaM, **ADTTE** contains time-to-event information (e.g. PFS, OS).

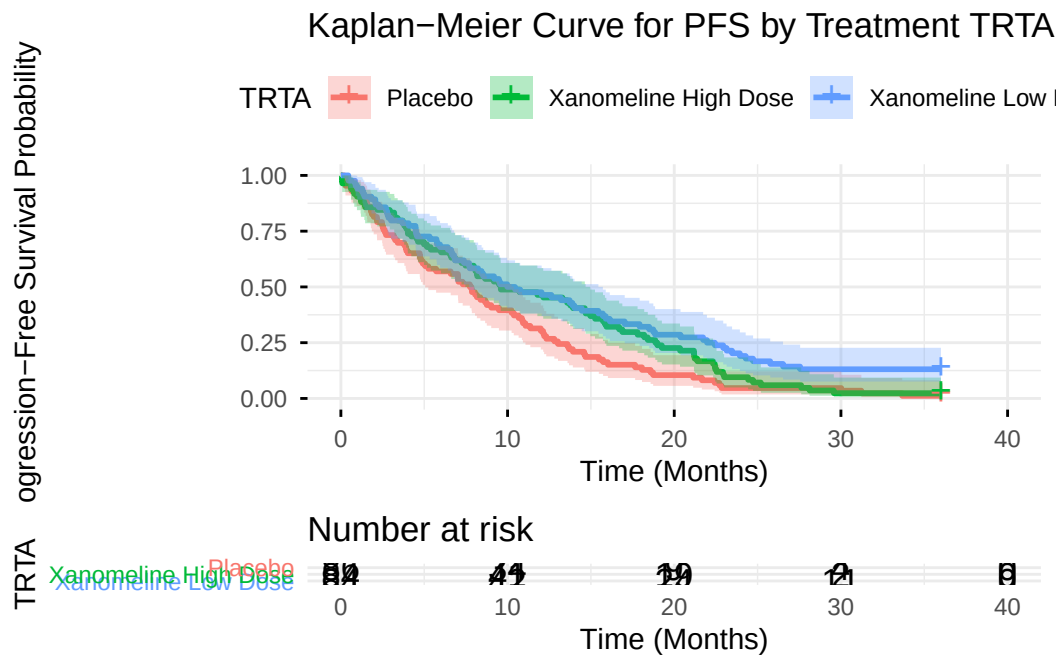
Here we show a simple KM plot using `survminer::ggsurvplot()` which uses `ggplot2` under the hood.

```
adtte_pfs <- ADTTE %>%
  filter(PARAMCD == "PFS")

fit_pfs <- survfit(Surv(AVAL, 1 - CNSR) ~ TRTA, data = adtte_pfs)

km_plot <- ggsurvplot(
  fit_pfs,
  data = adtte_pfs,
  conf.int = TRUE,
  risk.table = TRUE,
  risk.table.height = 0.25,
  ggtheme = theme_minimal(),
  legend.title = "TRTA",
  legend.labs = levels(factor(adtte_pfs$TRTA)),
  title = "Kaplan-Meier Curve for PFS by Treatment TRTA",
  xlab = "Time (Months)",
  ylab = "Progression-Free Survival Probability"
)

km_plot
```



For a **pure ggplot2** approach, you could extract the `survfit` summary into a data frame and use `geom_step()` manually, but `ggsurvplot()` is convenient and still fully compatible with ggplot theming.

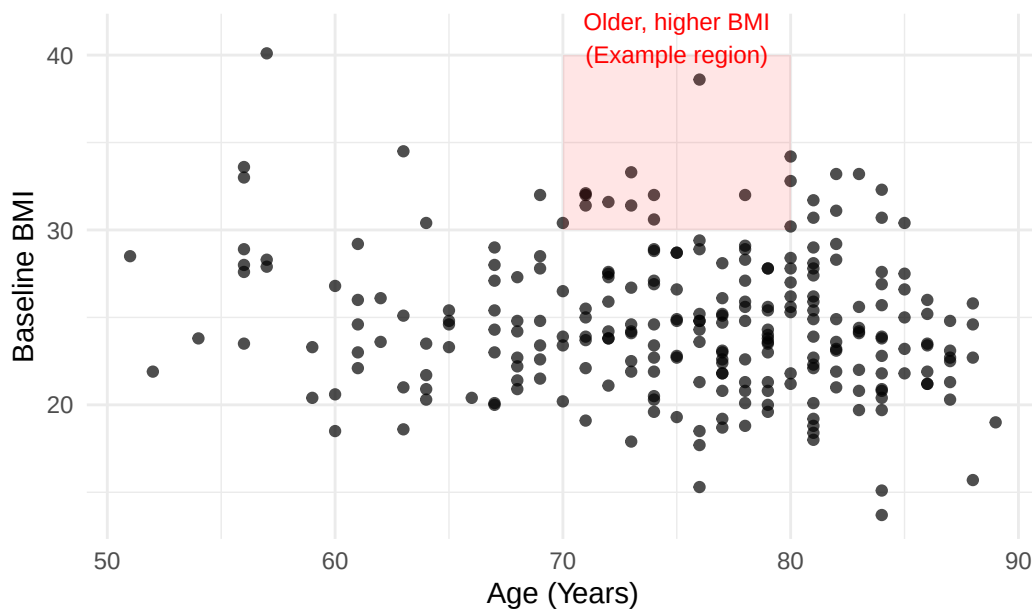
84 Advanced: Annotations and Secondary Axes

84.1 Annotations on an ADaM Plot

Example: highlight older patients with high BMI:

```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point(alpha = 0.7) +  
  annotate(  
    "rect", xmin = 70, xmax = 80,  
    ymin = 30, ymax = 40,  
    alpha = 0.1, fill = "red"  
  ) +  
  annotate(  
    "text", x = 75, y = 41,  
    label = "Older, higher BMI  
(Example region)",  
    size = 3, color = "red"  
  ) +  
  labs(  
    title = "Annotations on ADSL Scatterplot",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_minimal()
```

Annotations on ADSL Scatterplot

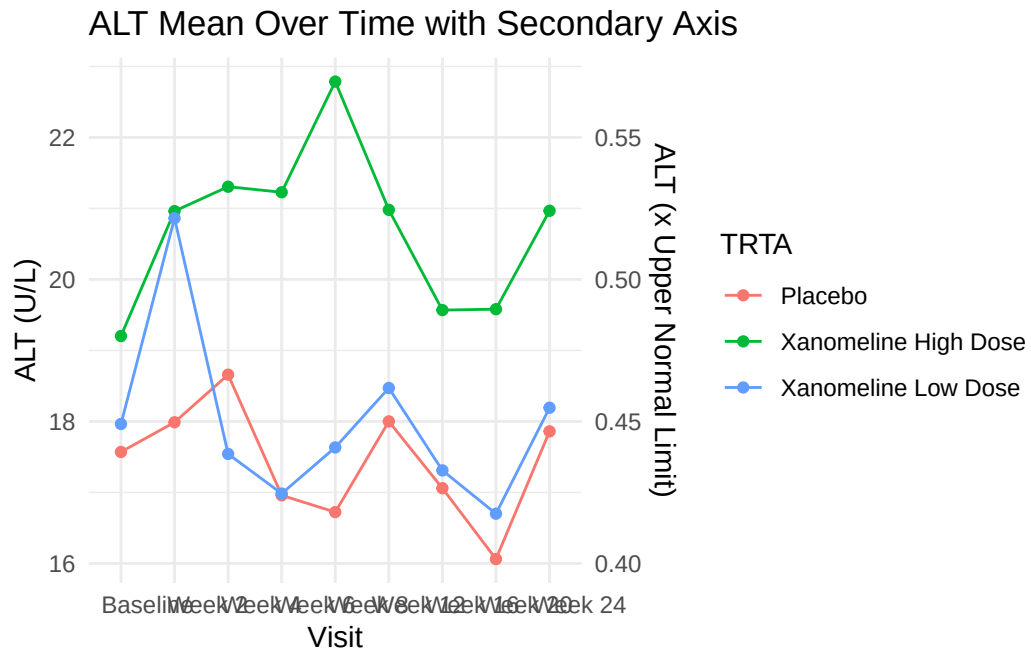


84.2 Secondary Axis (Use with Care)

ggplot2 allows secondary axes only if they are a simple transformation of the primary axis.

Example (purely illustrative) converting ALT to approximate multiple:

```
ggplot(adlb_alt_summary, aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +  
  geom_line() +  
  geom_point() +  
  scale_y_continuous(  
    name = "ALT (U/L)",  
    sec.axis = sec_axis(~ . / 40, name = "ALT (x Upper Normal Limit)")  
  ) +  
  labs(  
    title = "ALT Mean Over Time with Secondary Axis",  
    x = "Visit",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```

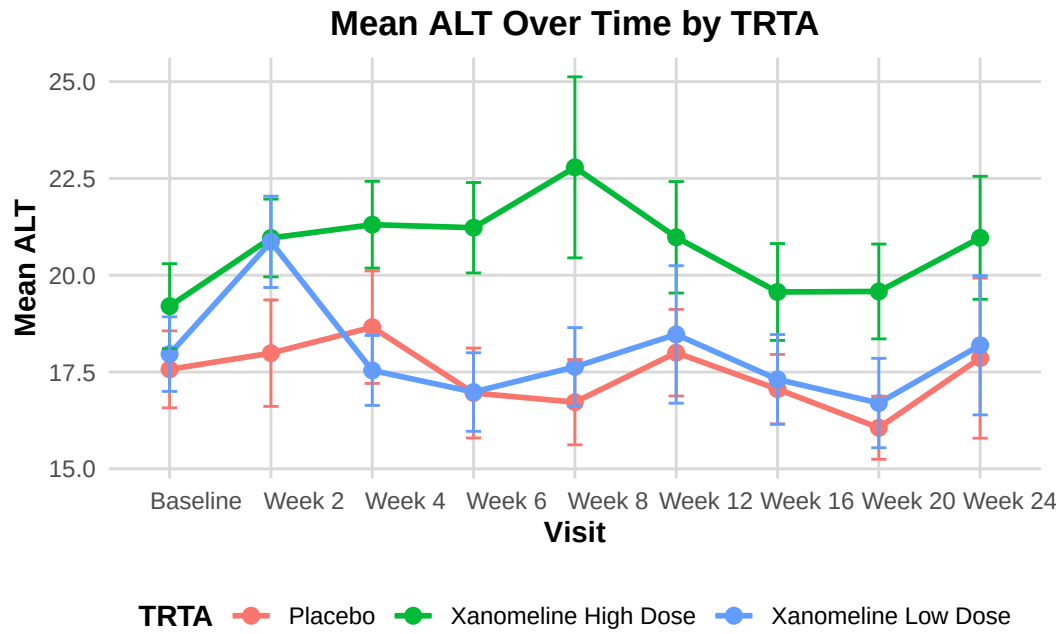



In production, ensure the secondary axis has a clinically correct and meaningful transformation.

85 Writing Reusable Plot Functions for ADaM

You can encapsulate standard plots (e.g., lab time course by TRTA) into functions.

```
plot_lab_timecourse <- function(adlb_df, paramcd = "ALT") {  
  df <- adlb_df %>%  
    filter(PARAMCD == paramcd) %>%  
    group_by(TRTA, AVISIT) %>%  
    summarise(  
      mean_aval = mean(AVAL),  
      se_aval   = sd(AVAL) / sqrt(n()),  
      .groups = "drop"  
    )  
  
  ggplot(df, aes(x = AVISIT, y = mean_aval, color = TRTA, group = TRTA)) +  
    geom_line(linewidth = 1) +  
    geom_point(size = 2.5) +  
    geom_errorbar(  
      aes(ymin = mean_aval - se_aval, ymax = mean_aval + se_aval),  
      width = 0.15  
    ) +  
    labs(  
      title = paste("Mean", paramcd, "Over Time by TRTA"),  
      x = "Visit",  
      y = paste("Mean", paramcd),  
      color = "TRTA"  
    ) +  
    theme_adam_pub()  
}  
  
plot_lab_timecourse(ADLB, paramcd = "ALT")
```



This approach is very powerful for building **standardized plotting utilities** for your ADaM pipeline.

86 Saving Plots for Reports

Use `ggsave()` to save plots for use in RTF, Word, or PowerPoint.

```
p <- ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRTA)) +  
  geom_point() +  
  labs(  
    title = "Baseline BMI vs Age by TRTA",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_adam_pub()  
  
# Save as PNG  
ggsave("bmi_vs_age_by_TRTA.png", plot = p, width = 6, height = 4, dpi = 300)  
  
# Save as PDF  
ggsave("bmi_vs_age_by_TRTA.pdf", plot = p, width = 6, height = 4)
```

87 Introduction

This document walks through **six common oncology graphs** step by step, using the exact code you provided:

1. Bar graph of **sum of lesion diameters** plus individual lesions
2. **Waterfall plot** of best percentage change from baseline
3. **Swimmer plot** for duration of response
4. **Spider plot** (tumor trajectories over time)
5. **Kaplan–Meier** survival curve
6. **Forest plot** for subgroup hazard ratios

For each figure, we will:

- Build a simple **example dataset** in R (similar structure to what you'd get from ADaM like ADTR/ADTTE).
- Explain the main **ggplot2 layers** and options.
- Show the **final code** to generate the plot.

In a real clinical project, you would replace the toy data here with your ADaM datasets, but the plotting logic stays the same.

88 Figure 1 – Bar Graph with Individual Lesion Lines

This plot shows, for a single subject:

- Bars: **Sum of Lesion Diameters (SUMLD)** at each visit
- Lines/points: each **individual lesion** (tumor) over time

Clinically, this helps you see how the total tumor burden and individual lesions evolve across visits.

88.1 1.1 Create bar-graph data

We first build a small dataset with:

- `visit`: visit label
- `SUMLD`: sum of lesion diameters
- `trstresn_tumor1/2/3`: individual lesion diameters

```
bar_data <- data.frame(  
  visit = c("Baseline", "Week 4", "Week 8", "Week 12", "Week 16"),  
  SUMLD = c(100, 85, 70, 65, 60), # Sum of Lesion Diameters  
  trstresn_tumor1 = c(50, 45, 35, 30, 28),  
  trstresn_tumor2 = c(30, 25, 20, 20, 18),  
  trstresn_tumor3 = c(20, 15, 15, 15, 14)  
)
```

`bar_data`

	visit	SUMLD	trstresn_tumor1	trstresn_tumor2	trstresn_tumor3
1	Baseline	100	50	30	20
2	Week 4	85	45	25	15
3	Week 8	70	35	20	15
4	Week 12	65	30	20	15
5	Week 16	60	28	18	14

In real ADaM:

- `visit` could map to `AVISIT`,
- `SUMLD` would be a derived variable (sum of target lesions),
- `trstresn_tumorX` are separate lesions (often separate rows in ADTR; here we keep them as columns for simplicity).

88.2 1.2 Reshape to long format for lines

To draw multiple lesions as separate lines, we reshape the wide columns (`trstresn_tumor1/2/3`) into a long format using `pivot_longer()`:

```
bar_data_long <- bar_data %>%
  pivot_longer(
    cols = starts_with("trstresn"),
    names_to = "tuloc",
    values_to = "trstresn"
  )

bar_data_long
```

```
# A tibble: 15 x 4
  visit    SUMLD tuloc      trstresn
  <chr>    <dbl> <chr>      <dbl>
1 Baseline  100 trstresn_tumor1    50
2 Baseline  100 trstresn_tumor2    30
3 Baseline  100 trstresn_tumor3    20
4 Week 4    85 trstresn_tumor1    45
5 Week 4    85 trstresn_tumor2    25
6 Week 4    85 trstresn_tumor3    15
7 Week 8    70 trstresn_tumor1    35
8 Week 8    70 trstresn_tumor2    20
9 Week 8    70 trstresn_tumor3    15
10 Week 12   65 trstresn_tumor1    30
11 Week 12   65 trstresn_tumor2    20
12 Week 12   65 trstresn_tumor3    15
13 Week 16   60 trstresn_tumor1    28
14 Week 16   60 trstresn_tumor2    18
15 Week 16   60 trstresn_tumor3    14
```

- `tuloc` acts like **tumor location / lesion identifier**.

- `trstresn` holds the lesion diameter for each visit-lesion combination.

88.3 1.3 Build the bar + line plot

Now we combine:

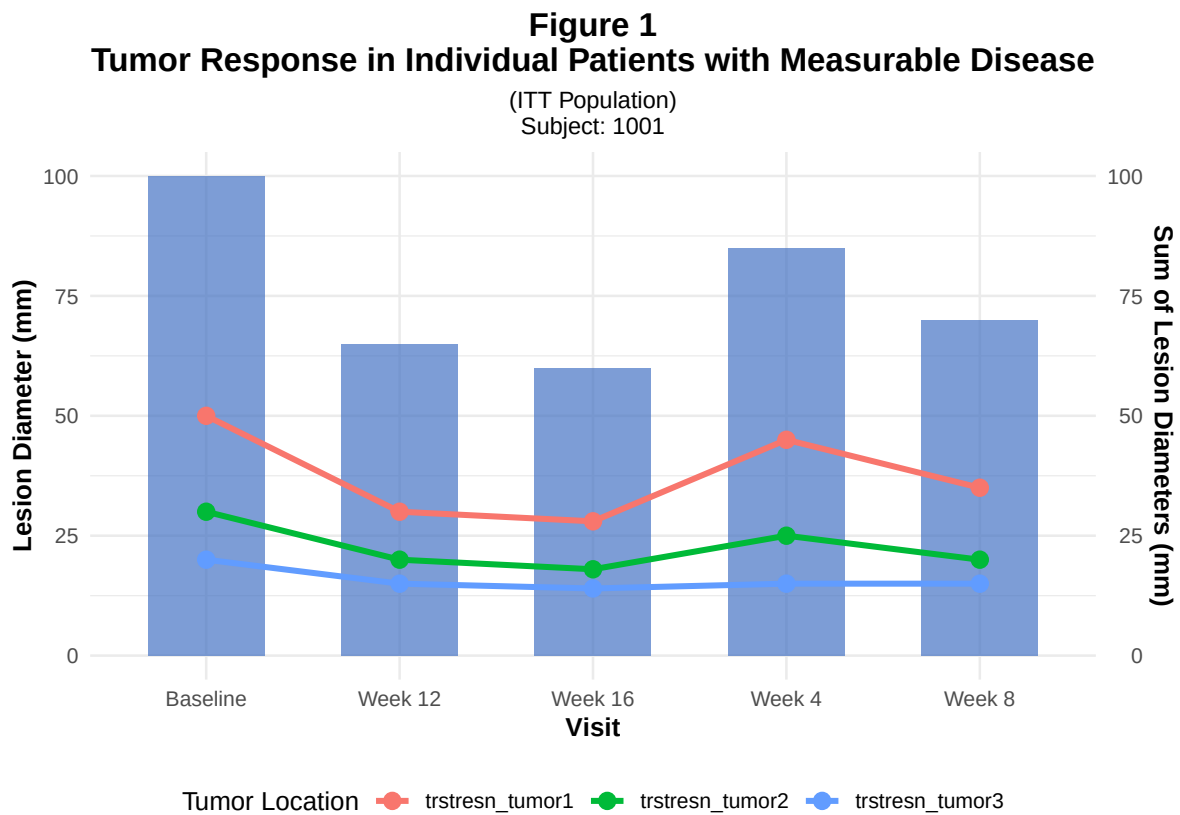
- `geom_col()` for SUMLD (bars)
- `geom_line()` and `geom_point()` for individual lesions
- `scale_y_continuous()` with a **secondary axis** label (note: both axes use same numeric scale, only labels differ)

```
p1 <- ggplot() +
  # Bars = Sum of lesion diameters per visit
  geom_col(
    data = bar_data,
    aes(x = visit, y = SUMLD),
    fill = "#4472C4",
    width = 0.6,
    alpha = 0.7
  ) +
  # Lines = individual lesions
  geom_line(
    data = bar_data_long,
    aes(x = visit, y = trstresn,
        group = tuloc, color = tuloc),
    size = 1.2
  ) +
  # Points = individual lesion measurements
  geom_point(
    data = bar_data_long,
    aes(x = visit, y = trstresn,
        color = tuloc),
    size = 3
  ) +
  # Primary and secondary y-axes (same numeric scale)
  scale_y_continuous(
    name = "Lesion Diameter (mm)",
    sec.axis = sec_axis(~., name = "Sum of Lesion Diameters (mm)")
  ) +
  labs(
    title = "Figure 1"
```



```
Tumor Response in Individual Patients with Measurable Disease",
  subtitle = "(ITT Population)
Subject: 1001",
  x = "Visit",
  color = "Tumor Location"
) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  plot.subtitle = element_text(hjust = 0.5, size = 10),
  axis.title = element_text(face = "bold", size = 11),
  legend.position = "bottom"
)
```

p1



Key points:

- We used an **empty** `ggplot()` and passed data separately to each layer.
 - This is useful when we have different data frames for bars (`bar_data`) and lines (`bar_data_long`).
-

89 Figure 2 – Waterfall Plot of Best Percentage Change from Baseline

Waterfall plots show each patient's **best % change from baseline** in tumor size. They're very common in oncology to visualize response categories like CR/PR/SD/PD.

89.1 2.1 Create waterfall data

We simulate:

- **patientid**: patient identifier
- **pchg**: best % change from baseline in sum of diameters
- **response**: categorical RECIST-like response (CR/PR/SD/PD)

```
waterfall_data <- data.frame(  
  patientid = paste0("PT", sprintf("%03d", 1:30)),  
  pchg = c(-65, -55, -48, -42, -38, -35, -30, -28, -25, -22,  
           -18, -15, -12, -10, -8, -5, -2, 5, 8, 12,  
           15, 20, 25, 30, 35, 40, 45, 50, 55, 60),  
  response = c(rep("CR", 2), rep("PR", 13), rep("SD", 10), rep("PD", 5))  
)  
  
waterfall_data
```

	patientid	pchg	response
1	PT001	-65	CR
2	PT002	-55	CR
3	PT003	-48	PR
4	PT004	-42	PR
5	PT005	-38	PR
6	PT006	-35	PR
7	PT007	-30	PR
8	PT008	-28	PR
9	PT009	-25	PR

10	PT010	-22	PR
11	PT011	-18	PR
12	PT012	-15	PR
13	PT013	-12	PR
14	PT014	-10	PR
15	PT015	-8	PR
16	PT016	-5	SD
17	PT017	-2	SD
18	PT018	5	SD
19	PT019	8	SD
20	PT020	12	SD
21	PT021	15	SD
22	PT022	20	SD
23	PT023	25	SD
24	PT024	30	SD
25	PT025	35	SD
26	PT026	40	PD
27	PT027	45	PD
28	PT028	50	PD
29	PT029	55	PD
30	PT030	60	PD

89.2 2.2 Sort patients by change

To get the typical waterfall shape (bars sorted from best to worst response), we sort by `pchg` and then set factor levels:

```
waterfall_data <- waterfall_data %>%
  arrange(pchg) %>%
  mutate(patientid = factor(patientid, levels = patientid))

response_colors <- c(
  "CR" = "#00B050",
  "PR" = "#92D050",
  "SD" = "#FFC000",
  "PD" = "#FF0000"
)

waterfall_data
```

```
patientid pchg response
```

1	PT001	-65	CR
2	PT002	-55	CR
3	PT003	-48	PR
4	PT004	-42	PR
5	PT005	-38	PR
6	PT006	-35	PR
7	PT007	-30	PR
8	PT008	-28	PR
9	PT009	-25	PR
10	PT010	-22	PR
11	PT011	-18	PR
12	PT012	-15	PR
13	PT013	-12	PR
14	PT014	-10	PR
15	PT015	-8	PR
16	PT016	-5	SD
17	PT017	-2	SD
18	PT018	5	SD
19	PT019	8	SD
20	PT020	12	SD
21	PT021	15	SD
22	PT022	20	SD
23	PT023	25	SD
24	PT024	30	SD
25	PT025	35	SD
26	PT026	40	PD
27	PT027	45	PD
28	PT028	50	PD
29	PT029	55	PD
30	PT030	60	PD

89.3 2.3 Build the waterfall plot

- `geom_col()` for bars
- `geom_hline()` to draw RECIST cut-off lines (e.g. -30%, +20%)
- `coord_cartesian()` to fix y-axis limits

```
p2 <- ggplot(waterfall_data, aes(x = patientid, y = pchg, fill = response)) +
  geom_col(width = 0.8) +
```

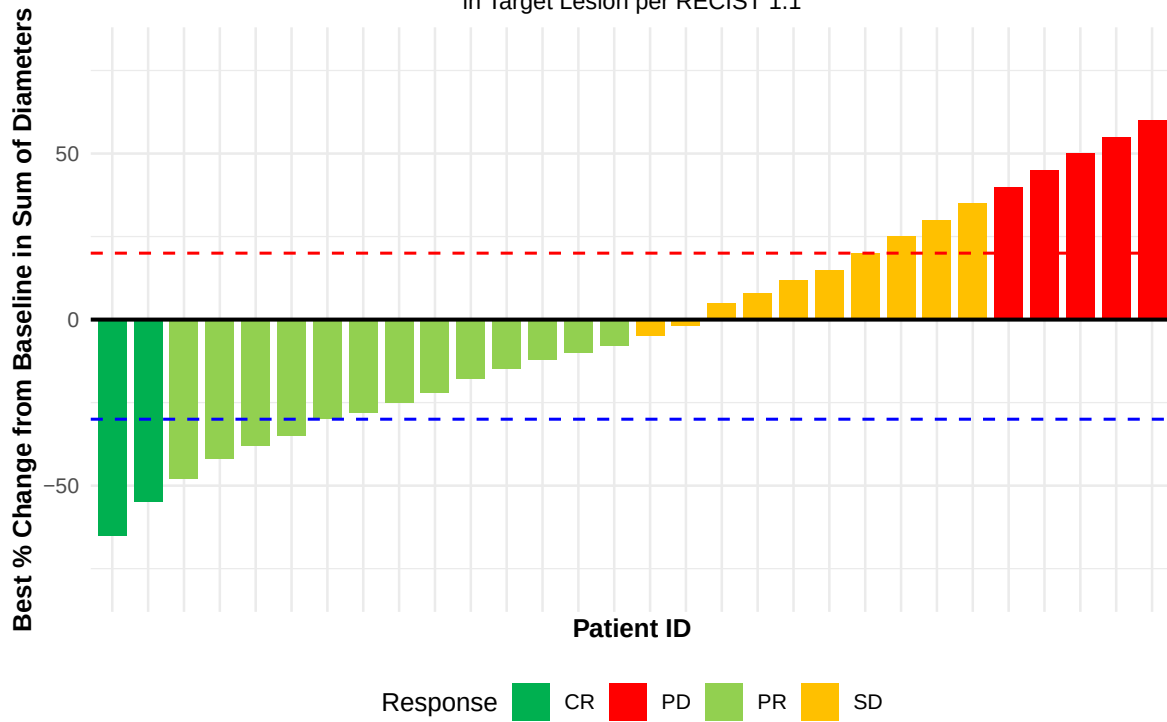
```

# Baseline line at 0% change
geom_hline(yintercept = 0, linetype = "solid", color = "black", size = 0.8) +
# Typical RECIST thresholds
geom_hline(yintercept = -30, linetype = "dashed", color = "blue", size = 0.6) +
geom_hline(yintercept = 20, linetype = "dashed", color = "red", size = 0.6) +
scale_fill_manual(values = response_colors) +
labs(
  title = "Figure 2
Waterfall Plot of Best Percentage Change from Baseline",
  subtitle = "in Target Lesion per RECIST 1.1",
  x = "Patient ID",
  y = "Best % Change from Baseline in Sum of Diameters",
  fill = "Response"
) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  plot.subtitle = element_text(hjust = 0.5, size = 10),
  axis.title = element_text(face = "bold", size = 11),
  axis.text.x = element_blank(),
  axis.ticks.x = element_blank(),
  legend.position = "bottom"
) +
coord_cartesian(ylim = c(-80, 80))

```

p2

Figure 2
Waterfall Plot of Best Percentage Change from Baseline
 in Target Lesion per RECIST 1.1



In real ADaM:

- You'd compute `pchg` from ADTR using baseline and best post-baseline sum of diameters.

90 Figure 3 – Swimmer Plot: Duration of Response

Swimmer plots show **time on treatment** / **response** for each patient:

- Horizontal bar per patient: **treatment duration**
- Color: response category
- Symbols/markers: progression, death, etc.

90.1 3.1 Create swimmer data

We simulate:

- **start_time** / **end_time**: treatment/response duration (months)
- **response**: CR/PR/SD/PD
- **progression** / **death**: logical indicators

```
set.seed(123) # for reproducibility

swimmer_data <- data.frame(
  patientid = paste0("PT", sprintf("%03d", 1:20)),
  start_time = rep(0, 20),
  end_time = c(12, 15, 8, 20, 18, 14, 10, 22, 16, 19,
               7, 25, 13, 17, 11, 9, 21, 24, 15, 18),
  response = sample(c("CR", "PR", "SD", "PD"), 20, replace = TRUE),
  progression = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE,
                  TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE),
  death = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, TRUE,
            FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE)
)

# Sort by duration so longest bars at top
swimmer_data <- swimmer_data %>%
```



```

arrange(desc(end_time)) %>%
mutate(patientid = factor(patientid, levels = patientid))

swimmer_data

```

	patientid	start_time	end_time	response	progression	death
1	PT012	0	25	PR	FALSE	FALSE
2	PT018	0	24	CR	FALSE	TRUE
3	PT008	0	22	PR	FALSE	FALSE
4	PT017	0	21	PD	TRUE	FALSE
5	PT004	0	20	PR	FALSE	FALSE
6	PT010	0	19	CR	FALSE	TRUE
7	PT005	0	18	SD	TRUE	FALSE
8	PT020	0	18	SD	FALSE	FALSE
9	PT014	0	17	CR	FALSE	TRUE
10	PT009	0	16	SD	TRUE	FALSE
11	PT002	0	15	SD	FALSE	TRUE
12	PT019	0	15	SD	TRUE	FALSE
13	PT006	0	14	PR	FALSE	TRUE
14	PT013	0	13	PR	TRUE	FALSE
15	PT001	0	12	SD	TRUE	FALSE
16	PT015	0	11	PR	TRUE	FALSE
17	PT007	0	10	PR	TRUE	FALSE
18	PT016	0	9	SD	FALSE	FALSE
19	PT003	0	8	SD	TRUE	FALSE
20	PT011	0	7	PD	TRUE	FALSE

90.2 3.2 Build the swimmer plot

- `geom_segment()` draws horizontal bars
- Triangles (`shape = 17`) mark progression
- Crosses (`shape = 4`) mark death

```

p3 <- ggplot(swimmer_data, aes(y = patientid)) +
  # Treatment/response duration
  geom_segment(
    aes(x = start_time, xend = end_time,
        yend = patientid, color = response),

```

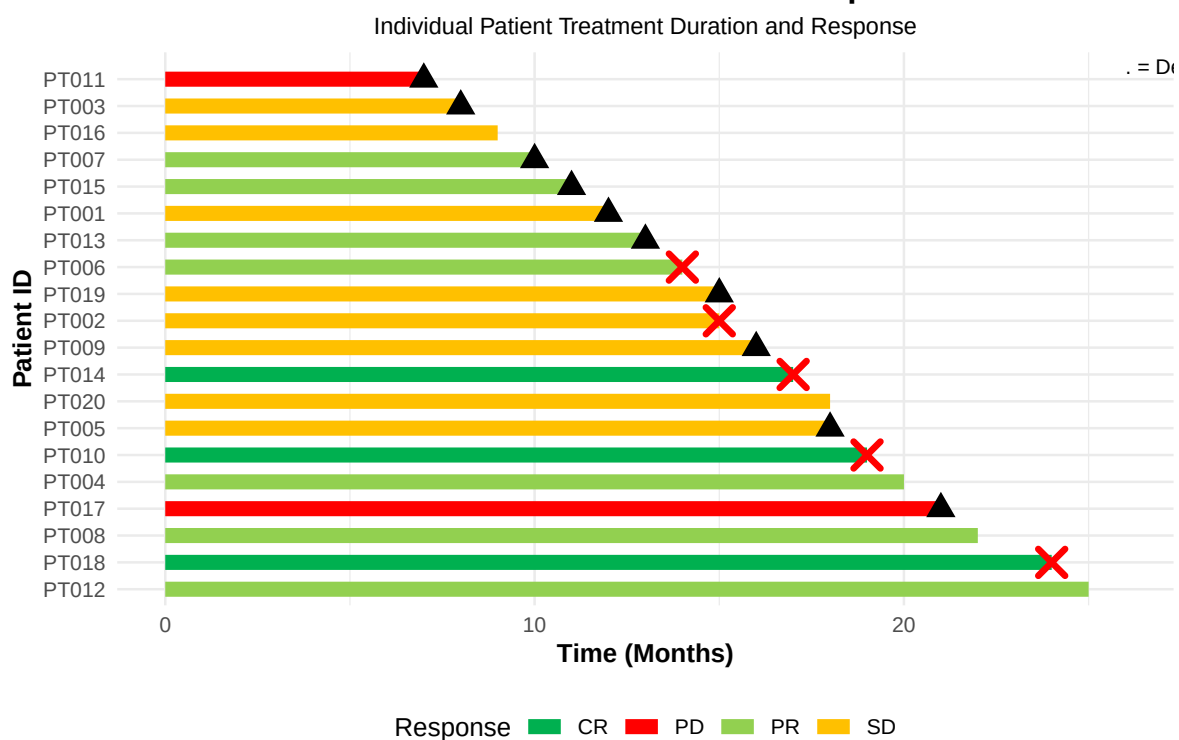
```

    size = 3
  ) +
  # progression (black triangle)
  geom_point(
    data = swimmer_data %>% filter(progression),
    aes(x = end_time),
    shape = 17, size = 4, color = "black"
  ) +
  # death (red cross)
  geom_point(
    data = swimmer_data %>% filter(death),
    aes(x = end_time),
    shape = 4, size = 4, color = "red", stroke = 2
  ) +
  scale_color_manual(values = c(
    "CR" = "#00B050", "PR" = "#92D050",
    "SD" = "#FFC000", "PD" = "#FF0000"
  )) +
  labs(
    title = "Figure 3
Swimmer Plot - Duration of Response",
    subtitle = "Individual Patient Treatment Duration and Response",
    x = "Time (Months)",
    y = "Patient ID",
    color = "Response"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    plot.subtitle = element_text(hjust = 0.5, size = 10),
    axis.title = element_text(face = "bold", size = 11),
    legend.position = "bottom"
  ) +
  annotate(
    "text", x = 26, y = 21,
    label = " = Progression
= Death",
    hjust = 0, size = 3
  )

```

p3

Figure 3
Swimmer Plot – Duration of Response



In an ADaM workflow, these durations often come from ADTTE / custom response datasets (first response date, progression date, death date, etc.).

91 Figure 4 – Spider Plot: Tumor Trajectories Over Time

Spider plots show multiple **patients' tumor trajectories** on the same axes:

- x-axis: time (e.g. months from baseline scan)
- y-axis: relative tumor size (ratio vs baseline)

Each line represents one patient.

91.1 4.1 Create spider data

We create data for 15 patients, with tumor size values over time:

```
set.seed(123)

spider_data <- expand_grid(
  patientid = paste0("PT", sprintf("%03d", 1:15)),
  dur = seq(-10, 100, by = 10)
) %>%
  group_by(patientid) %>%
  mutate(
    # Simple random trend around baseline = 1
    size = 1 + rnorm(1, 0, 0.3) + (dur/100) * rnorm(1, -0.5, 0.4)
  ) %>%
  ungroup()

spider_data
```

```
# A tibble: 180 x 3
  patientid  dur  size
  <fct>      <dbl> <dbl>
1 PT001     -10 0.891
```

```

2 PT002      -10 1.51
3 PT003      -10 1.02
4 PT004      -10 1.24
5 PT005      -10 0.862
6 PT006      -10 1.40
7 PT007      -10 1.17
8 PT008      -10 0.812
9 PT009      -10 1.28
10 PT010     -10 1.28
# i 170 more rows

```

- At `dur = 0` we are around `size = 1` (baseline).
- Later timepoints can go up (> 1) or down (< 1).

91.2 4.2 Build the spider plot

```

p4 <- ggplot(
  spider_data,
  aes(
    x = dur, y = size,
    group = patientid,
    color = patientid
  )
) +
  geom_line(size = 1.2, alpha = 0.7) +
  geom_point(size = 2, alpha = 0.8) +
  # Horizontal line at baseline (1.0)
  geom_hline(
    yintercept = 1.0, linetype = "solid",
    color = "black", size = 0.8
  ) +
  scale_y_continuous(breaks = seq(0, 2, by = 0.20)) +
  scale_x_continuous(breaks = seq(-100, 100, by = 10)) +
  labs(
    title = "Figure 4
Relative Change in Tumor Size",
    x = "Months from Baseline Scan (Time 0)",
    y = "Tumor Size Relative to Baseline"
  ) +

```

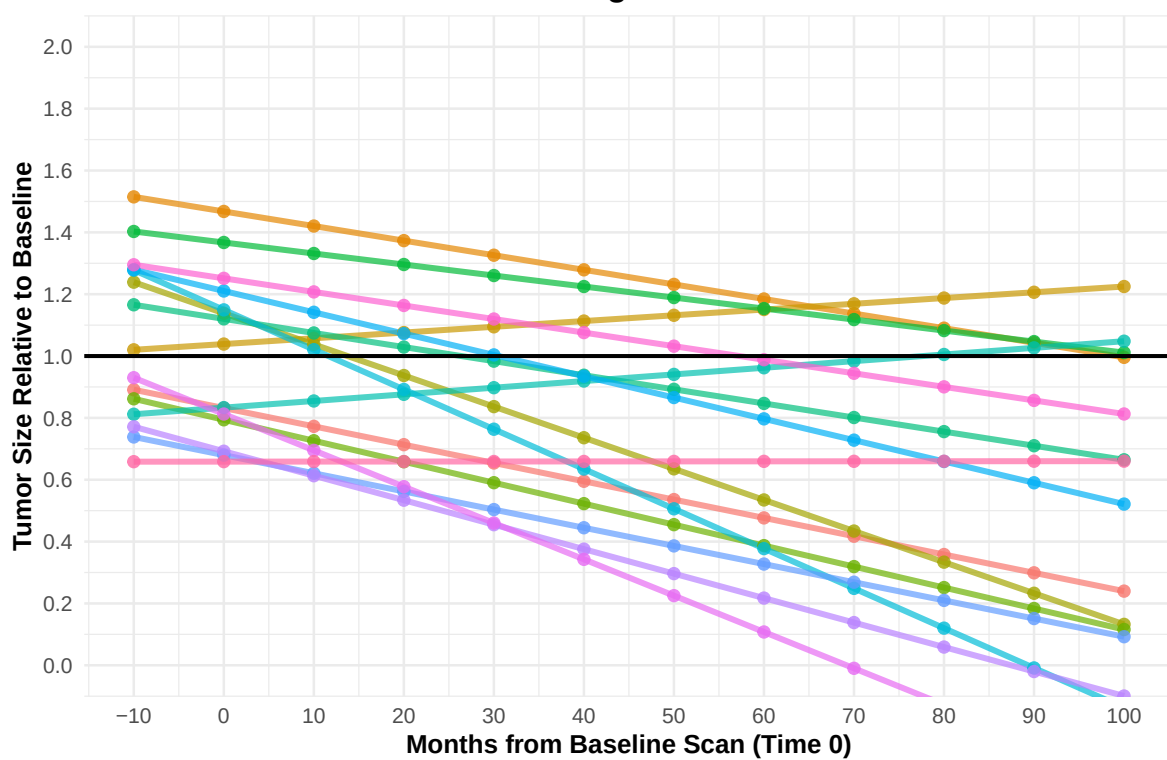
```

theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  axis.title = element_text(face = "bold", size = 11),
  legend.position = "none"
) +
coord_cartesian(xlim = c(-10, 100), ylim = c(0, 2))

```

p4

Figure 4
Relative Change in Tumor Size



In practice, this would also come from **ADTR** (longitudinal tumor measurements) with a derived `relative_size = AVAL / BASE`.

92 Figure 5 – Kaplan–Meier Survival Curve

Kaplan–Meier (KM) plots visualize **time-to-event endpoints** like PFS or OS:

- Step function for survival probability over time
- Separate curve for each treatment arm
- Optionally: confidence intervals and number-at-risk table

92.1 5.1 Create KM data

We simulate:

- **time**: event/censoring time
- **status**: 1 = event, 0 = censored
- **treatment**: Treatment A / Treatment B

```
set.seed(456)

km_data <- data.frame(
  time = c(rexp(100, 0.05), rexp(100, 0.03)),
  status = c(rbinom(100, 1, 0.7), rbinom(100, 1, 0.6)),
  treatment = rep(c("Treatment A", "Treatment B"), each = 100)
)

head(km_data)
```

	time	status	treatment
1	50.245343	1	Treatment A
2	41.407858	0	Treatment A
3	9.318211	1	Treatment A
4	4.601942	1	Treatment A

```
5 47.967306      1 Treatment A
6 16.705099      1 Treatment A
```

92.2 5.2 Fit the KM model and plot

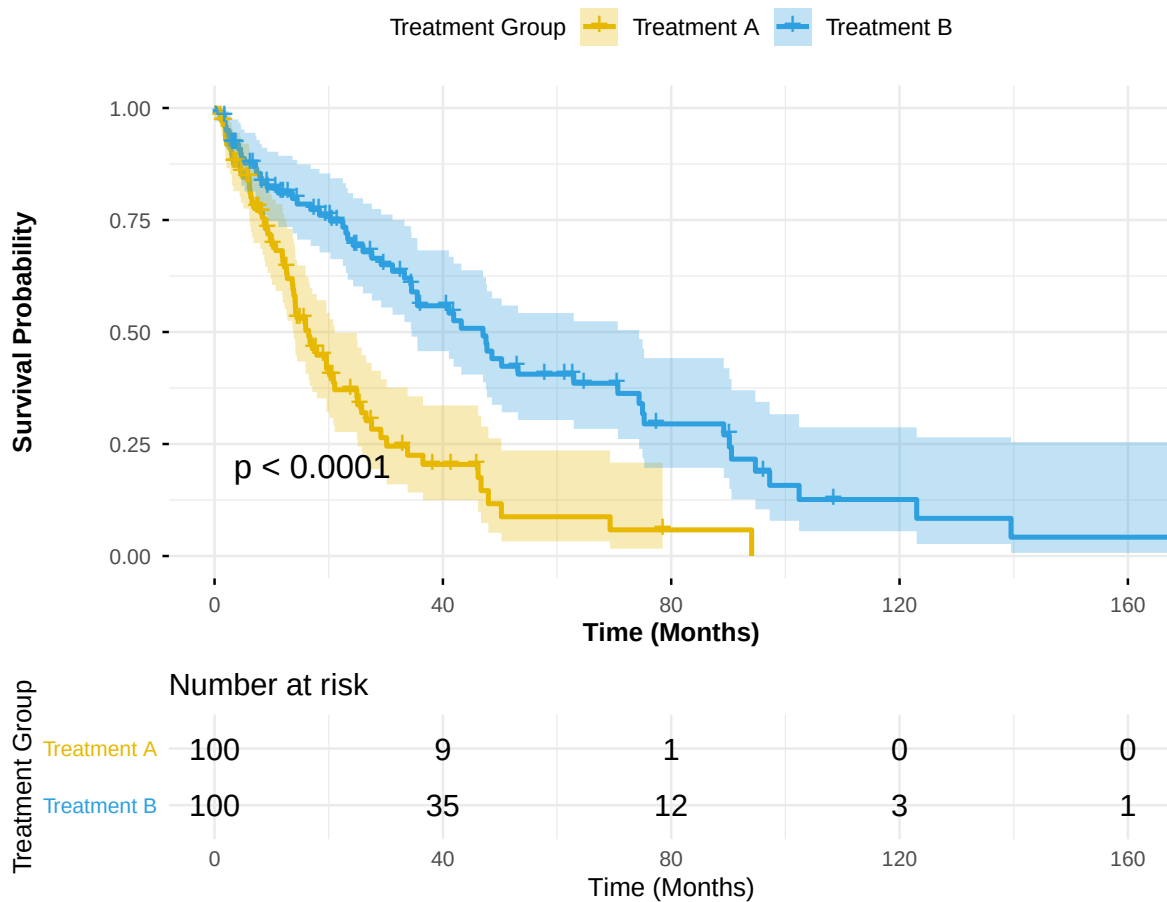
We use `survival::survfit()` and `survminer::ggsurvplot()`:

```
km_fit <- survfit(Surv(time, status) ~ treatment, data = km_data)

p5 <- ggsurvplot(
  km_fit,
  data = km_data,
  pval = TRUE,          # log-rank p-value
  conf.int = TRUE,      # confidence interval bands
  risk.table = TRUE,    # number at risk under the plot
  risk.table.height = 0.25,
  ggtheme = theme_minimal(),
  palette = c("#E7B800", "#2E9FDF"),
  title = "Figure 5
Kaplan-Meier Survival Curve",
  xlab = "Time (Months)",
  ylab = "Survival Probability",
  legend.title = "Treatment Group",
  legend.labs = c("Treatment A", "Treatment B"),
  font.main = c(14, "bold"),
  font.x = c(11, "bold"),
  font.y = c(11, "bold"),
  font.legend = c(10)
)

p5
```


Figure 5
Kaplan–Meier Survival Curve



In ADaM:

- This would typically be based on **ADTTE**, using variables like **AVAL** (time), **CNSR** (censor flag), and **TRT01A** or similar.

93 Figure 6 – Forest Plot for Subgroup Hazard Ratios

Forest plots summarize **effect estimates** (e.g. **hazard ratios**) across subgroups:

- Each row: one subgroup
- Point: HR
- Horizontal line: 95% CI
- Vertical reference line at $HR = 1$

Here we use the `forestploter` package and its example data.

93.1 6.1 Load example data and prepare

```
dt <- read.csv(system.file("extdata", "example_data.csv", package = "forestploter"))  
head(dt)
```

	Subgroup	Treatment	Placebo	est	low	hi	low_gp1	
1	All Patients	781	780	1.869694	0.13245636	3.606932	0.1507971	
2	Sex	NA	NA	NA	NA	NA	NA	
3	Male	535	548	1.449472	0.06834426	2.830600	1.9149515	
4	Female	246	232	2.275120	0.50768005	4.042560	0.6336414	
5	Age	NA	NA	NA	NA	NA	NA	
6	<65 yr	297	333	1.509242	0.67029394	2.348190	1.7679431	
	low_gp2	low_gp3	low_gp4	est_gp1	est_gp2	est_gp3	est_gp4	hi_gp1
1	0.35443249	0.3939730	1.1515801	1.181926	1.5163554	1.612725	1.949433	2.924330
2	NA	NA	NA	NA	NA	NA	NA	NA
3	0.09953409	0.3803214	0.3213258	3.266615	0.8291053	1.642294	1.607950	3.996409
4	2.57367694	1.0229365	0.3510777	1.971712	3.9719741	1.751471	1.742973	3.965617
5	NA	NA	NA	NA	NA	NA	NA	NA

6	0.57329716	0.8433183	2.1637576	2.447051	1.9990500	1.390913	3.136695	3.674451
	hi_gp2	hi_gp3	hi_gp4					
1	2.919964	3.485182	2.862447					
2	NA	NA	NA					
3	1.553593	2.573538	2.228632					
4	5.439582	3.674689	3.249342					
5	NA	NA	NA					
6	3.399146	2.346744	4.299211					

We:

- Indent subgroup labels when there is a number in **Placebo** column (for visual hierarchy).
- Replace NA with empty strings for display columns.
- Compute standard error (**se**) from CI.
- Add a blank column (" ") for the plot.
- Create a text column for **HR (95% CI)**.

```
# Indent subgroup if there is a number in the placebo column
dt$Subgroup <- ifelse(
  is.na(dt$Placebo),
  dt$Subgroup,
  paste0(" ", dt$Subgroup)
)

# NA to blank for display columns
dt$Treatment <- ifelse(is.na(dt$Treatment), "", dt$Treatment)
dt$Placebo <- ifelse(is.na(dt$Placebo), "", dt$Placebo)

# Standard error derived from CI on log scale
dt$se <- (log(dt$hi) - log(dt$est)) / 1.96

# Blank column used by forestploter for CI area
dt$` ` <- paste(rep(" ", 20), collapse = " ")

# Display column for HR (95% CI)
dt$`HR (95% CI)` <- ifelse(
  is.na(dt$se), "",
  sprintf("%.2f (%.2f to %.2f)", dt$est, dt$low, dt$hi)
)
```

```
)
```

```
head(dt)
```

```
      Subgroup Treatment Placebo      est      low      hi      low_gp1
1 All Patients      781      780 1.869694 0.13245636 3.606932 0.1507971
2      Sex              NA      NA      NA      NA
3      Male      535      548 1.449472 0.06834426 2.830600 1.9149515
4      Female      246      232 2.275120 0.50768005 4.042560 0.6336414
5      Age              NA      NA      NA      NA
6      <65 yr      297      333 1.509242 0.67029394 2.348190 1.7679431
      low_gp2 low_gp3 low_gp4 est_gp1 est_gp2 est_gp3 est_gp4 hi_gp1
1 0.35443249 0.3939730 1.1515801 1.181926 1.5163554 1.612725 1.949433 2.924330
2      NA      NA      NA      NA      NA      NA      NA      NA
3 0.09953409 0.3803214 0.3213258 3.266615 0.8291053 1.642294 1.607950 3.996409
4 2.57367694 1.0229365 0.3510777 1.971712 3.9719741 1.751471 1.742973 3.965617
5      NA      NA      NA      NA      NA      NA      NA      NA
6 0.57329716 0.8433183 2.1637576 2.447051 1.9990500 1.390913 3.136695 3.674451
      hi_gp2 hi_gp3 hi_gp4      se
1 2.919964 3.485182 2.862447 0.3352463
2      NA      NA      NA      NA
3 1.553593 2.573538 2.228632 0.3414741
4 5.439582 3.674689 3.249342 0.2932884
5      NA      NA      NA      NA
6 3.399146 2.346744 4.299211 0.2255292
      HR (95% CI)
1 1.87 (0.13 to 3.61)
2
3 1.45 (0.07 to 2.83)
4 2.28 (0.51 to 4.04)
5
6 1.51 (0.67 to 2.35)
```

93.2 6.2 Define forest theme and build the plot

We define a theme (fonts, refine color, arrow labels) and then call `forest()`:

```
tm <- forest_theme(
  base_size = 10,
  refline_col = "red",
```

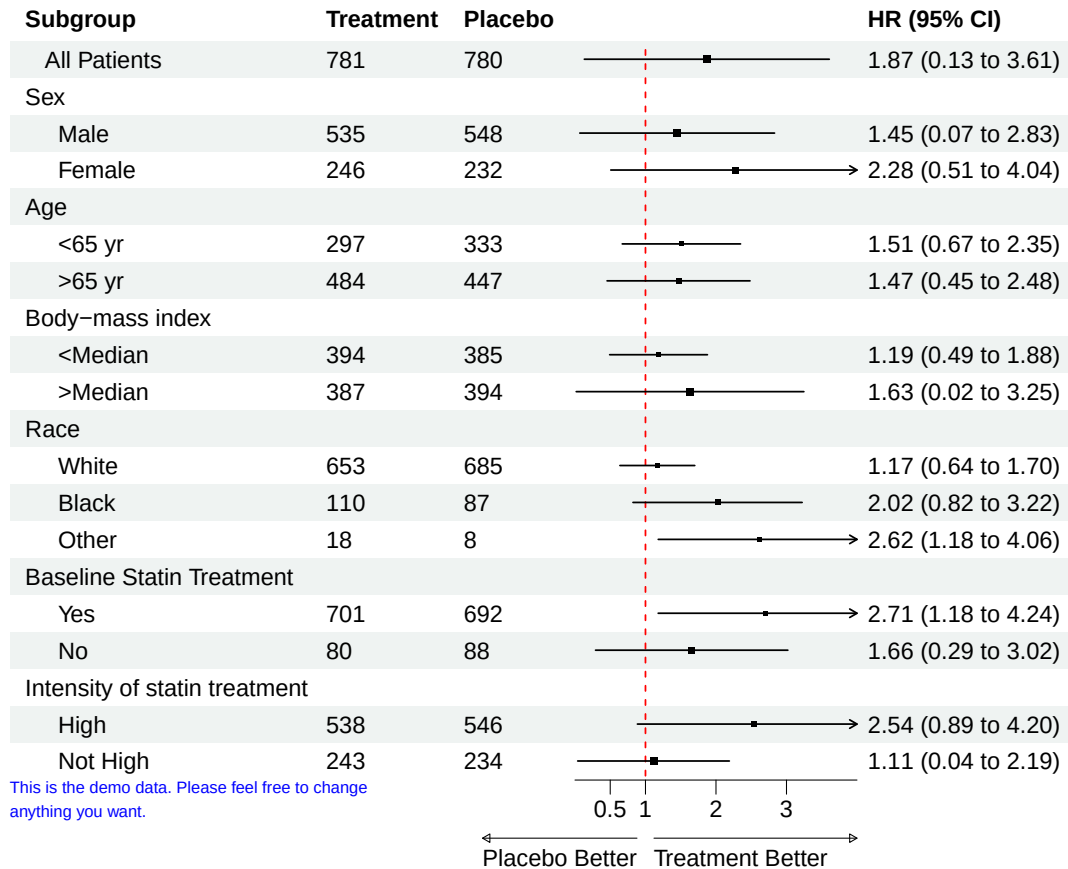
```

    arrow_type = "closed",
    footnote_gp = gpar(col = "blue", cex = 0.6)
)

p <- forest(
  dt[, c(1:3, 20:21)],
  est      = dt$est,
  lower    = dt$low,
  upper    = dt$hi,
  sizes    = dt$se,
  ci_column = 4,
  ref_line  = 1,
  arrow_lab = c("Placebo Better", "Treatment Better"),
  xlim      = c(0, 4),
  ticks_at  = c(0.5, 1, 2, 3),
  footnote  = "This is the demo data. Please feel free to change
anything you want.",
  theme     = tm
)

plot(p)

```



Interpretation:

- Values < 1 favor **treatment**, values > 1 favor **placebo** (depending on how you define HR).
- The arrow labels at the bottom make this direction explicit.

In real analyses, the columns **est**, **low**, **hi** would be generated from Cox models or other regressions per subgroup.

94 Summary and Next Steps

In this tutorial, you:

- Learned the **ggplot2 grammar of graphics** using **ADSL, ADLB, and ADTTE** style data
- Built common clinical plots:
 - Subject counts by arm
 - Histograms/boxplots of demographics
 - Longitudinal lab plots with summary statistics
 - Simple KM curves from ADTTE
- Customized scales, themes, coordinates, and annotations
- Combined **dplyr + ggplot2** for tidy ADaM workflows
- Wrote a small **reusable plotting function** tailored to ADaM

From here, you can:

- Swap the simulated data with your real **ADaM datasets**
- Extend functions for specific TLFs (e.g., safety labs, efficacy endpoints)
- Integrate these plots into **R Markdown/Quarto CSRs** or **Shiny dashboards** for interactive review.

Happy plotting with ggplot2 and ADaM!

95 Capstone: End-to-End Mini Workflow

This chapter ties everything together: **read data** → **derive ADSL** → **produce TLFs** → **render a report**.

95.1 Parameters

```
# You could parametrize paths via YAML; here we keep inline defaults.
dm_path <- "data/sdtm/dm.sas7bdat"
ex_path <- "data/sdtm/ex.sas7bdat"
```

95.2 1) Read (or Synthesize) SDTM

```
library(haven); library(dplyr); library(lubridate)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:60),
    ARM = sample(c("Placebo", "Active"), 60, replace=TRUE),
    AGE = round(rnorm(60, 60, 8)),
    SEX = sample(c("M", "F"), 60, replace=TRUE),
    RANDDT = as.Date("2025-01-15") + sample(0:40, 60, replace=TRUE)
  )
}
if (file.exists(ex_path)) {
  ex <- read_sas(ex_path)
} else {
  ex <- tibble::tibble(
    USUBJID = dm$USUBJID,
    EXSTDTC = dm$RANDDT + sample(0:3, nrow(dm), replace=TRUE)
  )
}
```

95.3 2) Derive ADSL (Minimal Demo)

```
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  mutate(
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo", "Active"))),
    TRT01A = TRT01P,
    TRT01AN = TRT01PN,
    SAFFL = "Y",          # demo only; define rules in real life
    FASFL = "Y"
  ) |>
  dplyr::select(STUDYID.x, USUBJID, TRT01P, TRT01PN, TRT01A, TRT01AN, AGE, SEX, EXSTDTC, SAFI
```

[				
Table	1.	Baseline	by	Treatment
Table	1.	Baseline	by	Treatment
Description of Planned Arm	N	mean_age	sd_age	pct_female
Placebo	226	75.04867	8.503715	60.61947
Screen Failure	52	75.09615	9.699928	69.23077
Xanomeline High Dose	184	74.01087	7.939656	48.36957
Xanomeline Low Dose	181	75.29834	8.277778	60.77348

95.4 3) TLFs

```
library(gt); library(ggplot2); library(survival)
```

```
tbl1 <- adsl |>
  group_by(TRT01P) |>
  summarise(N=n(),
            mean_age = mean(AGE), sd_age = sd(AGE),
            pct_female = mean(SEX=="F")*100)
tbl1_gt <- gt(tbl1) |> tab_header(title="Table 1. Baseline by Treatment")
tbl1_gt
```

```
set.seed(123)
adsl$time <- rexp(nrow(adsl), rate=ifelse(adsl$TRT01P=="Active", 0.08, 0.1))
adsl$status <- rbinom(nrow(adsl), 1, 0.7)
fit <- survfit(Surv(time, status) ~ TRT01P, data=adsl)
# reuse plotting function from prior chapter
ggsurv <- function(fit) {
  ss <- summary(fit)
  dd <- data.frame(time=ss$time, surv=ss$surv, strata=rep(names(fit$strata), fit$strata))
  ggplot(dd, aes(x=time, y=surv, linetype=strata)) + geom_step() + theme_minimal() +
    labs(title="KM Curve (Toy)", x="Time", y="Survival", linetype="Treatment")
}
#ggsurv(fit)
```

95.5 4) Save Outputs

```
# Example: Save Table 1 as PNG  
#gtsave(tbl1_gt, "tlf-table1.png")
```

Challenge: Convert this chapter into a **parameterized report** (e.g., treatment subset or different cohort) and render multiple outputs.

96 Appendix: Tips, Profiles, .libPaths

96.1 Useful Profiles

Create ~/.Rprofile to set options (be careful on shared systems):

```
options(  
  repos = c(CRAN = "https://cloud.r-project.org"),  
  scipen = 999  
)
```

96.2 Custom Library Paths

```
# In .Rprofile or project-level .Rprofile  
.libPaths(c("/path/to/Rlibs", .libPaths()))
```

96.3 Format vs formatC (quick recap)

```
x <- c(123.456, 0.00123456)  
format(x, digits = 4)
```

```
[1] "1.235e+02" "1.235e-03"
```

```
format(x, nsmall = 2)
```

```
[1] "1.23456e+02" "1.23456e-03"
```

```
formatC(x, digits = 3, format = "f")
```

```
[1] "123.456" "0.001"
```

96.4 POSIXct vs POSIXlt

- **POSIXct**: seconds since epoch (numeric), compact, fast.
- **POSIXlt**: list-like with components (year, mon, mday...), easier to extract parts.

96.5 Recommended Packages

- tidyverse, lubridate, janitor, gt, gtsummary, survival, broom, here.
- Pharma/CDISC: admiral, tlf/tern, pharmaverse meta-packages (explore as you grow).

96.6 Short Glossary

- **SDTM**: Study Data Tabulation Model (FDA submission standard for raw domains).
- **ADaM**: Analysis Data Model (derived analysis-ready datasets).
- **TLF**: Tables, Listings, Figures for reporting.